

DAZN Fantasy League

Simulación de Desarrollo de Aplicación Web Profesional

Desarrollo de Aplicaciones Web (DAW 2)

AUTORES:

Jon Blanco
Pablo Rivero
Alejandro Sainz

C.F.P. Decroly

NEXORA

PROYECTO POR NEXORA · 2026

Índice General

1. INTRODUCCIÓN 9
1.1 Contexto del proyecto 9
1.1.1 Alcance 10
1.1.2 Limitaciones 10
2. NEXORA 10
2.1 Quiénes somos 10
2.2 Misión, visión y valores 11
2.3 Estructura organizativa 11
2.4 Situación actual y posicionamiento 12
3. ANÁLISIS DE ENTORNO Y COMPETENCIA 13
3.1 Estado del sector 13
3.2 Análisis de empresas competidoras 15
Evenbytes 15
CIC Consulting Informático 16
Plain Concepts 17
Inetum 18
3.3 Competencia Directa: Mondocore (Futmondo) 19
3.4 Representación visual de los competidores 20
3.5 Conclusión del análisis 20
3.6 Análisis de productos competidores 21
3.6.1 Comunio 21
3.6.2 Biwenger 23
3.6.3 Movistar Fantasy Marca 24
4. ANÁLISIS ESTRATÉGICO 26
4.1 PUNTOS A NUESTRO FAVOR 26
4.2 PUNTOS EN NUESTRA CONTRA 26
4.3 DAFO y CAME 27
5. PROPUESTA DEL PROYECTO 28
5.1 Descripción general de DAZN Fantasy League 28
5.2 Propuesta de valor 28
Diseño y experiencia modernizada: 28
Gamificación: 28

Flexibilidad: 29
Transparencia: 29
6. PLANIFICACIÓN GENERAL DEL PROYECTO 29
6.1 Metodología de Trabajo 29
6.2 Fases del proyecto 30
6.3 Organización de los equipos 30
6.4 Visión general de los Sprints 31
6.5 Fases de desarrollo 32
7. ESTIMACIÓN Y CÁLCULO DE COSTES DEL PROYECTO 34
7.1 Costes de personal 34
7.2 Cotizaciones y cargas sociales 36
7.3 Gastos administrativos 37
7.4 Equipos Informáticos 39
7.5 Gastos derivados del uso de herramientas digitales 40
7.5.1 Listado de Tecnologías 41
7.6 Otros gastos propios de la empresa 41
7.7 Otras consideraciones 42
7.8 Cálculos totales 43
7.9 Determinación del Precio Final de Venta (PVP) 43
7.9.1 Beneficio Industrial y Margen de Contingencia 44
7.9.2 Cálculo de la Base imponible 44
7.9.3 Impuesto sobre el Valor Añadido (IVA) 45
8. ESTRUCTURA DEL PROYECTO 45
8.1 Arquitectura y estructura. 45
8.2 Capa de Presentación – El Frontend 47
8.3 Capa intermedia – El Proveedor – Backend 48
8.3 Capa de datos – La persistencia – SGBD 49
8.4 Otras tecnologías que se tuvieron en cuenta 50
8.4.1 JAVA + SPRINGBOOT 50
8.4.2 MYSQL 51
8.4.3 React y Vue.js 51
9. DISEÑO DE WIREFRAMES, MOCKUPS Y UI 53
10. SISTEMA DE IDENTIDAD VISUAL Y BRAND GUIDELINES 61
10.1 Misión y visión 61
10.2 Logotipo 62
10.3 Colores 63

10.4 Tipografía 65
11. DESARROLLO DE LA APLICACIÓN 66
11.1 La base de datos 66
11.1.1 EL DESARROLLO 67
11.1.2 Evolución de nuestro modelo. 71
Tabla usuarios 71
Tabla ligas 74
11.2 El backend 80
11.3 El frontend 90
11.5 Arquitectura de seguridad 100
11.4 Demostración de funcionalidad 111
11.4.1 Registro y Login 111
11.4.2 Página principal para usuarios registrados 113
11.4.3 Gestión de ligas 114
11.4.4 Sorteo de jugadores y de alineación. 115
12 DESPLIEGUE DE LA APLICACIÓN 118
13. FUTURAS ITERACIONES. 127
12.1 Próximas Iteraciones y Escalabilidad del Sistema 127
13 Conclusiones 128
Bibliografía 129

Índice de Figuras

Figura 1 Esquema simplificado de organización departamental 12
Figura 2 Logotipo oficial de Evenbytes. 15
Figura 3 Logotipo oficial de CIC Consulting Informático. 16
Figura 4 Logotipo oficial de Plain Concepts. 17
Figura 5 Logotipo oficial de Inetum. 18
Figura 6 Logotipo de la aplicación Futmondo, propiedad de Mondocore. 19
Figura 7 Representación visual del ecosistema de competidores. 20
Figura 8 Logo de Comunio 21
Figura 9 Interfaz de Comunio 22
Figura 10 Logo de Biwenger 23
Figura 11 Interfaz móvil de Biwenger 23
Figura 12 Logo de Liga Fantástica Marca 24
Figura 13 Interfaz Liga Fantastica Marca 24
Figura 14 Stats Performs 25
Figura 15 DAFO y CAME 27
Figura 16 Diagrama de Gantt orientativo de la organización del proyecto 32
Figura 17 Lenovo ThinkPad 14 Gen 6 39
Figura 18 Logotipo de Angular 47
Figura 19 Logotipos de Node.js y Typescript 48
Figura 20 Logotipo de PostgreSQL 49
Figura 21 Logotipo de SpringBoot 50
Figura 22 Logotipo de MySql 51
Figura 23 Logotipos de React y Vue.js 51
Figura 24 Primeros bocetos de la aplicación 54
Figura 25 Wireframes y Mock-Ups 55
Figura 26 Logotipo de DAZN Fantasy League 62
Figura 27 Primer boceto del logotipo 62
Figura 28 Versión final del logotipo de la aplicación 63
Figura 29 Paleta de colores corporativos y contraste entre colores y tipografía (Aa). 64
Figura 30 Tipografía Segoe UI 65
Figura 31 Primer Diagrama Entidad Relación. Un boceto de lo que sería después. 67
Figura 32 Fragmento de nuestro Script. 68

Figura 33 Estructura básica del proyecto del Backend 80
Figura 34 Ejemplo de Clase o Model. 81
Figura 35 Controller en Node.js 82
Figura 36 Estructura y código de un Service. 83
Figura 37 Código de otro servicio para realizar diferentes funciones. 83
Figura 38 Ejemplo de Middleware 84
Figura 39 Archivos de Rutas 85
Figura 40 Interfaz de Swagger-UI en nuestro navegador. 86
Figura 41 Creación de usuario mediante Swagger-UI 87
Figura 42 Usuario Creado 88
Figura 43 Login a través de Swagger 88
Figura 44 Estructura básica del proyecto. 90
Figura 45 Estructura extendida del proyecto en Angular 91
Figura 46 Interfaces o modelos 92
Figura 47 Guarda de Administrador 93
Figura 48 Guarda Usuario Regular 93
Figura 49 Código de un interceptor. 94
Figura 50 Ejemplo de servicio en Angular 94
Figura 51 Ejemplo de servicio de administrador. 95
Figura 52 Fragmento de código html de la landing page 95
Figura 53 Fragmento de código sección Alineaciones 96
Figura 54 Google Lighthouse 97
Figura 55 Google Lighthouse Performance 98
Figura 56 Google Lighthouse Accesibility 99
Figura 57 Página de Bienvenida o Landing 111
Figura 58 Formulario de registro y login 112
Figura 59 Logeandose y Registrandose 113
Figura 60 Landing Page Usuarios Registrados 113
Figura 61 Creando nueva liga. 114
Figura 62 Rellenando los datos requeridos para crear una liga 115
Figura 63 Liga creada. 115
Figura 64 Página de plantilla y alineación. 116
Figura 65 Mercado de Jugadores 117
Figura 66 Página principal de Vercel 118
Figura 67 Selección de cuenta de GitHub en Vercel 119
Figura 68 Selección del repositorio del frontend 119

Figura 69 Importación del proyecto en Vercel 120
Figura 70 Configuración del proyecto y variables de entorno 120
Figura 71 Página principal de Railway 121
Figura 72 Selección de PostgreSQL como base de datos 122
Figura 73 Variables de entorno de PostgreSQL 122
Figura 74 Configuración adicional de PostgreSQL 123
Figura 75 Despliegue completado de PostgreSQL 123
Figura 76 Selección del repositorio del backend 124
Figura 77 Proceso de despliegue del backend 124
Figura 78 Aplicación desplegada y accesible desde Internet 125
Figura 79 Acceso autenticado a la aplicación 126

Índice de Tablas

Tabla 1 Distribución de salarios brutos anuales y mensuales por puesto 34
Tabla 2 Coste salarial bruto por semanas de trabajo aportadas al proyecto 35
Tabla 3 Coste de las cotizaciones totales de los trabajadores aportadas al proyecto 36
Tabla 4 Coste salarial personal administrativo 1 37
Tabla 5 Coste salarial personal administrativo 2 38
Tabla 6 Amortización equipos informáticos 40
Tabla 7 Herramientas y tecnologías utilizadas en el desarrollo del proyecto. Iconos obtenidos de (Simple Icons, s. f.) y (Iconify, s. f.) 41
Tabla 8 Resumen de gastos 43
Tabla 9 Cálculo de la base imponible 44
Tabla 10 Cálculo del precio total más el IVA 45

FINALIDAD DE ESTE DOCUMENTO

Jon Blanco **Pablo Rivero** **Alejandro Sainz**

Este documento presenta el Trabajo de Fin de Ciclo del ciclo formativo de Desarrollo de Aplicaciones Web (DAW), realizado en el C.F.P. Decroly durante el curso 2025-2026. A lo largo de los siguientes apartados se exponen los objetivos, el proceso de desarrollo, las tecnologías empleadas y los resultados obtenidos en el proyecto, que representa la fase final de nuestra formación académica.

1. INTRODUCCIÓN

A lo largo de este documento se desarrollará una **explicación técnica** y contextual del **proyecto DAZN Fantasy**, comenzando con un **contexto de la empresa** desarrolladora **Nexora**, empresa encargada de una **aplicación de liga fantástica** solicitada por la empresa **DAZN**.

En ella se gestionarán equipos que puntuarán en función de los partidos de La Liga de esa semana. Busca ser una competición entre amigos para aquellos que disfrutan el fútbol día a día.

Paso a paso trataremos de dar visibilidad a nuestra metodología de trabajo, a los pasos seguidos durante el desarrollo y en la mayor medida posible, a los porqués de nuestras acciones y elecciones durante el mismo.

1.1 Contexto del proyecto

La empresa **DAZN**, actualmente una de las grandes distribuidoras de entretenimiento y difusión de eventos deportivos, se ha puesto en contacto con Nexora para implementar un plan de **diseño** y **desarrollo** de una **Fantasy League** que implemente la competición española de La Liga.

Con el objetivo de competir con otras empresas del sector, la aplicación contará con un diseño capaz de escalar a futuro para poder implementar más ligas y competiciones, tanto españolas como extranjeras.

1.1.1 Alcance

El **objetivo principal** en el desarrollo de esta aplicación es dotar a DAZN de una **plataforma de liga Fantasy** que pueda competir semanalmente con otras empresas patrocinadoras de este tipo de aplicaciones, intentando obtener el mayor número de usuarios posible.

Dado los resultados de los **estudios de aplicaciones** de tipo «Fantasy» obtenidos de diferentes análisis (Global Market Insights, 2023) , en los cuales se vislumbra un **gran crecimiento del sector**, DAZN ha decidido apostar por este tipo de plataforma.

La naturaleza de este tipo de aplicaciones encaja perfectamente con la marca y el sector de la compañía. Para este cometido, contactan con nuestra empresa para llevar a cabo el desarrollo del proyecto.

1.1.2 Limitaciones

Se debe tener en cuenta que Nexora es una **empresa pequeña** dentro del sector de las consultorías, con las limitaciones que esto puede provocar.

Aun llevando años en el sector, la falta de experiencia específica en el desarrollo de este tipo de productos y la alta competencia existente pueden suponer un reto importante.

2. NEXORA

2.1 Quiénes somos

Nexora es una **empresa española** dedicada al **desarrollo de software** y soluciones digitales para empresas.

Con sede en **Cantabria** y alrededor de **150 empleados**, la compañía se encuentra en una etapa de consolidación tras varios años de crecimiento.

Su enfoque se centra en ofrecer **servicios personalizados de desarrollo de aplicaciones web y móviles**, así como soluciones digitales adaptadas a las necesidades de cada cliente.

2.2 Misión, visión y valores

Nuestra compañía es una **entidad joven** que busca **establecerse** y asentarse en el sector tecnológico desarrollando productos y soluciones personalizadas y de **calidad**.

Esperamos con el tiempo situarnos a la altura de las empresas referentes del sector.

«Ver Manual de Marca de Nexora que se adjunta con este documento.»

2.3 Estructura organizativa

En sus comienzos, como muchas otras empresas tecnológicas, Nexora comenzó como una pequeña startup desarrollando soluciones de software orientadas a resolver problemas empresariales.

Desde un principio abrazó metodologías de trabajo **Agile**.

Para comprender mejor este tipo de organizaciones, se han tomado como referencia diferentes recursos y modelos organizativos del sector tecnológico (ASEST, s. f.).

Tras un tiempo de **crecimiento exponencial**, Nexora se ha consolidado como una **empresa sólida a nivel nacional**. Esto ha quedado demostrado en diferentes estudios a nivel nacional como el desarrollado por (AEC Consultoras, 2024)

Esto ha provocado una evolución significativa en su organigrama empresarial.

Actualmente, la empresa cuenta con:

- Director Ejecutivo (CEO)
- Director de Operaciones
- Director de Tecnologías e Innovación
- Director de Recursos Humanos

Por debajo de estos cargos, la organización de técnicos y desarrolladores es flexible y adaptable según las necesidades de cada proyecto.

El tamaño y composición de los equipos varía según la envergadura del trabajo asignado.

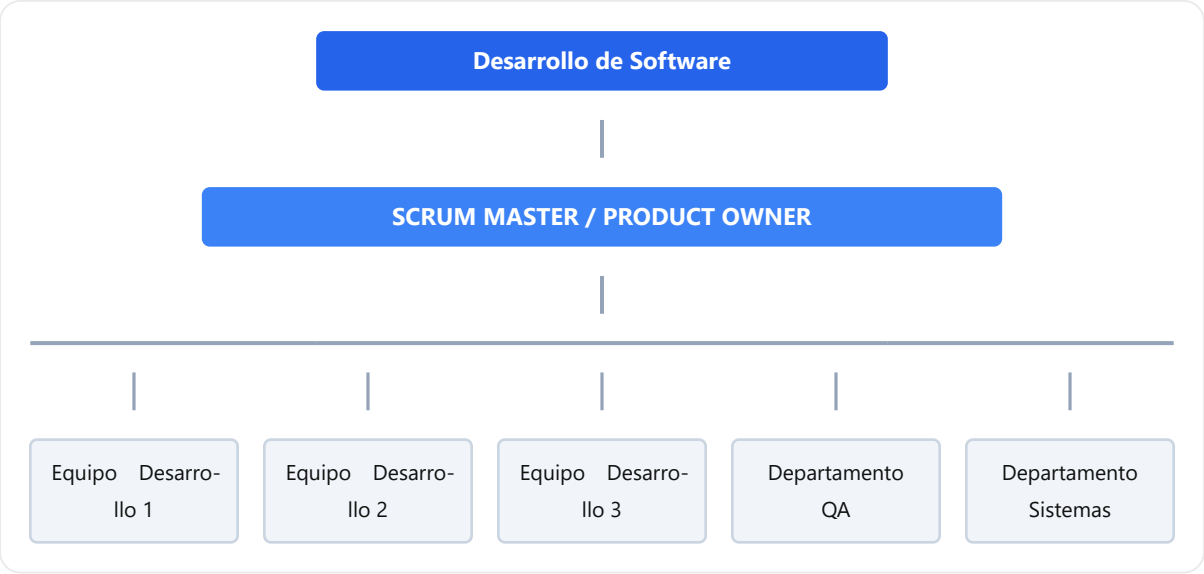


Figura 1 — Esquema simplificado de organización departamental

2.4 Situación actual y posicionamiento

La empresa Nexora cuenta tanto con **clientes y partners** del sector privado como con entidades públicas.

La compañía ofrece soluciones tecnológicas diseñadas específicamente para las necesidades de cada cliente.

Dado que todavía es una **empresa en auge**, no alcanza las cifras de facturación de las grandes compañías tecnológicas nacionales comparándola con otras reflejadas en diferentes estudios (Fundación VASS, 2024) .

Sin embargo, según sus últimos informes, la facturación por empleado ronda los **90.000 € anuales**, alcanzando aproximadamente unos **13,5 millones de euros anuales**.

3. ANÁLISIS DE ENTORNO Y COMPETENCIA

En este apartado se realiza un análisis del entorno en el que se desarrolla el proyecto, centrándose en el estudio del sector tecnológico y en las principales empresas que compiten directa o indirectamente con la propuesta de DAZN Fantasy.

El objetivo de este análisis es comprender el contexto actual del mercado, identificar los principales actores del sector y evaluar su nivel de influencia dentro del ámbito del desarrollo de soluciones digitales y plataformas de entretenimiento deportivo.

De esta forma, se podrá situar el proyecto dentro de un marco realista, valorando tanto la competencia existente como las oportunidades de posicionamiento que ofrece el mercado actual, especialmente en el ámbito de las aplicaciones tipo Fantasy y las plataformas deportivas digitales.

3.1 Estado del sector

El sector de las **fantasy leagues** y los deportes virtuales ha experimentado un crecimiento significativo a nivel global en los últimos años, impulsado principalmente por la digitalización del consumo deportivo y la expansión de las plataformas móviles.

Según distintos informes del sector, el mercado global de fantasy sports se estimó en más de 24.800 millones de dólares en 2024, con previsiones de crecimiento hasta superar los 56.000 millones de dólares en 2030, con tasas de crecimiento anual cercanas al 15%.

Este crecimiento está directamente relacionado con factores como:

- La expansión del acceso a internet de alta velocidad
- El aumento del uso de smartphones
- La integración de sistemas de gamificación en plataformas deportivas
- El crecimiento del consumo de contenido deportivo en streaming

En el ámbito deportivo, el fútbol es el segmento dominante dentro de las fantasy leagues, especialmente en Europa, donde destacan plataformas como la Fantasy Premier League o LaLiga Fantasy, que han consolidado el modelo como una forma de interacción social entre aficionados.

Además, la tendencia actual del sector apunta hacia la incorporación de tecnologías como la inteligencia artificial, el análisis predictivo y la personalización de experiencias de

usuario, lo que está transformando las fantasy leagues en plataformas más complejas y competitivas.

En este contexto, el desarrollo de nuevas plataformas como DAZN Fantasy se sitúa en un entorno de alto crecimiento y con oportunidades claras de innovación.

Para poder enfrentarnos a este reto con las suficientes garantías debemos de realizar un estudio del sector, la competencia y otros productos similares que nos permitan ser lo suficientemente realistas con nuestras aspiraciones y expectativas.

De entre aquellas que podemos encontrar a nuestro alrededor, seleccionaremos aquellas de las cuales creamos que podemos aprender más sobre el estado actual de las empresas de la industria Tech. No se nos debe olvidar que también debemos de buscar entre aquellas cuyos desarrollos van más a la par de la naturaleza de este proyecto, lo que nos permitirá tener una referencia exacta, o por lo menos muy aproximada, de la situación en la que nos encontramos y de los retos que se nos van a plantear a partir de ahora.

3.2 Análisis de empresas competidoras

A continuación, se detalla el perfil de las principales empresas del sector...

Evenbytes



Figura 2 — Logotipo oficial de Evenbytes.

Experiencia e Historia

Consultora especializada que cuenta con una trayectoria sólida de más de 15 años de experiencia en el sector tecnológico.

Línea de Negocio y Trabajo Principal

Disponen de productos propios especializados en entornos corporativos de Google Drive y Shared Drives, destacando sus plataformas de trabajo *METADRIVE* y *DRIVEWATCHER*.

Clientes Importantes

Cuentan con una cartera de clientes de gran envergadura y relevancia en el mercado, entre los que destacan corporaciones como **B/S/H**, **Ferrovial** e **IMEM**.

Datos Adicionales y Certificaciones

A nivel estratégico, la empresa ostenta la certificación oficial de *Google Partner* y ha sido galardonada con el premio *PYME Innovadora*, lo que demuestra su compromiso con el desarrollo de vanguardia.

Infraestructura y Datos Laborales

La empresa cuenta con un tamaño de plantilla compacto, situándose en un rango de entre 20 y 25 empleados.

Puntos a Favor (Fortalezas)

- Demuestran un perfil corporativo altamente innovador.
- Cuentan con un equipo técnico experto y muy especializado en su nicho de mercado.

Puntos en Contra (Debilidades)

- Sus productos actuales y experiencia en desarrollos van enfocados a tecnologías de almacenamiento y gestión Cloud.

(Evenbytes, s. f.)

CIC Consulting Informático



Figura 3 — Logotipo oficial de CIC Consulting Informático.

Experiencia e Historia

Empresa de ingeniería y consultoría que cuenta con aproximadamente 30 años de experiencia consolidada en el sector tecnológico.

Línea de Negocio y Trabajo Principal

Su actividad principal se centra en la consultoría tecnológica, la ingeniería de sistemas y el desarrollo de software a medida.

Clientes Importantes

Disponen de un amplio programa de alianzas estratégicas y *partners*, destacando su colaboración oficial con corporaciones como **Microsoft**.

Tecnologías Utilizadas

- **Backend:** Principalmente Java, .NET y Python.
- **Bases de Datos:** Experiencia tanto en entornos SQL como NoSQL.
- **Frontend:** Dominio robusto de Angular y conocimientos valorables en React y Vue.js.

Infraestructura y Datos Laborales

Cuenta con una plantilla de más de 300 empleados organizados en células de desarrollo. El salario de entrada estimado para perfiles junior ronda los 22.000€ anuales, según portales de empleo.

Datos Adicionales

Recientemente fue galardonada con el premio a la **tercera mejor empresa para trabajar en España**, lo que refleja un alto índice de motivación en sus equipos técnicos.

Puntos a Favor (Fortalezas)

- Trayectoria muy amplia y consolidada en el sector.
- Equipos de gran tamaño con capacidad para un desarrollo rápido y adopción de tecnologías variadas.

Puntos en Contra (Debilidades)

- Su orientación y cartera de proyectos actuales se enfocan con fuerza hacia la Industria 4.0, Big Data y analítica de datos. Esto reduce su interés o prioridad por soluciones comerciales de entretenimiento deportivo B2C.

(CIC Consulting Informático, s. f.)

Plain Concepts



Figura 4 — Logotipo oficial de Plain Concepts.

Experiencia e Historia

Multinacional tecnológica de vanguardia con casi 20 años de experiencia en el desarrollo de soluciones de software avanzadas.

Línea de Negocio y Trabajo Principal

Especializados en la automatización de procesos mediante Inteligencia Artificial (IA) y en el desarrollo de aplicaciones empresariales de alta disponibilidad.

Tecnologías Utilizadas

- **Data & Cloud:** Enfoque en Big Data usando Python, Azure, .NET, Scala y Spark.
- **Bases de Datos:** Dominio de entornos SQL y NoSQL.
- **Frontend:** Uso intensivo de C# y de prácticamente todos los frameworks del mercado, a excepción de Astro.

Infraestructura y Datos Laborales

Cuenta con más de 700 empleados a nivel global, otorgándoles una gran capacidad de desarrollo, adaptación y conectividad internacional.

Muestran una media salarial competitiva en el sector: unos 40.500€ promedio para perfiles técnicos generales y hasta 54.000€ para desarrolladores Frontend senior.

Puntos a Favor (Fortalezas)

- Modelo de empresa muy moderno, ágil e innovador.
- Abanico tecnológico sumamente amplio para adaptarse a cualquier tipo de proyecto.
- Gran capacidad y músculo de desarrollo simultáneo.

Puntos en Contra (Debilidades)

- La naturaleza de los proyectos que publicitan indica una clara especialización en infraestructura empresarial e IA corporativa, alejándose por completo de productos de gamificación o aplicaciones de entretenimiento deportivo.

(Plain Concepts, s. f.)

Inetum



Figura 5 — Logotipo oficial de Inetum.

Experiencia e Historia

Gran corporación tecnológica internacional con una sólida y dilatada trayectoria de experiencia en el sector de los servicios digitales.

Línea de Negocio y Trabajo Principal

Disponen de divisiones integrales de Consultoría Tecnológica, desarrollo de software y gestión de infraestructuras de hardware. Están especializados en soluciones empresariales complejas, Industria 4.0 y proyectos a gran escala tanto para el sector privado como para Administraciones Públicas.

Tecnologías Utilizadas

Especialización en arquitecturas empresariales robustas y tradicionales, utilizando principalmente: SpringBoot, Java, C# y Angular.

Infraestructura y Datos Laborales

Cuentan con un músculo operativo colosal de más de 27.000 consultores y especialistas a nivel global.

El salario medio para un desarrollador en España se sitúa en torno a los 35.000€ anuales, registrando variaciones significativas en función de la tecnología y especialización del puesto.

Puntos a Favor (Fortalezas)

- Infraestructura corporativa masiva y una variedad tecnológica muy alta.
- Respaldo de muchos años de experiencia en integraciones críticas y despliegues complejos.

Puntos en Contra (Debilidades)

- Su modelo de negocio generalista y corporativo está completamente orientado en otra dirección (B2B/Administración Pública).
- Debido a sus costes estructurales, el presupuesto por desarrollar una aplicación a medida de este tipo sería, con diferencia, el más elevado de la comparativa.

(Inetum, s. f.)

3.3 Competencia Directa: Mondocore (Futmondo)



Figura 6 — Logotipo de la aplicación Futmondo, propiedad de Mondocore.

El principal competidor directo del proyecto DAZN Fantasy es la aplicación **Futmondo** (Futmondo, s. f.), desarrollada por la empresa Mondocore.

Futmondo es una plataforma de fantasy sports centrada principalmente en competiciones de fútbol, permitiendo a los usuarios crear equipos virtuales basados en jugadores reales de distintas ligas. Su modelo de negocio se basa en la gamificación del seguimiento deportivo, ofreciendo una experiencia interactiva similar a la propuesta de este proyecto.

A diferencia de las grandes consultoras analizadas en el apartado anterior, Futmondo representa una competencia directa a nivel funcional, ya que se trata de un producto final ya implantado en el mercado y dirigido al mismo tipo de usuario objetivo.

Entre sus principales fortalezas destacan:

- **Experiencia previa:** En el desarrollo de una plataforma fantasy totalmente funcional.
- **Audiencia:** Comunidad de usuarios ya establecida y fidelizada.
- **Especialización:** Producto orientado específicamente al entretenimiento deportivo.

Sin embargo, también presenta ciertas limitaciones:

- **Escalabilidad:** Limitada frente a grandes plataformas tecnológicas.
- **Integración:** Menor capacidad de conexión con servicios externos o broadcasters.
- **Evolución:** Menor capacidad de evolución tecnológica comparado con las grandes corporaciones del sector.

En este contexto, el proyecto DAZN Fantasy se posiciona como una alternativa con mayor potencial de crecimiento, al estar respaldado por una gran plataforma de distribución de contenido deportivo como DAZN, lo que le permite aspirar a una mayor integración entre contenido audiovisual y experiencia interactiva del usuario.

3.4 Representación visual de los competidores



Instalaciones de EvenBytes



Instalaciones de CIC



Plain Concepts



Inetum

Figura 7 — Representación visual del ecosistema de competidores.

3.5 Conclusión del análisis

Tras el análisis del entorno y de las principales empresas competidoras del sector, se puede observar que el mercado tecnológico en el ámbito del desarrollo de software y plataformas digitales está dominado principalmente por grandes consultoras y empresas de servicios IT con una alta capacidad de recursos y experiencia en proyectos de gran escala.

Sin embargo, en el caso concreto de las **fantasy leagues**, el número de competidores directos es reducido, destacando principalmente plataformas especializadas como Futmondo, lo que indica que se trata de un nicho de mercado en crecimiento y con margen de expansión.

Este contexto representa una oportunidad estratégica clara para el desarrollo del proyecto DAZN Fantasy, ya que combina un sector tecnológico en crecimiento con un ámbito deportivo altamente consolidado y con gran volumen de usuarios potenciales.

Además, la posición de Nexora como empresa desarrolladora permite abordar el proyecto desde un enfoque flexible, adaptado a las necesidades del cliente y con capacidad de integración de soluciones modernas y escalables.

En conclusión, el análisis realizado confirma la viabilidad del proyecto y refuerza la existencia de una oportunidad real de mercado dentro del sector de las plataformas de entretenimiento deportivo digital.

3.6 Análisis de productos competidores

3.6.1 Comunio

Figura 8 — Logo de Comunio

Es uno de los primeros Fantasy del futbol español y tiene una comunidad muy fiel.

Posee una interfaz y reglas muy sencillas, el sistema de puntuación está basado en notas otorgadas por periodistas lo cual las hace más fácil de entender, además tiene una versión gratuita funcional, por lo que no hace falta pagar una entrada a la liga. También permite crear ligas privadas entre amigos con un mercado exclusivo para cada una de estas.

Por otro lado, como podemos observar si visitamos su sitio web (Comunio, s. f.) , el diseño y la interfaz que tiene esta completamente desfasada (no lo han renovado casi desde su creación), la aplicación del móvil es muy limitada; es poco fluida e intuitiva comparada con otros productos existentes. Tiene pocas funciones adicionales como podrían ser logros, estadísticas avanzadas, etc. Otro punto en su contra seria las puntuaciones; esta correcto que sean basadas en opiniones de periodistas, pero al mismo tiempo esto las hace ser dadas de forma completamente subjetiva.

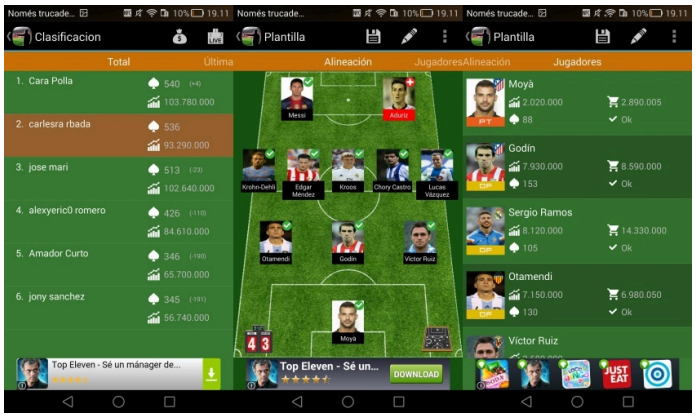


Figura 9 — Interfaz de Comunio

3.6.2 Biwenger



Figura 10 — Logo de Biwenger

Posee una interfaz moderna y con gran usabilidad; es de las mejores valoradas por diseño y experiencia de usuario, es altamente personalizable en cuanto al modo de fichajes, el presupuesto inicial de cada jugador, etc.



Figura 11 — Interfaz móvil de Biwenger

Al pertenecer al periódico As está completamente integrado con medios oficiales como este mismo o la propia La Liga, dándole así más veracidad y credibilidad al tener datos en tiempo real y estadísticas completas.

Existe la opción de crear Multiligas haciendo que no se limite solo a la liga española.

Por otro lado, la versión web que tiene es mucho menos fluida que la aplicación móvil, el sistema de puntuación es mucho más complejo y por experiencias compartidas por usuarios puede llegar a ser abrumador entenderlo.

Por último, la queja mayoritaria de usuario de esta Fantasy League es la publicidad que posee, muchos anuncios y muy intrusivos.

Logotipo e imágenes obtenidas de recursos online de la aplicación y pagina web de Biwenger (As-Biwenger, s. f.).

3.6.3 Movistar Fantasy Marca

Figura 12 — Logo de Liga Fantástica Marca

Su punto más fuerte es el respaldo del periódico Marca, teniendo de base una gran publicidad y visibilidad en medios deportivos.

Tiene una interfaz moderna y correcta comparado con otras aplicaciones de Fantasy Leagues.

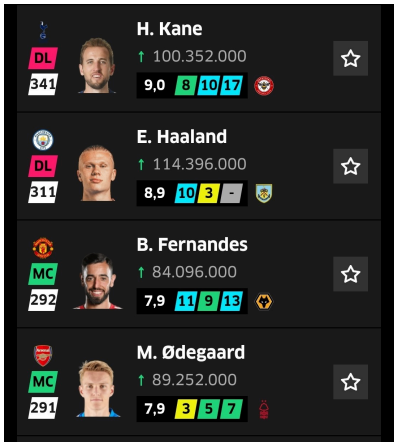


Figura 13 — Interfaz Liga Fantastica Marca

Su sistema de puntuación está basado en datos objetivos proporcionados por Stats Perform (Stats Perform, s. f.) , empresa especializada en datos deportivos y análisis avanzados y posee actualizaciones a tiempo real.



Figura 14 — Stats Performs

Realizan eventos con premio oficiales para mantener el interés de los usuarios.

No tiene casi personalización comparado con competidores como Biwenger, está enfocado completamente en la competición y deja de lado el apartado social; esto hace que haya menos interacción entre usuarios y menos ligas privadas.

Depende directamente de Marca en cuanto a la promoción y publicidad, y por consecuencia sus servidores son inestables ocasionalmente en días de jornada laboral.

4. ANÁLISIS ESTRATÉGICO

Para poder encarar con confianza el desarrollo de esta aplicación debemos, además de estudiar como ya hemos hecho las empresas y productos rivales, estudiar las que creemos que son nuestras **fortalezas y debilidades**.

Vamos a comenzar primero analizando grosso modo los que creemos que son nuestros puntos fuertes y débiles, para luego pasar a analizarlo de una forma un tanto más exhaustivo.

4.1 PUNTOS A NUESTRO FAVOR

Salario medio de la empresa en este rango: Alrededor de **21000**, lo que nos hace competitivos en ese sentido.

Sin abarcar una gran cantidad de tecnologías usamos aquellas que son líderes en el mercado, como **Angular**, **Typescript**, **Node.js** y **Java**. Esto hace que nos mantengamos a la vanguardia de las tecnologías que se utilizan actualmente.

Nuestro sistema de organizar y trabajar por proyectos y por tecnologías nos da **gran flexibilidad** a la hora de desarrollar.

Nuestra empresa no se vuelca todavía en campos como el BIG DATA, DATA ANALISIS e Industria 4,0 lo que hace que seamos aún más competitivos en centrarnos en desarrollar aplicaciones web y móviles.

4.2 PUNTOS EN NUESTRA CONTRA

Somos una empresa más pequeña en comparación con la mayoría de las empresas con las que nos comparamos, lo que puede limitarnos en algunos aspectos.

Contamos con menos años de experiencia en general en comparación al resto de empresas.

4.3 DAFO y CAME



Figura 15 — DAFO y CAME

5. PROPUESTA DEL PROYECTO

5.1 Descripción general de DAZN Fantasy League

A la hora de desarrollar un concepto para la aplicación lo primero que se nos viene a la mente es generar **interfaces más modernas para la aplicación**, sobre todo en la parte web. La parte móvil de todos nuestros competidores suele estar bien trabajada, algunas mejores que otras. Sin embargo, las plataformas de navegador suelen ser sitios web con scrolls verticales larguísimos, en los cuales, se van alternando secciones que se refieren a la propia liga o aplicación con secciones dedicadas exclusivamente a publicidad.

Una de nuestras principales metas es crear un sitio web más **moderno**, en el que todo esté dividido en secciones más pequeñas a las que se pueda **acceder fácilmente** sin una sección principal que sea monolítica y difícil de navegar o de encontrar algún elemento.

5.2 Propuesta de valor

Para proporcionar a nuestra aplicación con valores añadidos que nos diferencien de la competencia hemos decidido enfocarnos en los siguientes puntos:

Diseño y experiencia modernizada:

- Ofrecemos una **experiencia intuitiva y atractiva**, accesible tanto para jugadores nuevos como experimentados a través de la combinación de simplicidad junto a funcionalidades más avanzadas.
- Nuestra **presentación** de la interfaz de usuario es más **moderna e intuitiva** que la de nuestros competidores.

Gamificación:

- Nuestros competidores se centran en ligas privada mientras que nuestra idea trata de incorporar **elementos adicionales** como **logros, retos diarios/semanales, recompensas virtuales**

Flexibilidad:

- Así como otras aplicaciones son muy personalizables hasta el punto de abrumadoras o totalmente lo contrario, nuestra aplicación va a **permitir realizar configuraciones diferentes sin llegar a complicar la experiencia a los usuarios**, permitiendo así que tanto un grupo de amigos pueda crear su liga fantasy simple o una liga competitiva con un reglamento completo.

Transparencia:

- Mientras otras aplicaciones usan métodos de calificación subjetivos como pueden ser periodistas profesionales o empresas que proporcionan datos de forma abrumadora nosotros podemos ofrecer **estadísticas claras, objetivas y fáciles de entender** con explicaciones visuales, combinando así **transparencia y comprensión sencilla**.

6. PLANIFICACIÓN GENERAL DEL PROYECTO

6.1 Metodología de Trabajo

Para este proyecto, como en muchos otros desarrollados por Nexora, nos apoyaremos en una de las **metodologías Ágiles** más utilizadas en la actualidad, la metodología **Scrum**, tal y como indican diferentes estudios y encuestas (Echometer, 2025) .

Esta metodología nos da importantes ventajas a la hora de llevar a cabo un desarrollo de este tipo:

- **Flexibilidad**: Podemos adaptar los flujos de trabajo a nuevos requisitos o problemas del proyecto sin que se paralice todo el desarrollo.
- **Feedback**: Que nuestro cliente pueda comprobar sprint a sprint si el producto cumple con sus expectativas dándonos valiosa información y opinión al respecto que nos permita adaptarlo a sus gustos y necesidades.
- **Mayor calidad final**: El desarrollo de la aplicación en **constante comunicación con el cliente** hace que seamos mucho más conscientes de las fortalezas y debilidades de este, lo que hace que podamos refinarlo mucho más obteniendo así un **mejor resultado final**.
- **Desarrollo en paralelo**: Podemos llevar a cabo un **desarrollo simultaneo de las diferentes funciones** del proyecto a la vez, lo que reduce tiempos y nos da mayor oportunidad de

comprobar el funcionamiento e interacción de las diferentes partes según va avanzando el proyecto.
- Podemos comprobar las ventajas del uso de esta metodología basándonos en diferentes opiniones que concuerdan en diferentes puntos tales como los presentados por la Universidad Internacional de la Rioja (UNIR, 2025) , INMUNE (INMUNE, 2023) o el publicado en Ofiprix por Álvaro García Martins (A. G. Martins, s. f.) .

6.2 Fases del proyecto

Para este proyecto nos plantearemos **cuatro fases**:

Definición y requisitos: Durante esta primera fase nos centraremos en **concretar los requisitos** de nuestro cliente, analizando la **viabilidad de las propuestas** y definiendo las **acciones necesarias** a realizar para llegar a los **objetivos acordados**.

Planificación del desarrollo: En esta segunda etapa **definiremos los equipos** que se encargarán del desarrollo de nuestro producto, calcularemos la **duración estimada** del mismo, las **tecnologías** sobre las cuales desarrollaremos la aplicación y analizaremos los **posibles problemas** que nos podamos encontrar durante la duración de este.

Sprints y desarrollo: Esta será la fase en la que se definirán los **objetivos de los distintos Sprints** en los que dividiremos nuestro proyecto, estableciendo los objetivos de cada uno, las tareas a realizar durante el mismo y la dirección de cada uno de ellos. Con todo esto podremos tener una **estimación del porcentaje de desarrollo** que vamos completando además de que iremos recibiendo **comentarios y feedback** por parte de nuestro cliente de cara a futuros sprints.

Implementación y despliegue: En esta última parte nos aseguraremos de tener un **producto terminado y funcional**, que ya puede ser usado y accedido por los diferentes usuarios. Se tendrán en cuenta las plataformas a usar para el despliegue del producto, así como **futuras mejoras** para el mismo e incluso posibles **correcciones de errores y problemas**.

6.3 Organización de los equipos

Para el desarrollo de esta aplicación se van a destinar los siguientes profesionales de entre nuestra plantilla:

1 Product Owner/Scrum Master. En muchos casos o proyectos suelen ser personas separadas, en nuestro caso consideramos que con **una sola persona** que se encargue de los dos roles nos permitirá concentrar mejor sus responsabilidades y conseguiremos un **resultado**

mucho más homogéneo entre los que nos pide nuestro cliente con lo que se lleva a cabo por parte del equipo de desarrollo. Las funciones de esta persona serán las de obtener los requisitos, organizar a cada miembro del equipo, asignación de tareas, definición de tiempos y sprints, enlace entre el cliente y el equipo de desarrollo.

1 Senior Developer. Va a ser el miembro más experimentado del equipo de desarrollo. Llevará el **liderazgo en el desarrollo del producto** además de que será el que marcará las pautas a seguir por el resto de los miembros del equipo. En este caso, se buscará de entre todos nuestros empleados un **perfil de full-stack developer**, pues nuestra aplicación debe contar tanto con elementos web, de backend y bases de datos.

1 Diseñador UI/UX. Se encargará del **diseño de la interfaz de usuario** además de la **experiencia de usuario**. Para llevar a cabo esta tarea debe de fijarse en otras interfaces similares para encontrar la manera de proveer a nuestra aplicación de una **interfaz intuitiva y amigable**, además de **moderna y limpia**.

3 Junior Developer. Acompañarán al Senior durante todo el desarrollo de la aplicación implementando las diferentes soluciones elegidas para nuestro proyecto. Realizarán tareas tanto de front-end, back-end y base de datos, siempre que estas sean acordes a su experiencia y a su nivel de conocimientos.

1 Técnico en QA. Será el encargado, cerca de la finalización del desarrollo, de **implementar planes de pruebas y tests** para comprobar la funcionalidad de nuestra aplicación, además de **generar informes** que sirvan para subsanar los errores que se encuentren y que sirvan para validar si se pueden dar por terminadas las tareas que habían sido asignadas.

1 DevOps o Técnico en Sistemas. Serán los encargados de, una vez se considere que el producto ya puede pasar a producción, **desplegar la aplicación completa** preparando todo el entorno y la infraestructura necesaria para sostener y hacer accesible el producto. Su actividad ocupará la última parte del proyecto.

6.4 Visión general de los Sprints

Se establece un periodo semanal de trabajo de **7,5 horas** por jornada laboral extendido durante **tres meses**. Se ha realizado una estimación previa en cuanto al recuento de horas necesarias para el desarrollo del proyecto, con lo que deducimos que no será necesaria la realización de horas extraordinarias. Por lo tanto, se podrán calcular las horas totales de desarrollo de cada sprint definido en el proyecto, así como las de cada equipo.

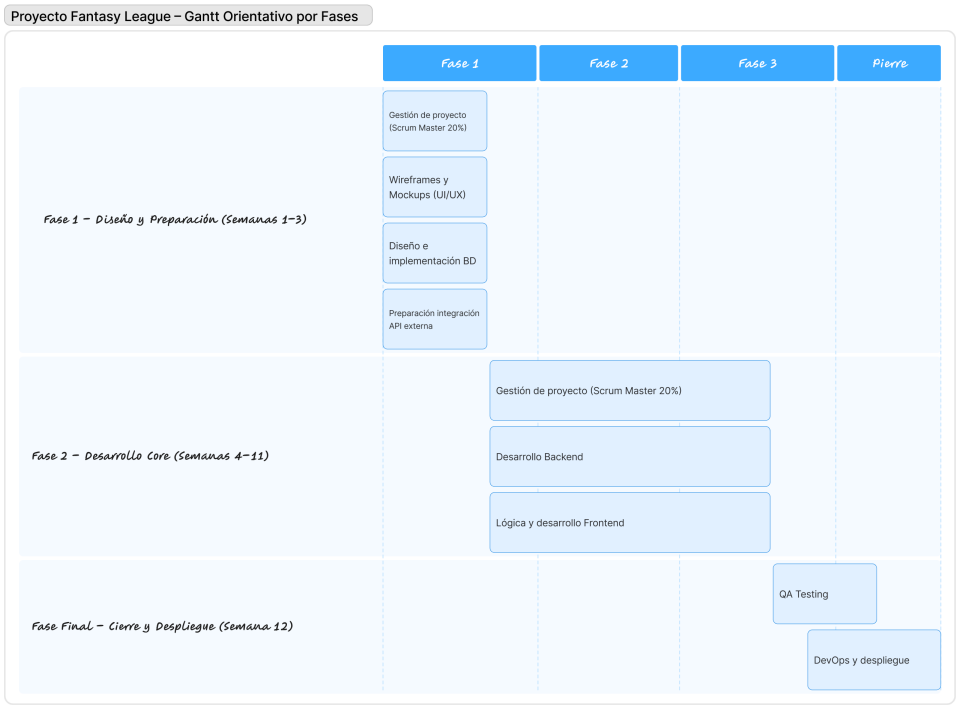


Figura 16 — Diagrama de Gantt orientativo de la organización del proyecto

6.5 Fases de desarrollo

Primera fase del Desarrollo (Semanas 1 a 3):

Durante las tres primeras semanas del proyecto el primer equipo de desarrollo se encargará principalmente de la parte de **Backend**.

- Creación y definición de la **base de datos necesaria** para la aplicación. En esta se incluirán los usuarios, ligas privadas y públicas.
- **Conexión a la API**; pensada específicamente para el proveedor; y gestión de los datos de esta para integrarlos en la base de datos según sean necesarios.
- Diseño del **sistema de creación de ligas** privadas, así como de las ligas públicas o comunitarias oficiales.

- **Integración completa** de todo lo previamente mencionado para asentar una base única y cohesionada; siendo esto posible gracias a tener un solo equipo dedicado específicamente a esta fase.

Considerando una jornada diaria de **7,5 horas**, distribuida en cinco días laborables por semana, con la participación de **cuatro trabajadores** durante un periodo total de **tres semanas**, se obtiene un cómputo global de **450 horas de trabajo**.

Por otro lado, el **equipo de diseño** se encargará del **diseño y desarrollo de la interfaz**. Este diseño se realizará al principio para asentar una interfaz base sobre la que el resto de los equipos de desarrollo puedan trabajar e ir realizando cambios o nuevas implementaciones cuando sean necesarias, ya sea por cambios planteados como mejoras o soluciones a problemas existentes.

En el caso de este equipo de diseño tendremos en consideración una jornada diaria de **7,5 horas**, distribuida en cinco días laborables por semana, con la participación de **un trabajador** durante un periodo de una semana, obteniendo un cómputo global de **105 horas de trabajo**.

Segunda fase del Desarrollo (Semana 4):

Durante esta semana el primer equipo de desarrollo se encargará de **afinar al máximo posible el producto** obtenido durante la primera fase, asegurándose de la **perfecta integración de la base de datos, API e interfaz web** para terminar de **asentar la base** desde la que proseguir con el desarrollo y la futura unión del segundo equipo de desarrollo al proyecto.

Considerando que la jornada diaria es de **7,5 horas**, distribuida en cinco días laborales, trabajando durante esta fase **4 trabajadores**, obtendríamos un total de **140 horas laborales**.

Tercera fase del Desarrollo (Semanas 5 a 11):

Durante esta tercera fase de desarrollo, el equipo de desarrollo se centrará en la **creación de toda la parte web**, cerciorándose de que todo funciona correctamente a través de la realización de **pruebas de funcionamiento e integración**, manteniendo así el plan establecido cumpliendo tanto los tiempos como objetivos y previsiones establecidas.

Considerando que la jornada diaria es de **7,5 horas**, distribuida en **cinco días laborales**, trabajando durante esta fase **4 trabajadores** en un periodo de **7 semanas** obteniendo un total de **1050 horas**.

La última semana se centrará en el **equipo de despliegue**, encargados de realizar las **pruebas y ajustes necesarios para desplegar la aplicación** y asegurarse de que el proyecto incluye las dependencias pertinentes en cada una de sus versiones. Con esta última semana de desarrollo nos aseguraremos de que la aplicación saldrá de producción en la fecha estimada.

El tiempo total estimado será de **37,5 horas laborales** para esta última semana de desarrollo.

7. ESTIMACIÓN Y CÁLCULO DE COSTES DEL PROYECTO

En esta sección desglosaremos por apartados los diferentes gastos que repercuten directamente en el desarrollo de esta aplicación. Con ello se podrá obtener una aproximación mucho más fiable del precio final del producto.

Basaremos este cálculo en datos concretos y precisos. En aquellos casos en los que los datos sean estimados o variables, se tomarán referencias aproximadas al contexto real del proyecto para ofrecer un resultado coherente y objetivo.

7.1 Costes de personal

Pasamos ahora a detallar los costes salariales brutos correspondientes a los miembros integrantes del equipo de desarrollo. Debemos tener en cuenta que cada miembro participa durante periodos de tiempo distintos y que no se contemplan posibles desviaciones derivadas de incidencias o imprevistos.

Nuestra empresa paga los siguientes importes salariales:

BRUTOS ANUALES	BRUTOS MENSUALES	PUESTO
26.000,00 €	2.166,67 €	QA
34.000,00 €	2.833,33 €	WEB/UI-UX
41.000,00 €	3.416,67 €	DEVOPS
28.000,00 €	2.333,33 €	TS
32.800,00 €	2.733,33 €	SENIOR DEV
24.200,00 €	2.016,67 €	JUNIOR DEV
38.200,00 €	3.183,33 €	**SCRUM MASTER**

Tabla 1 — Distribución de salarios brutos anuales y mensuales por puesto

Conociendo los importes brutos anuales y mensuales pasamos a calcular los importes brutos semanales. Una vez calculados, dado que los diferentes trabajadores invierten diferentes cantidades de tiempo en el proyecto, podemos calcular el total bruto según el tiempo que cada individuo supone al costo del proyecto.

PUESTO	COSTE SEMANAL	SEMANAS	Nº TRABAJADORES	COSTE TOTAL
QA	541,67 €	1	1	541,67 €
WEB/UI-UX	708,33 €	3	1	2124,99 €
DEVOPS	854,17 €	1	1	854,17 €
TÉC. SISTEMAS	583,33 €	1	1	583,33 €
SENIOR DEV	683,33 €	12	1	8199,96 €
JUNIOR DEV	504,17 €	12	3	18150,12 €
SCRUM MASTER	159,17 €	12	1	1909,99 €

Tabla 2 — Coste salarial bruto por semanas de trabajo aportadas al proyecto

El coste bruto total correspondiente a salarios asciende a:

32364,23 €

*El Técnico de Sistemas (TS), aunque no está contemplado que participe en el proyecto, se prevé su salario por si se diese el caso de que en ultima instancia deba de participar en la última semana para apoyar en las tareas de despliegue.

En la tabla anterior hemos visto los diferentes salarios brutos que pagamos en Nexora. Los datos aportados en la misma se acomodan a los valores establecidos por la ley y creemos están acordes a la media de los mismos en el sector. Véanse los siguientes enlaces (Indeed, s. f.-a), (Indeed, s. f.-b), (Indeed, s. f.-c), (Jooble, s. f.-a), (Jooble, s. f.-b), (Jooble, s. f.-c) , (Talent.com, s. f.) .

7.2 Cotizaciones y cargas sociales

Derivados de los cálculos del apartado anterior debemos obtener ahora el importe de los gastos correspondientes al pago de seguros sociales de los propios trabajadores. Estos gastos vienen dados por las siguientes tablas que podemos encontrar en las plataformas oficiales de la Seguridad Social y de empresas asociadas (Asepeyo, 2025), (Seguridad Social Española, 2025) .

Para obtener estos datos solo debemos usar los totales obtenidos en el apartado anterior y aplicarles un cálculo del 30,57% que desglosaremos por puesto.

PUESTO	BASE	30,57 %	TOTAL
QA	541,67 €	30,57 %	165,59 €
WEB/UI-UX	2124,99 €	30,57 %	649,61 €
DEVOPS	854,17 €	30,57 %	261,12 €
TS	583,33 €	30,57 %	178,32 €
SENIOR DEV	8199,96 €	30,57 %	2506,73 €
JUNIOR DEV	18150,12 €	30,57 %	5548,49 €
SCRUM MASTER	1909,99 €	30,57 %	583,88 €

Tabla 3 — Coste de las cotizaciones totales de los trabajadores aportadas al proyecto

El coste total correspondiente a cotizaciones y cargas sociales asciende a:

9893,75 €

No desglosaremos las cuotas correspondientes a la seguridad social pero el total de las mismas se divide en:

- 23,60 % Contingencia Comunes
- 5,50 % Desempleo
- 0,20 % FOGASA

- 0,60 % Formación
- 0,67 % MEI (Mecanismo de Equidad Intergeneracional)

De esto se obtiene el 30,57 % aplicado al total de los costes en bruto de los salarios de los trabajadores.

7.3 Gastos administrativos

Nuestra empresa cuenta con su propio departamento de Recursos Humanos, el cual se encarga de los cálculos de nóminas, seguros sociales, prestaciones y trabajos administrativos derivados de la gestión del total de nuestra plantilla.

Se debe de incluir la parte correspondiente de los gastos salariales y cuotas correspondiente que corresponderían a nuestro equipo de desarrollo. Recordamos que nuestra empresa cuenta con 150 trabajadores, lo que hace que tengamos que calcular los gastos adicionales por trabajador y luego obtener un total de lo que correspondería a este equipo concreto.

Las personas encargadas de estas gestiones son las siguientes:

- 2 Oficiales de primera Administrativos con un sueldo bruto medio de 25.700 €.
- 1 Jefe de Departamento de RR.HH con un sueldo de 31.400 €.

Primero debemos de calcular, para que vaya acorde con los cálculos realizados anteriormente, el coste total de estos trabajadores por semana, incluido en esos costes los seguros sociales. Después debemos de tener en cuenta que, dado que cada miembro del equipo de desarrollo participa en periodos distintos de tiempo, cada uno de ellos llevará un importe distinto de gastos adicionales.

PUESTO	BRUTO ANUAL	BRUTO/SEMANA	CUOTAS/SEMANA	TOTAL/SEMANA	TOTAL (/150)
JEFE DEP.	31.400,00 €	654,17 €	199,98 €	854,15 €	5,69 €
ADMINISTRATIVO	51.400,00 €	1.070,83 €	327,35 €	1.398,18 €	9,32 €

Tabla 4 — Coste salarial personal administrativo 1

Como resultado de los cálculos vistos en la tabla superior obtenemos que, como gastos administrativos, debemos de añadir un total de 15,01 € adicionales a la semana por trabajador.

Ahora debemos de cruzar estos resultados con las semanas de trabajo para el desarrollo de este proyecto desglosándolas por trabajador, para así poder obtener el total de gastos administrativos.

PUESTO	TRABAJADORES	SEMANAS	TOTAL TRABAJADOR	TOTAL
QA	1	1	15,01 €	
WEB	1	3	45,03 €	
DO	1	1	15,01 €	
TS*	1	1	15,01 €	**846,56 €**
SENIOR DEV	1	12	180,12 €	
JUNIOR DEV	3	12	540,36 €	
SCRUM MASTER	1	12	36,02 €	

Tabla 5 — Coste salarial personal administrativo 2

Como podemos observar en la anterior tabla, nuestro proyecto deberá asumir un total de 846,56 € de gastos adicionales correspondientes a las gestiones administrativas de los miembros del equipo.

7.4 Equipos Informáticos

En este apartado desglosaremos los gastos en los que incurre nuestra empresa propiciados por los equipos informáticos de los que hará uso el equipo de desarrollo asignado al proyecto.

Para todos los trabajadores asignados a desarrollo se usan portátiles de unas especificaciones similares. Los equipos a los que nos referimos (Lenovo, s. f.) .



Figura 17 — Lenovo ThinkPad 14 Gen 6

El ciclo de vida útil y eficaz de cada uno de ellos se establece en un máximo de 10 años, el estándar para este tipo de dispositivos en cuanto a cálculos de amortización se refiere según lo mostrado por los estamentos oficiales del estado y que muestran en el siguiente informe (Agencia Tributaria, 2026) . Como podemos observar, según la tabla de la Agencia Tributaria, podemos amortizar estos equipos con un coeficiente máximo del 26%, lo que equivale a cerca de 4 años, sin llegar a ese número. Esta misma tabla, nos indica que el periodo máximo será de 10 años. En nuestro caso, dado el uso intensivo que se le da a estos equipos, optaremos por calcular su amortización en 4 años máximo. Como bien se indica también en organismos oficiales, el valor residual de este tipo de equipos es 0, pues una vez llegado el fin de su vida útil su valor de venta es mínimo. Por lo tanto, no tendremos en cuenta este valor para realizar los cálculos.

Una vez sabido y explicado todo esto, debemos acotar también este gasto de amortización, a la duración de nuestro proyecto, que como ya hemos explicado en otros apartados, es de 3 meses. Una vez explicados todos los factores que intervienen podemos desglosar este gasto de la siguiente manera:

COSTE TOTAL	AMOR. ANUAL	AMOR. MENSUAL	TOTAL MESES (3)	TOTAL EQUIPOS (9)	COSTE TOTAL
958,02 €	239,51 €	19,96 €	59,88 €	538,92 €	**538,92 €**

Tabla 6 — Amortización equipos informáticos

Como podemos ver, gracias a los cálculos desglosados en la tabla superior, al final del proyecto deberemos repercutir un total de 538,92 € adicionales en concepto de amortización de equipos informáticos.

7.5 Gastos derivados del uso de herramientas digitales

Aunque en principio no se prevé ningún gasto adicional derivado del uso de herramientas o plataformas digitales, consideramos buen criterio incluir esta sección aquí con el fin de reflejar los gastos que se podrían derivar del uso de tecnologías de desarrollo.

Aunque el gasto en este apartado sea nulo, la transparencia en este aspecto nos parece algo fundamental y consideramos esencial desglosar el listado de herramientas que se van a utilizar durante el desarrollo. En los casos en los que algunos de los desarrolladores hagan uso de elementos bajo subscripción o pago, es política de la empresa que sean los propios desarrolladores que se hagan cargo de estos gastos, pues la propia empresa considera que las herramientas proporcionadas son más que suficientes para el buen desempeño de sus tareas.

7.5.1 Listado de Tecnologías

Icono	Herramienta	Descripción
	Microsoft Visual Studio	Entorno de desarrollo de código abierto
	GitHub	Tecnología de control de versiones.
	GitHub Copilot e IA	Asisten en la resolución de conflictos y dudas.
	GitHub Projects	Gestor de proyectos de Github
	Microsoft Teams	Herramienta utilizada para optimizar el trabajo y la comunicación corporativa.
	Notion	Plataforma de organización empleada para centralizar información y tareas del proyecto.

Tabla 7 — Herramientas y tecnologías utilizadas en el desarrollo del proyecto. Iconos obtenidos de (Simple Icons, s. f.) y (Iconify, s. f.)

7.6 Otros gastos propios de la empresa

Por último, debemos de incluir los gastos de mantenimientos propios de la infraestructura de la empresa. En este apartado solo incluiremos gastos de limpieza y mantenimiento de las instalaciones.

Este apartado, mucho más sencillo, se reducirá a calcular el gasto por trabajador al mes en estos aspectos para poder así calcular el gasto total durante los 3 meses de desarrollo.

Podemos tomar como ejemplo de gastos los mostrados en el enlace de Habitissimo (habitissimo, s. f.) .

En nuestro caso la empresa pagaría por los siguientes servicios:

1. 600 €/mes en concepto de limpieza del edificio.
2. 1400 €/mes en concepto de mantenimientos de instalaciones y diferentes infraestructuras.

Esto hace un total de 2000 € al mes que, para ser correctos y exactos, debemos dividir entre la totalidad de trabajadores.

Esto añade un total de 14,67 € al mes por trabajador. Añadiendo al cálculo los 9 trabajadores del equipo de desarrollo y los 3 meses de duración supone un total de 396 € adicionales al costo total del proyecto.

7.7 Otras consideraciones

Aunque, en principio, **no correspondería a Nexora hacerse cargo del alojamiento** y el pago final de la infraestructura, se debe de **tener preparada una opción** en el caso de que pudiese darse este supuesto.

Simplemente y con carácter informativo, para hacernos una idea de los que podrían suponer estos costes adicionales, tomamos como referencia plataformas como **Firebase**, que podrían ser una opción más que aceptable para dicha tarea.

Como ya hemos indicado, en principio no nos corresponde a nosotros, pero como valor de referencia nos dirigiremos a la plataforma oficial como forma de guía (Firebase, s. f.) .

7.8 Cálculos totales

En resumen, teniendo en cuenta todo lo indicado en subsecciones anteriores, ahora si podemos indicar un valor concreto correspondiente a los gastos de desarrollo:

CONCEPTO	COSTE
SALARIOS BRUTOS	30.947,47 €
SEGUROS SOCIALES	9.460,67 €
GASTOS ADMINISTRATIVOS	846,56 €
HERRAMIENTAS Y SOFTWARE	0 €
EQUIPOS INFORMÁTICOS	538,92 €
COSTES MANTENIMIENTO	396 €
TOTAL	**42.189,62 €**

Tabla 8 — Resumen de gastos

Como podemos observar en la tabla superior, el coste total que debe de asumir Nexora durante la realización de este proyecto es de 42.189,62 €, cifra que se deberá tener en cuenta a la hora de calcular el precio que se deberá de cobrar por la totalidad de la aplicación.

7.9 Determinación del Precio Final de Venta (PVP)

Para estableces el presupuesto final, debemos de aplicar un margen comercial sobre el total de los costes que hemos calculado en los apartados anteriores (42.189,62 €). Este margen no solo representa el beneficio neto de Nexora, sino que actúa como colchón ante problemas en el desarrollo que modifiquen los costes.

7.9.1 Beneficio Industrial y Margen de Contingencia

En el sector del desarrollo de software, los márgenes pueden oscilar, pero rondan por encima del 20%. Esta variación, depende de factores tales como la complejidad del producto a desarrollar como otros riesgos inherentes al desarrollo o la adopción de nuevas tecnologías para el mismo.

En este caso, a fin de buscar un punto intermedio, se optará por realizar los cálculos sobre un 30% de margen de beneficio buscado.

Sabiendo esto podemos ir desglosando los pasos punto por punto.

7.9.2 Cálculo de la Base imponible

Este es el paso previo antes de incluir los impuestos correspondientes, en este caso el IVA. Para ello, lo único que tenemos en cuenta son los costes ya calculados para añadirles el 30% que se espera obtener como beneficio sobre esos costes.

CONCEPTO	IMPORTE
Coste Total de Producción	42.189,62 €
Margen de Beneficio (30%)	12.656,89 €
Base Imponible (Precio Sin Incluir IVA)	**54.846,51 €**

Tabla 9 — Cálculo de la base imponible

Una vez calculado este paso, podemos pasar al siguiente punto.

7.9.3 Impuesto sobre el Valor Añadido (IVA)

De acuerdo a la legislación vigente actualmente en España, se aplica el tipo general del 21% sobre la base imponible. Si añadimos estos conceptos a la base del punto anterior podremos desglosar estos pasos de la siguiente manera:

CONCEPTO	IMPORTE
Coste Total de Producción	42.189,62 €
Margen de Beneficio (30%)	12.656,89 €
Base Imponible (Precio Sin Incluir IVA)	54.846,51 €
IVA (21%)	11.517,77 €
PRECIO FINAL (PVP)	**66.364,28 €**

Tabla 10 — Cálculo del precio total más el IVA

8. ESTRUCTURA DEL PROYECTO

En este apartado comentaremos las **tecnologías que usaremos** para el desarrollo de las partes del proyecto, así como justificar los motivos que nos han llevado a elegirlas como las piezas centrales en nuestro desarrollo.

8.1 Arquitectura y estructura.

En este caso nos hemos decantado por una **arquitectura cliente servidor**. Para el trabajo en nuestra aplicación el cliente se conectará a nuestra página, tanto desde su ordenador personal como su teléfono u otros dispositivos móviles. Desde esa interfaz accederá al servidor de nuestra aplicación, encargado de procesar las peticiones de los clientes a la vez que proporciona todas las características necesarias de seguridad y autenticación.

Por último, ese servidor intermedio se conectará a la Base de Datos, encargada de almacenamiento y persistencia y, en última instancia, también realizará las últimas validaciones que se encargarán de asegurar la integridad de la información.

En nuestro caso y, por la forma en la que trabajamos y desarrollamos el proyecto, podemos hablar de un **modelo cliente-servidor en tres capas**, ya que cada una de las partes se despliega y funciona en una plataforma independiente.

Debemos de tener en cuenta que, aunque esto no modifique la estructura de nuestro proyecto, podríamos contar con una **cuarta capa**. En este caso se corresponde con las **APIs externas** de las que obtenemos datos y funcionalidades para el funcionamiento de nuestra aplicación.

Estas APIs son dos:

- **Football-API**: Plataforma externa que nos proporciona los **datos necesarios de las diferentes ligas de futbol profesional** de la que nosotros cogemos aquellos que son necesarios para el alcance de la aplicación.

  Link a API-FOOTBALL: https://www.api-football.com/

- **API de Noticias**: Hemos optado por una de las muchas **APIs de noticias existentes** para enriquecer la interfaz de nuestro proyecto y proporcionar a los usuarios un elemento adicional de información deportiva.

  Link a NEWS-API: https://newsapi.org/

8.2 Capa de Presentación – El Frontend



Figura 18 — Logotipo de Angular

En nuestro caso, para esta tarea, nos hemos decantado por desarrollar en **Angular**.

Angular es un Framework de vanguardia muy potente, enfocado en el **desarrollo de interfaces de usuario robustas, modulares y flexibles**. Durante los últimos años ha crecido de forma exponencial en captación de usuarios y profesionales gracias a las características ya mencionadas y a su casi condición de **Rolling Release**, sacando una nueva versión con actualizaciones y mejoras prácticamente cada 6 meses.

Su forma de trabajar se basa en la **reactividad**, cualidad que hace que cambios en un componente se puedan reflejar en cualquier parte de nuestra aplicación, lo que en nuestro caso supone una gran ventaja. La selección de un elemento o característica puede propagarse a cualquier elemento necesario, proporcionando un extra de velocidad a la hora de navegar por sus interfaces.

Además de todo esto, tiene las herramientas necesarias para proporcionar **seguridad e integridad** al funcionamiento de nuestro sistema.

En cuanto a su lenguaje de desarrollo, hablamos de **Typescript**. Typescript es un **superconjunto de JavaScript**, aportándole a este último características que lo hacen **más robusto** eliminando la posibilidad de bugs silenciosos que pueden estar atormentándonos de forma repentina. Si tenemos en cuenta, además, que, aun aportando nuevas características, sigue siendo básicamente JavaScript, estamos usando una herramienta extremadamente flexible y potente en cuanto a gestión de lógica y comportamiento de una interfaz gráfica.

8.3 Capa intermedia – El Proveedor – Backend

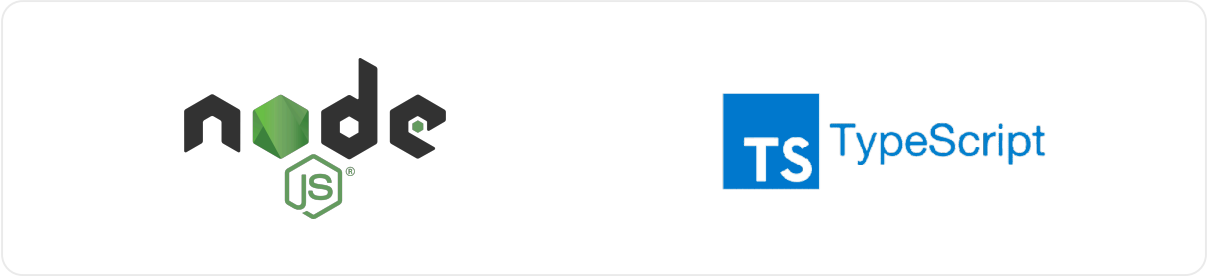


Figura 19 — Logotipos de Node.js y Typescript

Para nuestra capa intermedia nos hemos decantado por **Node.js acompañado de Express.js**. Estas dos herramientas en conjunto nos proporcionan una manera rápida y flexible de desarrollar todas las características necesarias para gestiona, validar y asegurar las transacciones entre nuestros usuarios y la capa de datos.

Estas dos tecnologías, al igual que el frontend, se basan en **JavaScript** y, en nuestro caso, también **apoyado por Typescript**, que nos proporcionará las **herramientas de tipado y verificación** necesaria para evitar inconsistencia en nuestro código.

Javascript por sí mismo se ha convertido en un **lenguaje muy potente** en este sentido gracias a los múltiples frameworks (Angular, React, Vue, Express, Node, Next) además de **versátil**, que le hacen ser apto para múltiples tipos de desarrollo. Además de ello, aunque posiblemente no sea el idóneo en cuanto a infraestructuras de gran tamaño, es capaz de adaptarse perfectamente independientemente del tamaño de nuestra aplicación.

Como **tecnologías añadidas** en este punto debemos de contar con las siguientes:

- **Sequelize**: Sequelize es un **ORM** (Object-relational mapping) que nos **permite trabajar y convertir datos** a un formato apto a las especificaciones de nuestra base de datos. Además, incluye herramientas para trabajar con **consultas personalizadas y manejar transacciones** que nos ayudaran a mantener la integridad de nuestros datos.
- **JWT**: JSON Web Token es un estándar tecnológico, (Jones et al., 2015) , que nos permite transmitir información entre partes en formato JSON y que está provisto de una firma digital. Este sistema se usa para **autenticar y autorizar usuarios**. Es más, se ha convertido en una herramienta fundamental respecto a la identidad de los usuarios y la verificación de sus permisos sin necesidad de consultas repetitivas.
- **Open-API y Swagger**: Open API, es un estándar utilizado para describir, definir y documentar RESTful APIs, pudiendo optar por formatos JSON o YAML. Su principal función es **detallar**

de una forma legible para usuarios y administradores los puntos finales o **endpoints** que exponen este tipo de aplicaciones. **Swagger**, (OpenAPI Initiative, 2023) por su parte es un **conjunto de herramientas** que se basan en las especificaciones de Open-API para llevar a cabo esta tarea, además de permitirnos mediante el navegador **acceder y probar** toda esta documentación de una forma más visual.

8.3 Capa de datos – La persistencia – SGBD



Figura 20 — Logotipo de PostgreSQL

Para nuestra capa de datos hemos elegido **PostgreSQL** por ser un **SGBD** muy valorado por su robustez, extensibilidad y amplio soporte para diferentes tipos de datos. Cuenta con una **gran fiabilidad** en entornos empresariales ya que muestra una gran capacidad de manejar consultas complejas además de modelos relacionales y no relacionales, todo ello dentro del mismo entorno.

Sus principales características:

- **Manejo de concurrencias**: Este sistema maneja múltiples transacciones simultáneas sin bloquear el sistema.
- **Código abierto y gratuito**: No tiene costes de licencia aun ofreciendo un gran rendimiento.
- **Alta extensibilidad**: Ofrece soporte para datos personalizados y funciones y extensiones para usos concretos.
- **Adaptabilidad**: Combina trabajo tanto con modelos relacionales SQL, del cual cumple con su estándar, como NoSQL.

- **Flexibilidad**: Dado lo dicho anteriormente vemos que Postgre es un sistema que puede adaptarse a gran cantidad de propósitos independientemente de las necesidades de nuestro desarrollo.

8.4 Otras tecnologías que se tuvieron en cuenta

8.4.1 JAVA + SPRINGBOOT



Figura 21 — Logotipo de SpringBoot

Se tuvo en cuenta el uso de esta tecnología en un inicio para el desarrollo del backend. Es una tecnología ampliamente usada a nivel empresarial, con una solidez y robustez fuera de toda duda. Sin embargo se decidió usar Node.js por su ligereza y por mantenernos dentro del mismo ecosistema de JavaScript - Typescript.

Conocíamos de las enormes capacidades de este lenguaje y que ofrece herramientas muy potentes en cuanto a seguridad y manejo de datos. Pero el conocimiento del entorno antes mencionado tuvo demasiado peso en nuestra decisión a la hora de decantarnos por un lenguaje u otro a la hora de desarrollar nuestro sistema.

8.4.2 MYSQL



Figura 22 — Logotipo de MySql

Otro gran standar de la industria en cuanto a gestión y almacenamiento de datos. Sin embargo consideramos que PostgreSQL se ha convertido en el estandar de facto en los tiempos actuales, además de que creemos que ofrece una mayor capacidad a la hora de gestionar gran número de solicitudes simultaneas, cosa que es algo habitual en aplicaciones como una Fantasy League, en la que miles de usuarios pueden estar consultando y trabajando con los mismos datos al mismo tiempo.

8.4.3 React y Vue.js

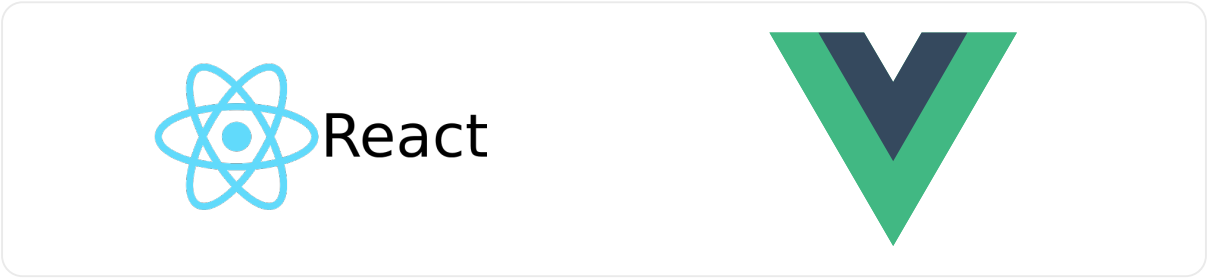


Figura 23 — Logotipos de React y Vue.js

Dentro de los diferentes ecosistemas en los que se puede desarrollar una aplicación web tenemos una ampial variedad de opciones, cada una de ellas con diferentes propósitos. Bien es cierto que, en la gran mayoría de casos, las opciones directas a considerar en nuestro caso son React y Vue.js. Cabe destacar que existe un amplio abanico pero al querer tener bien separada nuestra arquitectura en 3 capas se han descartado opciones como Next.js, Nest.js y otros frameworks de otros lenguajes como Laravel para pHp.

Estudiando nuestras dos alternativas, las razones para descartarlas son, en nuestro caso, que aunque React es un entorno ampliamente utilizado y soportado, con una gran aceptación, suele requerir de un gran número de librerías externas. Por otra parte, Vue, es extremadamente ligero y rápido, pero suele tener un escalado más difícil y requiere, del mismo modo que react, de la adición de librerías y elementos externos. Vease comparativas hechas en diferentes medios (Vaadin, 2026) .

Al poder cubrir todas esas opciones con Angular sin necesidad de añadir plugins, librerías o elementos externos, y teniendo una capacidad de escalado indiscutible, se desmarcó como claro ganador a la hora de elegir un framework para desarrollar el **front-end**. Cabe destacar que la mayoría de frameworks alternativos, exceptuando Laravel, nos permitían mantenernos dentro del ecosistema JavaScript, condición que nos parecía indispensable ya que es una gran ventaja para nosotros desarrollar 2/3 de nuestro proyecto en un solo lenguaje, hecho que a nuestro entender nos da mayor solidez y homogeneidad.

CAPÍTULO 9

DISEÑO DE WIREFRAMES, MOCKUPS Y UI

Evolución visual, prototipado, diseño responsive y construcción de identidad gráfica de DAZN Fantasy League.



Herramienta de diseño utilizada

Durante todo el proceso de diseño se utilizó **Figma** como herramienta principal de prototipado y diseño de interfaces. Esta elección permitió trabajar de forma colaborativa, iterar rápidamente sobre diferentes versiones de wireframes y construir prototipos de alta fidelidad antes del desarrollo final.

Evolución del proceso de diseño

El proceso de diseño de la aplicación se ha desarrollado de forma iterativa, comenzando con wireframes de baja fidelidad y evolucionando progresivamente hacia mockups más detallados. Este enfoque permitió validar la estructura de la aplicación antes de aplicar estilos visuales definitivos.

A lo largo del curso, el diseño fue refinándose gracias a la experiencia adquirida en la asignatura de Diseño de Interfaces Web, incorporando principios de usabilidad, jerarquía visual y diseño responsive.

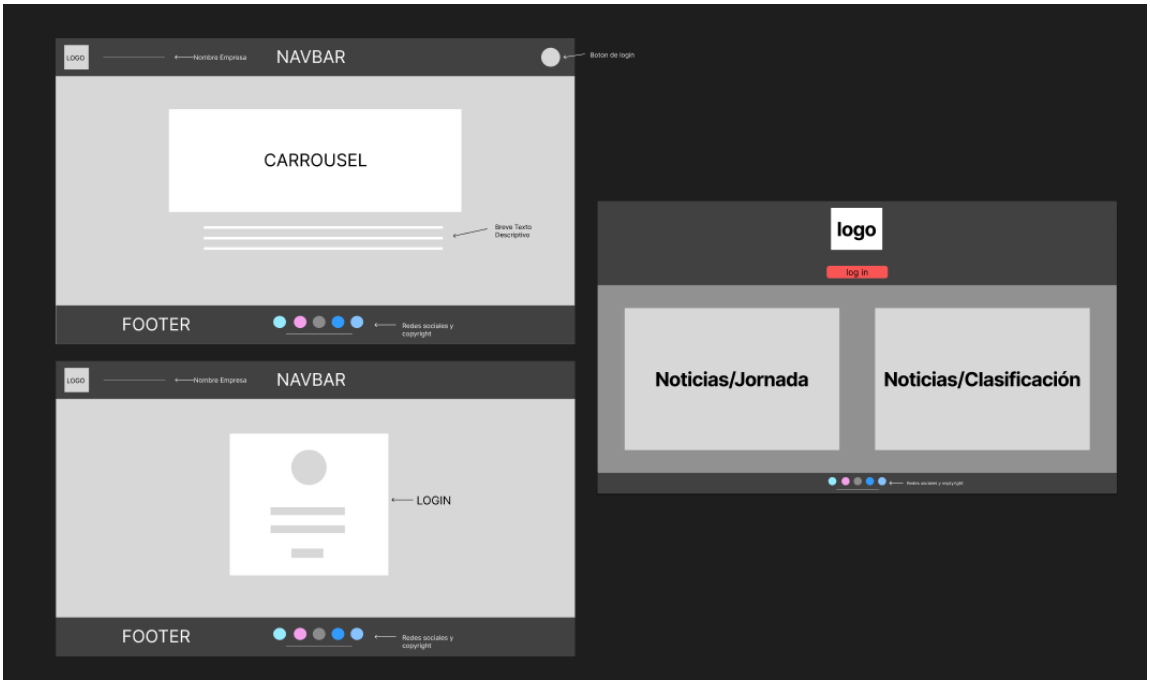


Figura 24 — Primeros bocetos de la aplicación

El **primer diseño**, que se puede apreciar en la imagen, es una versión super básica, la cual debido a la inexperiencia no sigue ninguna norma real de creación de mockups o wireframe. Aun así, es un buen primer modelo que nos sirve como base a partir de la cual podremos avanzar hacia diseños más avanzados.

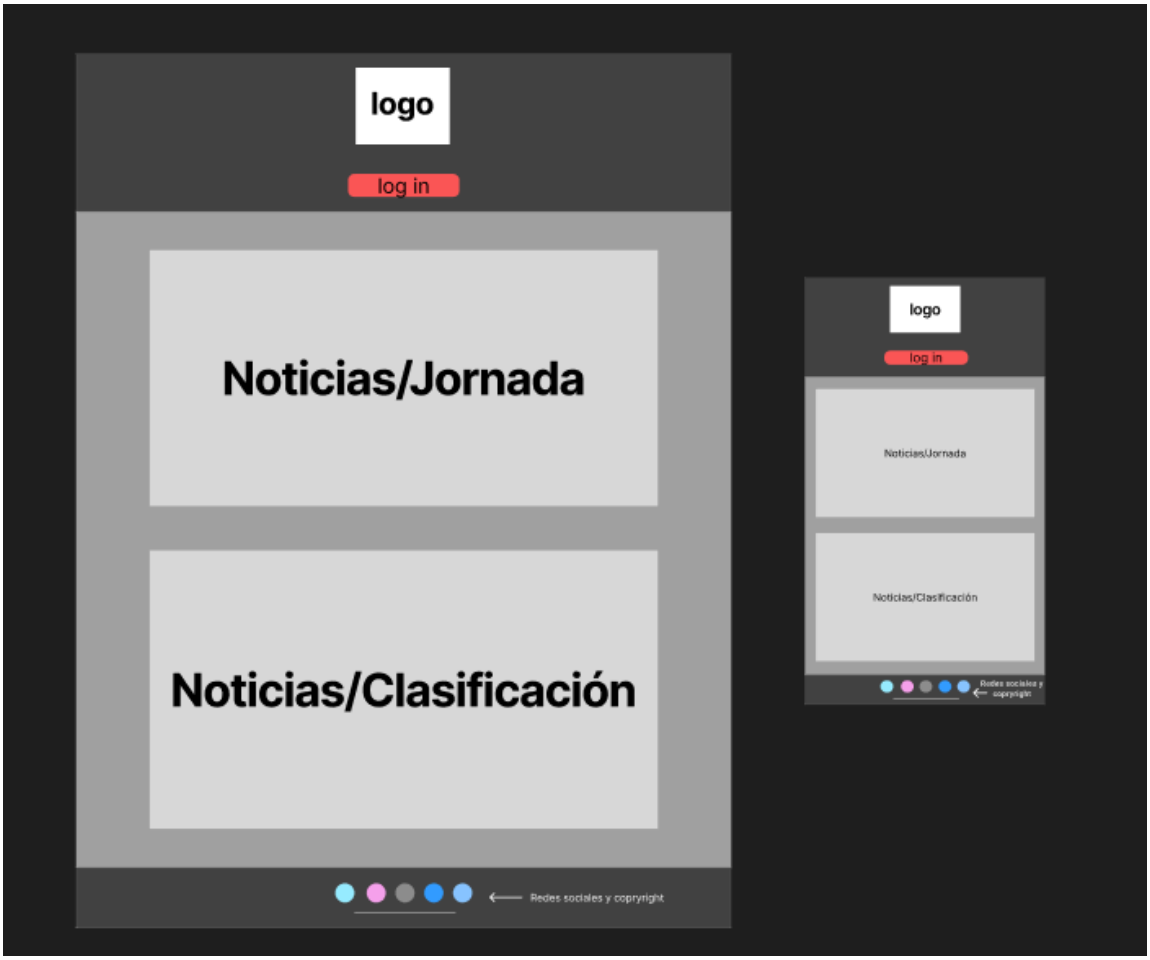


Figura 25 — Wireframes y Mock-Ups

Se puede intuir, gracias a la anterior imagen, que no teníamos ni definido el propio estilo que íbamos a seguir ya que mezclamos colores que actualmente ya no tienen ningún sentido usarlos y el diseño es bastante simple. También tendríamos las **versiones responsive** de una de las páginas con los mismos errores.

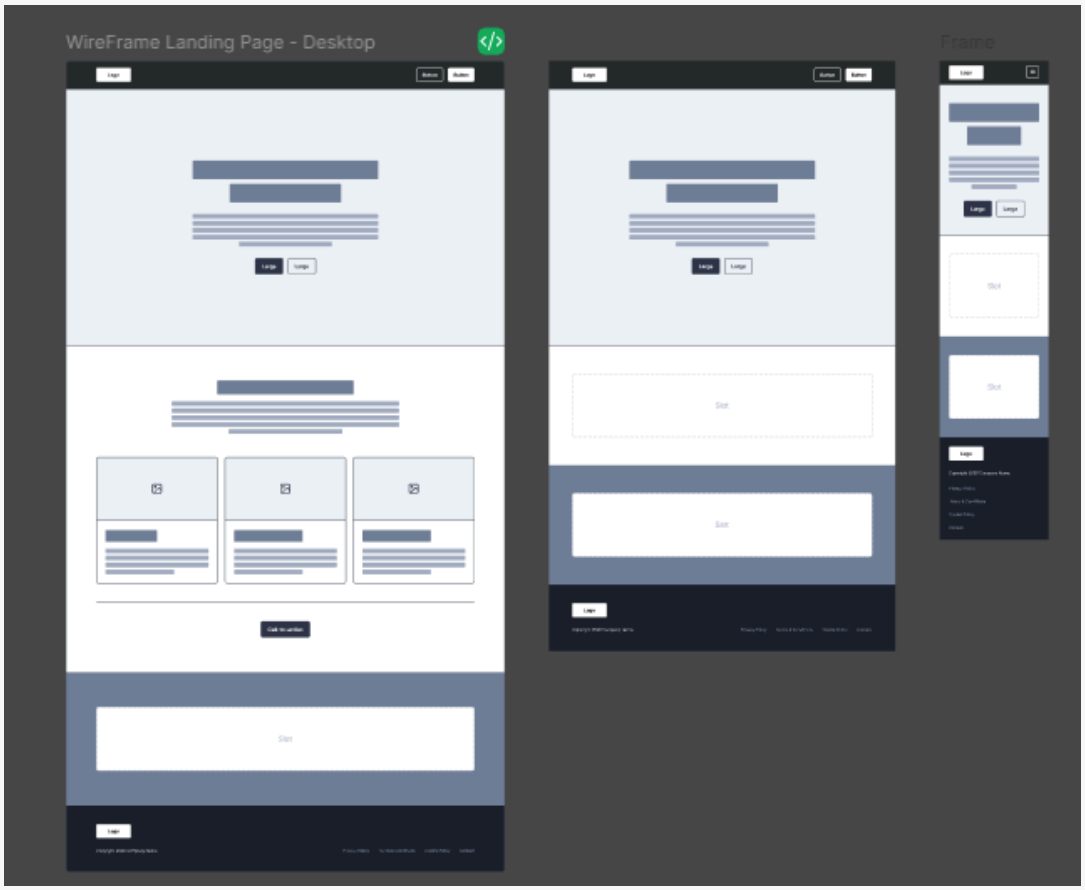
Pasado el tiempo, ya más adelantado en el curso y pensando ya en el proyecto al 100% realizamos una **segunda versión de los wireframes**, la cual terminaría siendo la que usaríamos como **plantilla final** para el diseño y maquetación de las páginas de la aplicación web final.

En las siguientes imágenes podemos observar como ya el diseño es más profesional utilizando elementos que vienen dados por una plantilla profesional de creación de wireframes con la que el propio diseñador ya puede hacer realidad sus ideas sin necesidad de tener que diseñar cada componente. En esta versión final usamos colores planos que no destaquen

demasiado entre ellos, pero manteniendo una estética coherente sobre la cual posteriormente podremos aplicar los colores y diferentes variantes de estos para que tengan relación con la empresa y marca DAZN que al fin y al cabo es la que nos ha contratado para realizar su aplicación.

Caso 1 — Landing Page

Diseño de la pantalla principal de la aplicación en sus diferentes versiones responsive (desktop, tablet y móvil). Esta pantalla define la primera interacción del usuario con la plataforma y establece la identidad visual del producto.



Diseño adaptativo de la Landing Page en múltiples dispositivos

Caso 2 — Pantalla de acceso (Login)

Diseño del sistema de autenticación de usuarios. Se priorizó la simplicidad y la reducción de fricción en el acceso, manteniendo coherencia visual con el resto de la plataforma.



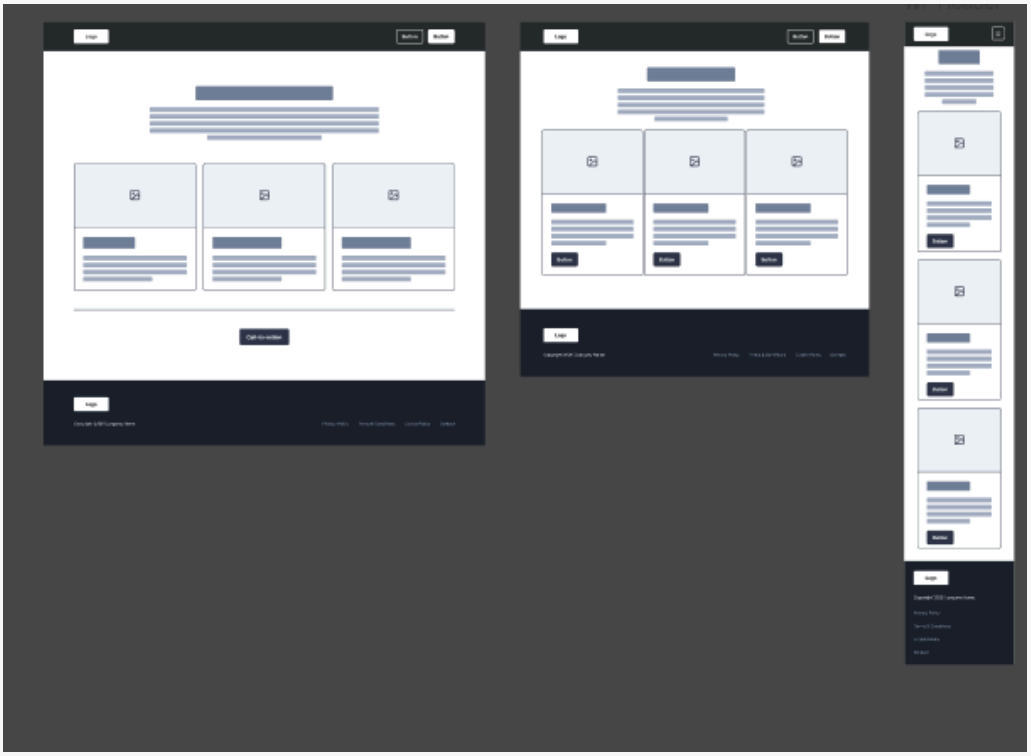
Interfaz de login optimizada para experiencia de usuario

Gracias al ensayo y error se van refinando ideas. Se separan aquellas que podrían funcionar de, aunque más simples quizás, aquellas que sabemos que funcionan. El ejemplo claro lo tenemos en las fotos anterior y posterior a este texto. En ellas comprobaremos que menos es más. Eliminaremos el ruido exterior de los diseños iniciales para centrarnos en algo mucho más simple, pero que no incita a duda al usuario durante la navegación por nuestra interfaz.

Caso 3 — Pantalla Home

La pantalla de inicio actúa como punto central de entrada a la aplicación. Su diseño está orientado a maximizar el engagement del usuario, mostrando información relevante de forma jerárquica y accesible.

Se prioriza la visualización rápida de datos clave, accesos directos a funcionalidades principales y elementos dinámicos relacionados con la competición fantasy.

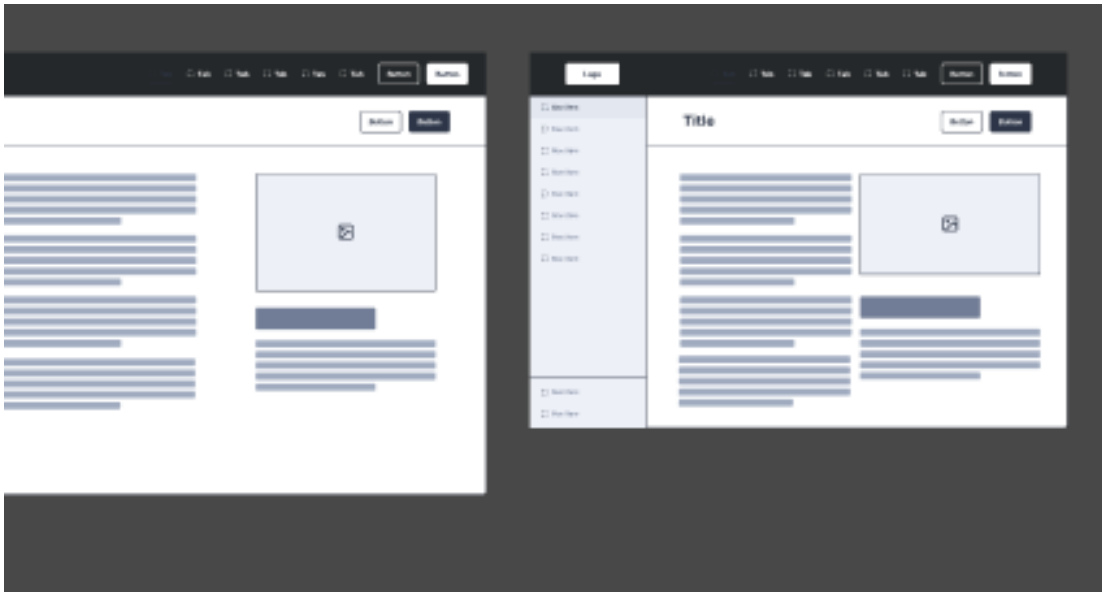


Estructura de la pantalla Home en diferentes estados y dispositivos

Caso 4 — Dashboard de usuario

El dashboard representa el núcleo funcional de la aplicación. Desde este punto, el usuario puede gestionar su equipo, consultar estadísticas y acceder a funcionalidades avanzadas de la plataforma.

Su diseño prioriza la organización modular de la información, permitiendo escalar nuevas funcionalidades sin comprometer la claridad visual ni la usabilidad.

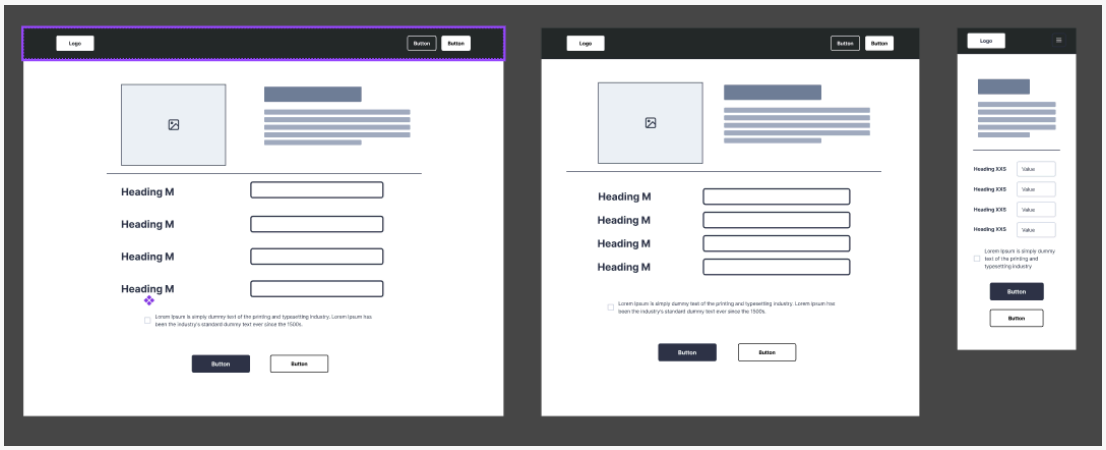


Vista principal del dashboard con estructura modular de información

Caso 5 — Configuración de usuario

La sección de configuración de usuario representa el control personal dentro de la plataforma. Desde este módulo, el usuario puede gestionar su perfil, preferencias y ajustes generales de la aplicación.

Este apartado es clave dentro de la arquitectura del sistema, ya que permite personalizar la experiencia y garantizar una adaptación flexible a diferentes tipos de usuarios.



Interfaz de configuración y ajustes de usuario

10. SISTEMA DE IDENTIDAD VISUAL Y BRAND GUIDELINES

El presente apartado define el **sistema de identidad visual** de la aplicación DAZN Fantasy League.

Este sistema establece las normas de uso de marca, composición visual, paleta cromática y tipografía corporativa aplicadas durante el desarrollo del producto.

El objetivo de este manual no es únicamente describir elementos gráficos, sino **garantizar la consistencia visual y comunicativa de la plataforma en todos sus entornos digitales**.

10.1 Misión y visión

Tanto DAZN como Nexora tienen como misión u objetivo **ofrecer a los aficionados del deporte una plataforma web de Fantasy League innovadora, dinámica y accesible** que combine entretenimiento, estrategia y competencia sana. Buscan potenciar la experiencia deportiva permitiendo a los usuarios convertirse en managers virtuales, tomar decisiones estratégicas basadas en datos reales y vivir cada jornada con mayor implicación y emoción. Fomentan la creación de comunidades activas donde tanto aficionados como amigos o compañeros puedan competir entre sí de una forma justa y transparente.

En cuanto a lo que visión se refiere el objetivo común es **convertirse en una plataforma referente en el sector de ligas fantasy deportivas** tanto a nivel nación como internacional, destacando por la innovación tecnológica, la experiencia de usuario y la capacidad de adaptación a las nuevas tendencias digitales. Queremos construir una comunidad solida donde no solo se consuma el deporte, sino que también se viva activamente. Aspiramos a integrar herramientas avanzadas de análisis de datos, inteligencia artificial y estrategias de gamificación que permitan a los usuarios tomar decisiones más estratégicas y personalizadas

10.2 Logotipo



Figura 26 — Logotipo de DAZN Fantasy League

Durante todo el proceso de creación de la aplicación web, hemos tenido diferentes ideas relacionadas con el logotipo, hemos pasado por etapas en las que pensábamos que lo más correcto sería usar simplemente el logotipo de DAZN, ya que al fin y al cabo va a formar parte de esta. Pero una vez ya más centrados en la creación de la aplicación decidimos que sí que sería optimo diseñar un logotipo exclusivo para la aplicación en sí, para esto usaríamos el logotipo de DAZN e inspirándonos en colaboraciones que tienen con otras marcas simplemente crearíamos una parte complementaria a este.

Como referencia tomamos el logotipo de DAZNBet, el cual consiste simplemente en el logo de DAZN y el logo de Bet unidos para formar uno solo como se puede ver en la siguiente imagen:

La **primera versión** de nuestro logotipo es bastante básica y no existe mucha coherencia entre ambas partes, pero al fin y al cabo lo diseños para que nos sirviera como **placeholder** para el diseño de las páginas y ver si quedaba correctamente dentro de estas.



Figura 27 — Primer boceto del logotipo

Ya pasado un tiempo decidimos que ya que teníamos las páginas más completas y preparadas con funciones básicas era momento de crear el logotipo final y el cual se quedaría como versión final. En esta versión final ya sí que nuestra parte tiene relación con el original de DAZN. Durante el proceso de diseño han surgido varios problemas como la tipografía que deberíamos usar ya que DAZN no tiene un manual de marcas con una tipografía estipulada, así que teníamos que improvisar.

La **versión final** quedaría tal que así, hay que tener en cuenta que el logotipo como tal es un **imagotipo** realmente y que es un archivo **SVG**, por lo que para que en este documento se pueda ver correctamente se le ha puesto un fondo gris para que se pueda apreciar el logo blanco de DAZN:



Figura 28 — Versión final del logotipo de la aplicación

10.3 Colores

Para este proyecto se han definido tanto un color primario con diferentes variantes como un secundario con variantes.

Los **colores primarios** que hemos elegido vienen muy dados por la imagen de marca de nuestro cliente. Elegimos 3 **variedades** del mismo tono, con diferentes valores de **luminosidad** y **saturación**. Con esto, aunque a primera vista no se note, conseguiremos crear diferentes contrastes y efectos, tales como blur y sombras, para enriquecer nuestra interfaz, hacerla más clara y, por último, poder conseguir que el usuario centre su atención en determinados elementos.

En esta imagen podemos ver los diferentes amarillos mencionados previamente, así como las nombres que hemos especificado dentro del desarrollo y maquetación para cada uno:



Figura 29 — Paleta de colores corporativos y contraste entre colores y tipografía (Aa).

Para los **colores secundarios** hemos escogido un color **negro** y una **variante** de este. la variante se aleja ligeramente del negro puro, ya que no es recomendado el uso de negro o blanco puro en ningún caso, ya sea por demasiada luminosidad o falta de ella. Eso hace un gran contraste con los colores más claros que elegimos como elementos primarios, referentes a la marca aunque, como veremos más adelante, las variantes de color negro se convertirán en el tono predominante mientras que los amarillos nos dan el acento y el contraste.

10.4 Tipografía

Para la tipografía hemos tenido un problema realmente serio a la hora de elegir la fuente ya que la **fuente de DAZN es privada** y aunque la encontráramos en la web no podríamos acceder al archivo .ttf o .otf donde tendríamos acceso a todos los caracteres para poder hacer uso de estos. Para resolver este problema al final hemos decidido usar una fuente lo más parecida posible a la oficial de DAZN sin dejar de lado que esta sea legible y visualmente agradable para el usuario.

Finalmente hemos optado por usar la tipografía **Segoe UI**, la cual tendría una **apariencia limpia, moderna y accesible**.

En la siguiente imagen vemos una muestra de la propia tipografía:



Figura 30 — Tipografía Segoe UI

11. DESARROLLO DE LA APLICACIÓN

Una vez hemos estudiado todo lo requerido, hemos decidido nuestro stack tecnológico para el desarrollo y hemos diseñado como queremos que se vea nuestra aplicación, podemos proceder al paso más importante, la escritura y desarrollo del código que va a hacer que nuestra aplicación llegue a buen puerto.

Para explicar esta sección vamos a dividirlo en las diferentes partes o capas de las que se compone el proyecto, tratando de explicar en profundidad su funcionamiento y como se relaciona con cada una de las otras capas.

11.1 La base de datos

Como la mayoría de las aplicaciones, se debe de prestar especial atención en el desarrollo de la base de datos, ya que, finalmente, esta se convierte en nuestra fuente de la verdad.

Arquitectura de persistencia

La base de datos fue diseñada como núcleo estructural del sistema, centralizando las reglas de integridad, relaciones y validaciones críticas de la aplicación. Este enfoque permite garantizar la consistencia de los datos incluso independientemente de la lógica implementada posteriormente en el backend.

En un principio, como concepto, se pensó en la ide de desarrollar nuestro sistema con una sola entidad de liga, pero ese no es el funcionamiento real de una Liga Fantasy. En este tipo de aplicaciones los usuarios pueden disponer de diferentes equipos participando en diferentes ligas. Por lo tanto debíamos desarrollar un sistema que nos permitiese almacenar datos de tal forma que al consultarlos nos proporcionasen históricos, clasificaciones y relaciones de una forma estructurada que no interfiriese en la lógica de negocio pero que, a su vez, nos permitiese desarrollarla de forma correcta.

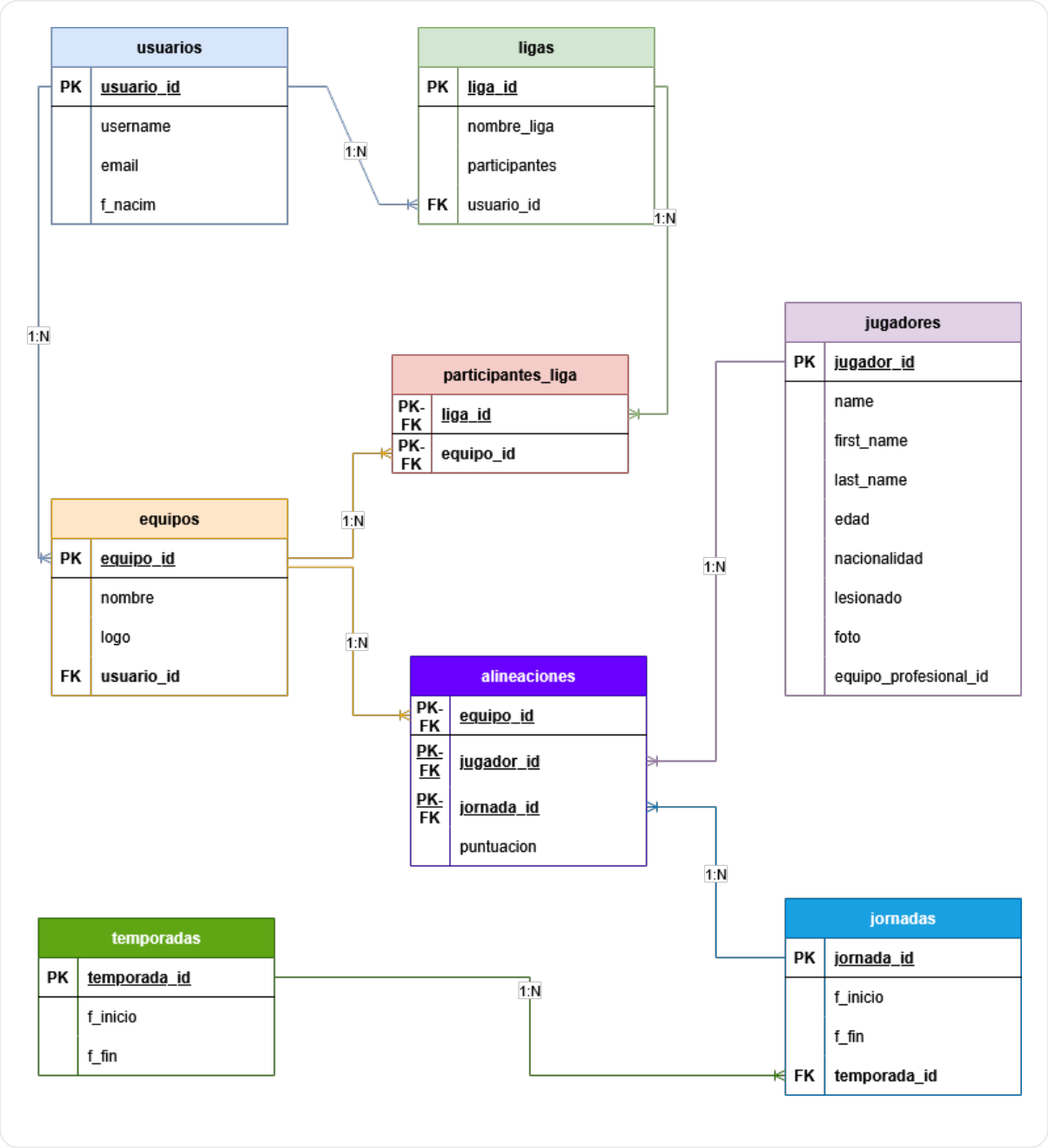


Figura 31 — Primer Diagrama Entidad Relación. Un boceto de lo que sería después.

11.1.1 EL DESARROLLO

En los primeros momentos, a la vez que se desarrollaban los wireframes, se diseñó una primera versión del modelo relacional que se iba a utilizar. Se buscó, en un primer momento, diseñar las entidades de la forma más sencilla posible, de forma que esto nos permitiese ir desarrollando nuestro sistema. Más adelante durante el desarrollo se irían añadiendo más

atributos y elementos que, aunque hicieron nuestro modelo más complejo y, a veces, más difícil de leer, también lo convertían en uno mucho más completo.

Aunque en apariencia no lo parece, el eje central de nuestro modelo, eran las ligas. Los usuarios se unen a ligas, los usuarios crean ligas. Cada liga tiene que gestionar de forma independiente el total de jugadores que participan en una liga, tanto los usuarios físicos, evitando que uno de ellos pueda tener dos equipos participantes en una liga, como los jugadores profesionales, haciendo que todos ellos estén disponibles en una liga, pero no pudiendo repetirse. De producirse alguno de los casos anteriores entraríamos en problemas de inconsistencia y duplicidad de los datos, lo que provocaría errores y situaciones no deseadas.

Para evitar estos casos, a la vez que íbamos puliendo nuestro modelo, debíamos de dotar a las tablas mediante el lenguaje SQL de herramientas que les permitiesen validar, ordenar e, incluso, impedir la grabación de datos en las tablas.

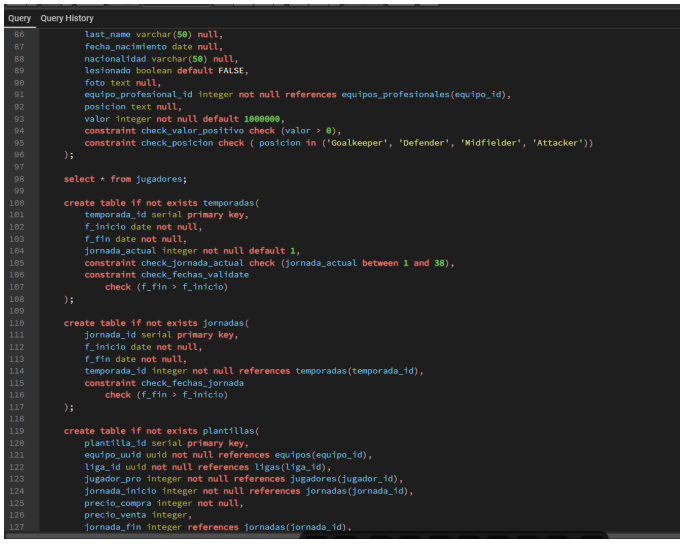


Figura 32 — Fragmento de nuestro Script.

Como las cosas se ven mucho mejor con un ejemplo, aquí tenemos un fragmento de nuestro script SQL. Como podemos observar en algunos de los campos nos aseguramos de que su valor no pueda ser nulo, de que tengan un valor por defecto para evitar valores nulos o 0. Y, una de las opciones más repetidas a lo largo de todo el script, es aquella en la que añadimos constraints y checks para validar que los datos que se tratan de insertar cumplan ciertas condiciones, como por ejemplo, que una fecha de fin no pueda ser anterior a un fecha

de inicio, que un valor numérico deba de ser mayor que 0 o que un campo de tipo texto solo pueda elegir uno de entre cuatro posibles valores.

Con todo ello conseguimos que nuestro modelo se adecue lo más posible a la lógica de nuestra aplicación. Si bien es cierto que en muchos desarrollos modernos, gran parte de este trabajo se le deja al backend, que se encarga de validar que datos pasan y cuales no, nosotros hemos optado por otro enfoque. Hemos considerado que nuestra aplicación, para asegurar su buen funcionamiento, no puede permitirse problemas en la estructura o validez de los datos. Por lo tanto, dentro de la propia Base de Datos, hemos añadido esta capa adicional de protección, que nos asegura estabilidad y fiabilidad. Además de esto, podemos estar tranquilos ya que, independientemente del backend que se conecte a nuestra base de datos, esta cuenta con sus propias medidas de protección para asegurar su integridad.

Hay que recalcar que PostgreSQL ofrece grandes herramientas para lograr este objetivo.

Una vez seguimos desarrollando, nuestro sistema se hace más complejo, vamos añadiendo más atributos a nuestras tablas y modificando los necesarios en caso de que encontremos algún error debido a la estructura que teníamos.

Cabe indicar, que el desarrollo de la base de datos se extendió desde el primer día hasta el último. Esto fue debido a cambios que tuvieron que realizarse para amoldarse a las necesidades de la aplicación, mejoras en la misma y a la naturaleza de los datos que obteníamos de las APIs externas, que en algunos momentos provocaron errores inesperados en nuestro sistema, al no amoldarse nuestro modelo a la estructura que poseían los datos que recibíamos para hidratar nuestro sistema.

Al ser una Liga Fantasy, que necesita de todo el plantel de jugadores profesionales de una liga de futbol, necesitábamos datos proporcionados por plataformas externas a nosotros, cuya estructura no podíamos controlar, por lo que hubo que llevar a cabo muchas operaciones por ensayo y error, hasta que se consiguió modelar esa estructura de datos de la mejor manera posible. Además de que hubo que filtrar gran parte de ellos, pues había algunos que no necesitábamos o que no eran relevantes para nosotros. Este paso se explicará más adelante al comentar una de las funcionalidades más importantes de nuestra aplicación.

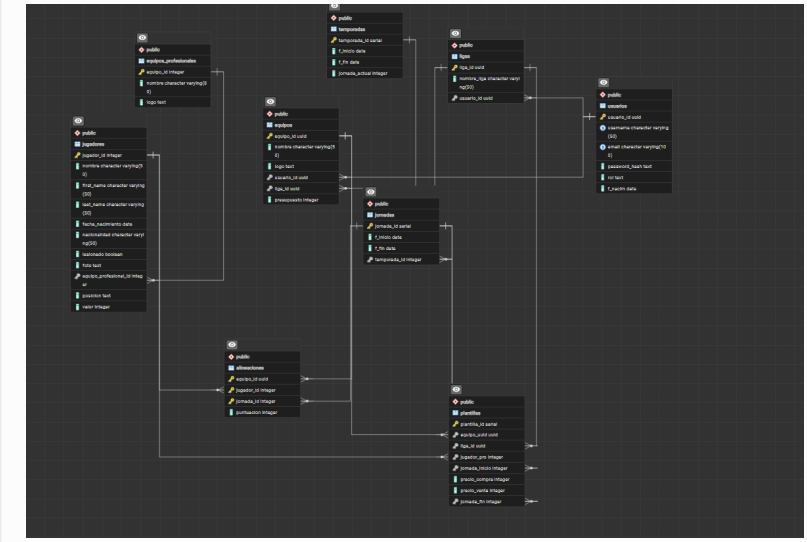


Diagrama final del modelo relacional en PostgreSQL.

Usando las herramientas propias de PostgreSQL, en este caso el reverse-engineering, una vez terminamos de modelar nuestra estructura, y estando satisfechos con ella, podemos mostrar el diagrama al completo.

En este caso se pueden ver las relaciones entre las diferentes entidades, como Ligas, Usuarios, Equipos, Jugadores, Alineaciones, Plantillas; además de los datos adicionales con respecto al diagrama inicial. Como ya se ha explicado anteriormente, la tabla jugadores_profesionales es la que sufrió mas cambios, de la misma forma que plantillas. Gracias a esos cambios podíamos acercarnos lo más posible al funcionamiento de otras aplicaciones de este tipo, pudiendo ofrecer datos históricos de resultados, puntuaciones de los jugadores, pertenencia de los jugadores (a que equipos y en que ligas), clasificaciones jornada a jornada, etcétera.

Con todos estos cambios, sin necesidad de la ayuda de nuestro backend, también nos asegurábamos de que se cumpliesen ciertas reglas a la hora de almacenar los datos. No podían existir dos equipos del mismo usuario en una misma liga, un jugador profesional no podía aparecer en diferentes equipos de una misma liga en un mismo periodo de tiempo, de la misma forma que no podía duplicarse en una misma liga. Se habían aprovisionado de la característica «Unique» a diferentes atributos dentro del sistema para evitar duplicados tales como el email, los nombres de los equipos creados por los usuarios, etcétera.

11.1.2 Evolución de nuestro modelo.

En el primer modelo tenemos prácticamente las mismas entidades que tendremos en la versión final. Pero siempre es buena idea de dar contexto al porque de las cosas. Vamos a ir explicando brevemente cada una de las entidades o tablas para que podamos entender como hemos estructurado nuestro proyecto. De todas formas, en este punto mostraremos como quedó cada una de las entidades finalmente para que podamos ver, comparándolo con la figura anterior la evolución de nuestro modelo relacional.

Tabla usuarios

SQL usuarios

```sql
CREATE TABLE IF NOT EXISTS usuarios (
    usuario_id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    rol TEXT NOT NULL DEFAULT 'user',
    f_nacim DATE NOT NULL,

    CONSTRAINT check_f_nacim_pasado
        CHECK (f_nacim <= CURRENT_DATE)
);
```

Estructura inicial de la entidad usuarios

En las primeras versiones del sistema, la tabla usuarios estaba formada únicamente por los atributos esenciales para identificar y autenticar a cada usuario dentro de la plataforma.

usuario_id — UUID · PRIMARY KEY

Identificador único del usuario dentro del sistema. Este atributo no sufrió modificaciones durante el desarrollo y permite relacionar al usuario con el resto de entidades de la base de datos de forma unívoca.

username — VARCHAR(50)

Nombre visible del usuario dentro de la plataforma. Se utiliza para identificar públicamente a cada participante una vez autenticado.

email — VARCHAR(100) · UNIQUE

Correo electrónico asociado a la cuenta. Además de utilizarse durante el inicio de sesión, actúa como identificador único para impedir registros duplicados.

f_nacim — DATE

Fecha de nacimiento del usuario. Aunque inicialmente su uso estaba orientado a futuras funcionalidades relacionadas con restricciones de edad o sistemas de suscripción, finalmente se mantuvo como parte de la estructura base del sistema.

Elementos añadidos durante la evolución del sistema

A medida que la aplicación evolucionó y comenzaron a implementarse funcionalidades más complejas, fue necesario ampliar la estructura inicial de la tabla para incorporar nuevos mecanismos relacionados con seguridad, permisos y validación de datos.

password_hash TEXT

Campo destinado al almacenamiento seguro de contraseñas cifradas. El backend aplica algoritmos de hash antes de persistir la información, evitando almacenar credenciales en texto plano.

rol TEXT · DEFAULT 'user'

Atributo utilizado para diferenciar usuarios estándar de administradores dentro de la plataforma. Este sistema permite controlar el acceso a funcionalidades específicas relacionadas con la gestión de la aplicación.

check_f_nacim_pasado CHECK CONSTRAINT

Restricción incorporada para impedir que se inserten fechas de nacimiento posteriores a la fecha actual. Esta validación garantiza coherencia e integridad en los datos almacenados.

Tabla ligas

SQL **Tabla ligas**

```sql
CREATE TABLE IF NOT EXISTS ligas (
    liga_id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    nombre_liga VARCHAR(50) NOT NULL,
    usuario_id UUID NOT NULL
        REFERENCES usuarios(usuario_id)
);
```

Descripción de la entidad ligas

La tabla ligas constituye uno de los núcleos principales del sistema, ya que representa el contenedor donde los usuarios crean y gestionan sus competiciones fantasy. Su diseño se ha mantenido estable desde las primeras fases del proyecto, ya que la relación usuario–liga estaba correctamente definida desde el inicio.

liga_id — UUID · PRIMARY KEY

Identificador único de cada liga. Permite referenciarla de forma unívoca desde el resto de entidades del sistema.

nombre_liga — VARCHAR(50)

Nombre asignado por el usuario creador. En futuras iteraciones se plantea añadir una restricción UNIQUE para evitar duplicados.

usuario_id — UUID · FOREIGN KEY

Identificador del usuario propietario de la liga. Establece la relación directa entre cada competición y su creador en la tabla usuarios.

SQL **Tabla equipos**

```sql
CREATE TABLE IF NOT EXISTS equipos (
    equipo_id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    logo TEXT,

    usuario_id UUID NOT NULL
        REFERENCES usuarios(usuario_id),

    liga_id UUID NOT NULL
        REFERENCES ligas(liga_id),

    presupuesto INTEGER NOT NULL DEFAULT 10000000,

    CONSTRAINT check_presupuesto_positivo
        CHECK (presupuesto >= 0),

    CONSTRAINT unique_usuario_liga
        UNIQUE(usuario_id, liga_id)
);
```

Diseño funcional de la entidad equipos

La tabla equipos resuelve dos problemas clave del sistema: la participación de usuarios en competiciones y la restricción de unicidad de equipos por usuario dentro de una misma liga.

Inicialmente esta funcionalidad estaba dividida en dos entidades separadas, pero fue unificada para optimizar consultas y simplificar la lógica de negocio.

equipo_id — UUID

Identificador único del equipo dentro del sistema.

nombre — VARCHAR(50)

Nombre asignado por el usuario. Se plantea su futura restricción UNIQUE.

logo — TEXT

URL del logotipo del equipo. En futuras versiones podría migrarse a almacenamiento de ficheros o SVG.

usuario_id · liga_id — FOREIGN KEYS

Relación que vincula el equipo con su creador y la liga en la que participa.

presupuesto — INTEGER · DEFAULT 10000000

Presupuesto inicial del equipo para operaciones de fichajes dentro del sistema fantasy.

Integración de datos externos (equipos y jugadores profesionales)

La estructura de estas tablas está condicionada por la API externa utilizada para importar datos oficiales de LaLiga.

Esto implicó múltiples iteraciones debido a inconsistencias en los datos recibidos, especialmente valores nulos o formatos no homogéneos.

equipos_profesionales INTEGER PRIMARY KEY

Identificador proveniente de la API externa. Se mantiene el formato original para garantizar compatibilidad.

jugadores INTEGER PRIMARY KEY

Entidad principal de jugadores profesionales. Incluye validaciones de valor, posición y relaciones con equipos.

SQL Tabla plantillas

```sql
CREATE TABLE IF NOT EXISTS plantillas (
    plantilla_id SERIAL PRIMARY KEY,
    equipo_uuid UUID NOT NULL REFERENCES equipos(equipo_id),
    liga_id UUID NOT NULL REFERENCES ligas(liga_id),
    jugador_pro INTEGER NOT NULL REFERENCES jugadores(jugador_id),
    jornada_inicio INTEGER NOT NULL REFERENCES jornadas(jornada_id),
    precio_compra INTEGER NOT NULL,
    precio_venta INTEGER,
    jornada_fin INTEGER REFERENCES jornadas(jornada_id),
    CONSTRAINT check_precio_compra CHECK (
        precio_compra >= 0 AND
        (precio_venta IS NULL OR precio_venta >= 0)
    ),
    CONSTRAINT check_jornada_fin CHECK (
        jornada_fin IS NULL OR jornada_fin > jornada_inicio
    )
);
```

Modelo de propiedad de jugadores en ligas fantasy

Esta tabla representa el núcleo del sistema fantasy, gestionando la propiedad temporal de jugadores dentro de cada liga.

El modelo permite que un mismo jugador exista en múltiples ligas simultáneamente, pero nunca duplicado dentro de la misma liga activa.

Se introduce un índice único parcial para garantizar integridad:

```
CREATE UNIQUE INDEX unique_jugador_por_liga
ON plantillas (liga_id, jugador_pro)
WHERE jornada_fin IS NULL;
```

Este mecanismo permite controlar la disponibilidad de jugadores en tiempo real dentro de cada competición.

SQL **Tabla alineaciones**

```
CREATE TABLE IF NOT EXISTS alineaciones (
    equipo_id UUID NOT NULL REFERENCES equipos(equipo_id),
    jugador_id INTEGER NOT NULL REFERENCES jugadores(jugador_id),
    jornada_id INTEGER NOT NULL REFERENCES jornadas(jornada_id),
    puntuacion INTEGER NOT NULL,
    PRIMARY KEY (equipo_id, jugador_id, jornada_id),
    CONSTRAINT check_puntuacion_notlesszero CHECK (puntuacion >= 0)
);
```

Sistema de puntuación por jornada

La tabla alineaciones registra qué jugadores han sido seleccionados por cada usuario en cada jornada, junto con su rendimiento obtenido.

Se utiliza una clave primaria compuesta para evitar duplicidades y garantizar coherencia temporal.

SQL **Tablas temporadas y jornadas**

```sql
CREATE TABLE IF NOT EXISTS temporadas (
    temporada_id SERIAL PRIMARY KEY,
    f_inicio DATE NOT NULL,
    f_fin DATE NOT NULL,
    jornada_actual INTEGER NOT NULL DEFAULT 1,
    CONSTRAINT check_jornada_actual CHECK (jornada_actual BETWEEN 1 AND 38),
    CONSTRAINT check_fechas_validate CHECK (f_fin > f_inicio)
);

CREATE TABLE IF NOT EXISTS jornadas (
    jornada_id SERIAL PRIMARY KEY,
    f_inicio DATE NOT NULL,
    f_fin DATE NOT NULL,
    temporada_id INTEGER NOT NULL REFERENCES temporadas(temporada_id),
    CONSTRAINT check_fechas_jornada CHECK (f_fin > f_inicio)
);
```

Gestión de calendario competitivo

El modelo de jornadas y temporadas permite gestionar el calendario deportivo completo del sistema.

Se introduce explícitamente el campo jornada_actual para evitar inconsistencias derivadas del comportamiento incremental de SERIAL.

Esto permite mantener coherencia histórica entre temporadas y jornadas.

11.2 El backend

Para desarrollar nuestro backend nos basamos en los estándares modernos. Modularidad, responsabilidad única y escalabilidad. Debíamos desarrollar cada una de las partes para que fuese independiente, que se encargase solamente de hacer una cosa, pero bien hecha y de que fuese fácilmente mantenible y escalable en caso de ser necesario. Eso conlleva que, en lugar de archivos y fragmentos de código mastodónticos, tengamos un mayor número de ellos pero de un tamaño más reducido. Una foto ilustrará mejor de lo que estamos hablando.

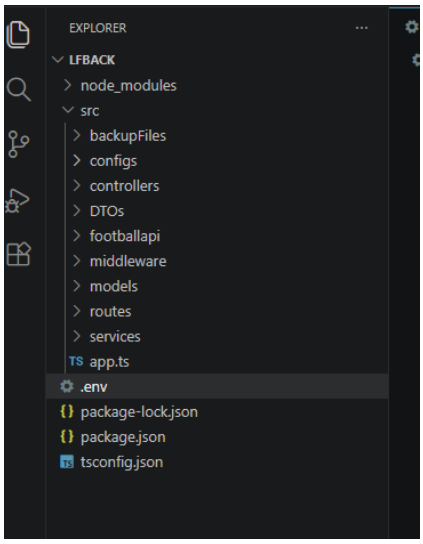
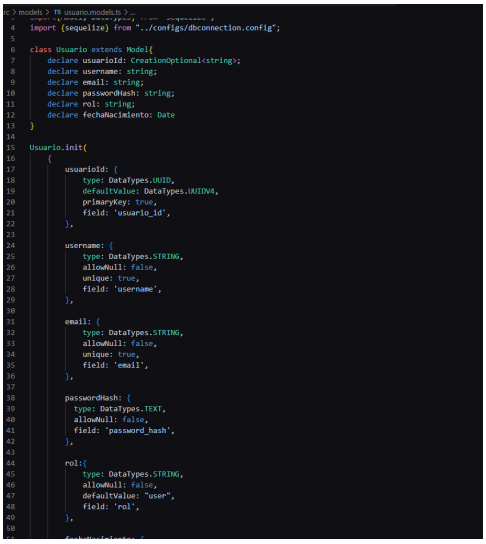


Figura 33 — Estructura básica del proyecto del Backend

Como podemos ver en la imagen anterior, hemos decidido dividir nuestro código y sus archivos según tipo y función. Estos archivos van desde aquellos que controlan el acceso a la base de datos, como los controllers y los servicios, los que se encargan de la seguridad e integridad de los datos, como el middleware, los models y los DTOs, y aquellos que se encargan de comunicarlo todo, como puede ser el archivo principal «app.ts», los archivos de rutas en routes, los archivos de configuración e incluso aquellos que se encargan de la comunicación con APIs externas.

Para el caso del backend lo primero que se hizo fue crear la estructura básica y luego, una vez tuvimos el modelo inicial de la base de datos, fue codificar los models que nos servirían para comunicar nuestro backend con nuestra BD.



```
import {sequelize} from "../configs/dbconnection.config";

class Usuario extends Model{
    declare usuarioId: CreationOptional<string>;
    declare username: string;
    declare email: string;
    declare passwordHash: string;
    declare rol: string;
    declare fechaNacimiento: Date
}

Usuario.init(
    {
        usuarioId: {
            type: DataTypes.UUID,
            defaultValue: DataTypes.UUIDV4,
            primaryKey: true,
            field: 'usuario_id',
        },

        username: {
            type: DataTypes.STRING,
            allowNull: false,
            unique: true,
            field: 'username',
        },

        email: {
            type: DataTypes.STRING,
            allowNull: false,
            unique: true,
            field: 'email',
        },

        passwordHash: {
          type: DataTypes.TEXT,
          allowNull: false,
          field: 'password_hash',
        },

        rol:{
            type: DataTypes.STRING,
            allowNull: false,
            defaultValue: "user",
            field: 'rol',
        },
```

Figura 34 — Ejemplo de Clase o Model.

Como vemos en la foto anterior, este mismo nos sirve como ejemplo, debemos de dar una estructura a nuestros modelos o clases para que coincidan con las entidades de nuestra base de datos. Con ello, Sequelize, nuestro ORM o encargado de la comunicación con la base de datos, puede transferir los datos de forma bidireccional de una forma mucho más fácil y segura. Para modelarlos de forma correcta, primero definimos todas las propiedades de una clase y luego le indicamos las mismas propiedades que tienen en la base de datos incluido el nombre. Así, nuestro ORM, sabe exactamente a donde se tiene que dirigir a la hora de leer y guardar los datos. Esto se debe hacer para todas las entidades de nuestra base de datos, indicando las relaciones entre las diferentes tablas, ya que eso, además de ser mandatorio de el modelo, ayuda a un mapeo mucho más fácil de las diferentes entidades. Cabe destacar que, de la misma forma que el modelo Entidad-Relación, los modelos han sufrido variaciones hasta el último momento. Cada vez que algo cambiaba en la BD debía de cambiar también en el backend.



```
import bcrypt from "bcrypt";

import jwt from "jsonwebtoken";
import {loginService} from "../services/login.service";

// Por ahora el registro y el login van aquí. Más adelante posiblemente será más recomendable dejarlo en su propio routes y controller

export async function registrarNuevoUsuario(req: Request, res: Response) {
    try {
        const { username, email, password} = req.body;
        const fechaNacimiento: string = res.locals.fechaNacimiento;

        const hash = await bcrypt.hash(password, 10);

        const existe = await Usuario.findOne({
            where: {
                [Op.or]: [{ username }, { email }]
            }
        });

        if (existe) {
            return res.status(400).json({
                error: 'El username o email ya existen'
            });
        }


        const usuarioCreado = await Usuario.create({
            username,
            email,
            passwordHash: hash,
            fechaNacimiento,
        });

        return res.status(201).json(`Nuevo usuario creado ${usuarioCreado.username}`);

    } catch (error: any) {
        console.error('ERROR AL CREAR USUARIO:', error);

        return res.status(500).json({
            error: error.original?.message || error.message
        });
    }
}
```

Figura 35 — Controller en Node.js

Para comunicar nuestro backend con la base de datos hacemos uso de controllers o controladores y servicios específicos que transfieren, filtran y limpian los datos para su correcta transferencia entre unos sistemas y otros. En la foto de arriba, como ejemplo, mostramos el código referente a la función que usamos para registrar usuarios nuevos en nuestro sistema. En esta función nos encargamos de validar que se han mandado bien los datos, que se hashea correctamente el campo password, que los datos se transfieren correctamente a la base de datos, que no existen usuarios con los mismos datos y, en caso de error, se devuelven los mensajes pertinentes a los usuarios para que estén al tanto de lo que ocurre con sus operaciones. En algunos momentos estos controladores llaman a servicios específicos que hemos creado para que desarrollen una tarea específica. Esto nos ayuda a tener un código más limpio y más manejable.

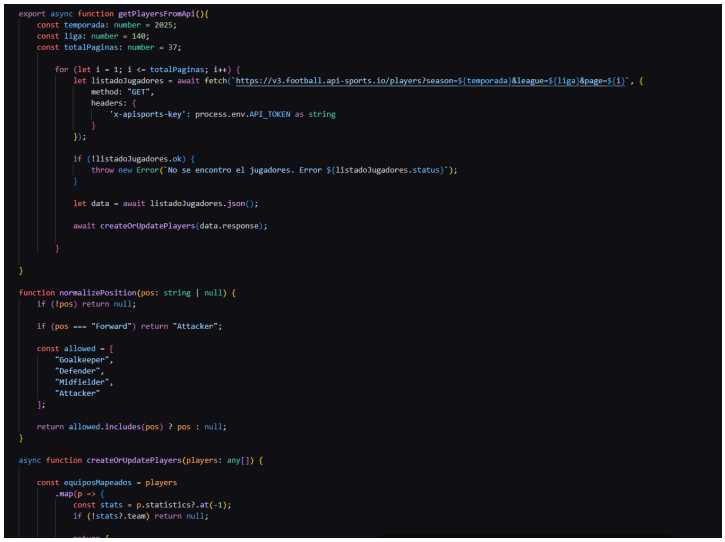


Figura 36 — Estructura y código de un Service.

Como podemos ver en la ilustración anterior, vemos el código de un servicio, el cual se encarga de llamar a la API externa que nos proporciona los datos de los jugadores, filtra y limpia esos datos para que tengan la forma correcta, paran la operación en caso de que encuentren algún error y, por último, realizan una inserción masiva de esos datos en la base de datos.

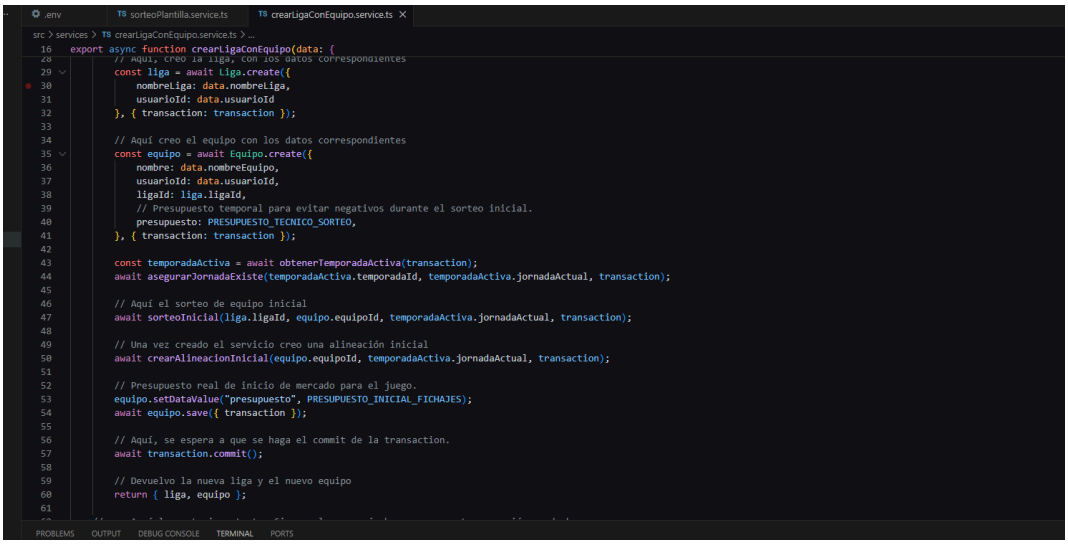


Figura 37 — Código de otro servicio para realizar diferentes funciones.

En algunos casos, varias operaciones de los endpoints se realizan unas seguidas detrás de otras, lo que implica llamar en una secuencia a diferentes funciones de los diferentes controllers. En el ejemplo que vemos, cuando se crea una liga por un usuario, a su vez el usuario debe de crear su equipo. Acto seguido se debe de realizar un sorteo aleatorio para otorgarle una plantilla y alineaciones iniciales. Para casos como ese, la primera función que es llamada puede llamar a un servicio como este. Esta clase de servicios se dedican a llamar en orden a todas las partes implicadas hasta que se han realizado todas con éxito. En caso contrario se devolverá un código de error y un mensaje al usuario, informándole de que algo no ha ido como se esperaba.

Para no permitir que se envíen datos incorrectos y para evitar realizar operaciones innecesarias que sabemos que son incorrectas usamos lo que se llaman middlewares. Los middlewares, o validadores, son una capa intermedia que se dedican a comprobar que se hayan mandado los datos requeridos, que su contenido sea el esperado o que tengan formatos concretos.

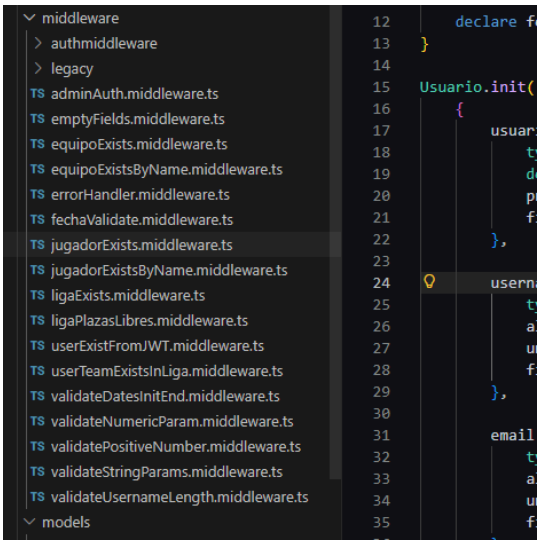


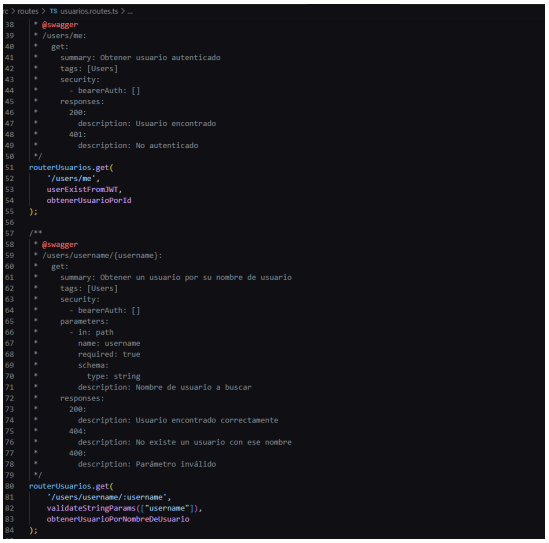
Figura 38 — Ejemplo de Middleware

La ilustración anterior nos da una idea de las validaciones que se llevan a cabo, para seguridad del sistema y para no realizar operaciones excesivas de lectura y escritura en la base de datos. Cada uno de esos archivos es una función concreta que se dedica a validar un tipo de dato concreto. Como ejemplos podemos ver funciones que validan la longitud de un username,

que un número sea positivo, que las fechas de inicio y fin de una jornada o una temporada tengan un formato concreto y que la fecha de fin no sea anterior a la de inicio, etcétera.

Node.js, de la misma forma que express.js, ofrecen frameworks, como express-validator que podrían hacer más fácil validar los datos. En nuestro caso, por la falta de experiencia en el uso de estas herramientas, optamos por hacerlo a mano. Aunque parezcan muchos archivos, su código no es muy extenso y, muchos de ellos, se pueden reutilizar varias veces, pues hay operaciones que requieren el mismo tipo de datos, lo que conlleva que utilicen el mismo validador.

Por último, vamos a poner ejemplos de lo que son nuestros archivos estructurales. Aquellos que nos sirven para dar forma a nuestro proyecto, funcionalidades extra y a gestionar los flujos de trabajo del mismo.



```
> routes > TS usuarios.routes.ts > ...
 * @swagger
 * /users/me:
 *   get:
 *     summary: Obtener usuario autenticado
 *     tags: [Users]
 *     security:
 *       - bearerAuth: []
 *     responses:
 *       200:
 *         description: Usuario encontrado
 *       401:
 *         description: No autenticado
 */
routerUsuarios.get(
    '/users/me',
    userExistFromJWT,
    obtenerUsuarioPorId
);

/**
 * @swagger
 * /users/username/{username}:
 *   get:
 *     summary: Obtener un usuario por su nombre de usuario
 *     tags: [Users]
 *     security:
 *       - bearerAuth: []
 *     parameters:
 *       - in: path
 *         name: username
 *         required: true
 *         schema:
 *           type: string
 *         description: Nombre de usuario a buscar
 *     responses:
 *       200:
 *         description: Usuario encontrado correctamente
 *       404:
 *         description: No existe un usuario con ese nombre
 *       400:
 *         description: Parámetro inválido
 */
routerUsuarios.get(
    '/users/username/:username',
    validateStringParams(["username"]),
    obtenerUsuarioPorNombreDeUsuario
);
```

Figura 39 — Archivos de Rutas

En este ejemplo tenemos un archivo que define las diferentes rutas que indicaremos en nuestra barra de direcciones del navegador para acceder a las diferentes funcionalidades. Cada endpoint o ruta va acompañado también de los middleware que va a utilizar y de la función correspondiente del controlador correspondiente necesaria para realizar sus tareas. Las líneas de comentarios que se ven en este caso vienen dadas de otra funcionalidad que añadimos en nuestro sistema. Esta funcionalidad es la documentación mediante el estándar Open-API. Open-API nos da una serie de directrices que nos ayudan a documentar nuestra aplicación y a explicar su flujo de trabajo. Para ello nos ayudamos de otra herramienta llamada

Swagger, esta nos proporciona una herramienta visual, que nos permite ver esa documentación a través de nuestro navegador. Además, es una vista funcional, es decir, nos permite realizar operaciones reales, sobre nuestro backend y nuestra base de datos, con lo que así podemos disfrutas de otra herramienta de pruebas para testear y comprobar la funcionalidad de nuestra aplicación.

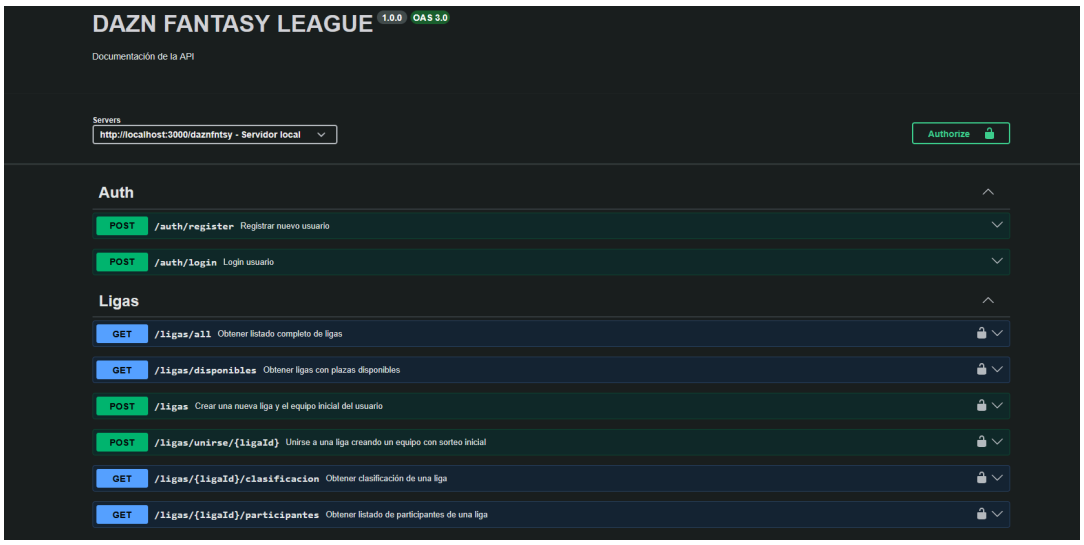


Figura 40 — Interfaz de Swagger-UI en nuestro navegador.

Como vemos en esta imagen, la documentación de Swagger nos proporciona de manera visual una vista detallada de la estructura de nuestro backend. En este caso podemos ver los diferentes endpoints que conforman nuestra aplicación mientras que el símbolo del candado nos indica que endpoints son de acceso público, los que no lo tienen, y cuales son de dominio privado y necesitan de autorización mediante JWT (jsonwebtoken).

JWT es otra herramienta, en este caso un estándar de seguridad, que en caso de que los endpoints los soliciten hace obligatorio que el usuario disponga de uno de estos tokens que le certifiquen como usuario autenticado y autorizado en nuestro sistema. En el apartado del frontend veremos una parte que dedicamos a la gestión de este elemento, pues es el backend es que lo proporciona y lo solicita, pero es el usuario final el que está en posesión del mismo en el navegador para poder identificarse.

Para comprobar el funcionamiento de la herramienta Swagger vamos a añadir dos ilustraciones más.

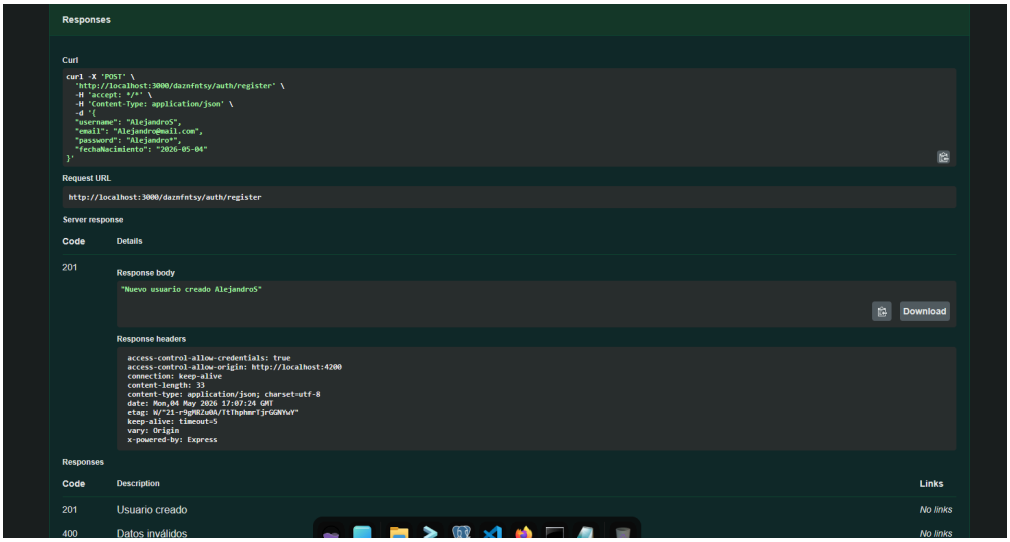


Figura 41 — Creación de usuario mediante Swagger-UI

Usando Swagger a través del navegador, proporcionándole los datos que se solicitan para crear un usuario, podemos ejecutar esa instrucción. Como vemos en la imagen hemos creado dicho usuario. De arriba hacia abajo podemos ver el código de la petición con los datos que adjuntamos, la URL a la que nos dirigimos para hacer dicha petición y el resultado, con el código de estado HTTP que nos devuelve la petición, en este caso 201 usuario creado, el mensaje que nos devuelve el backend al crear un usuario de forma exitosa e información adicional referente a la petición.

Si queremos podemos comprobar esta operación en nuestra base de datos ejecutando un SELECT * FROM usuarios.

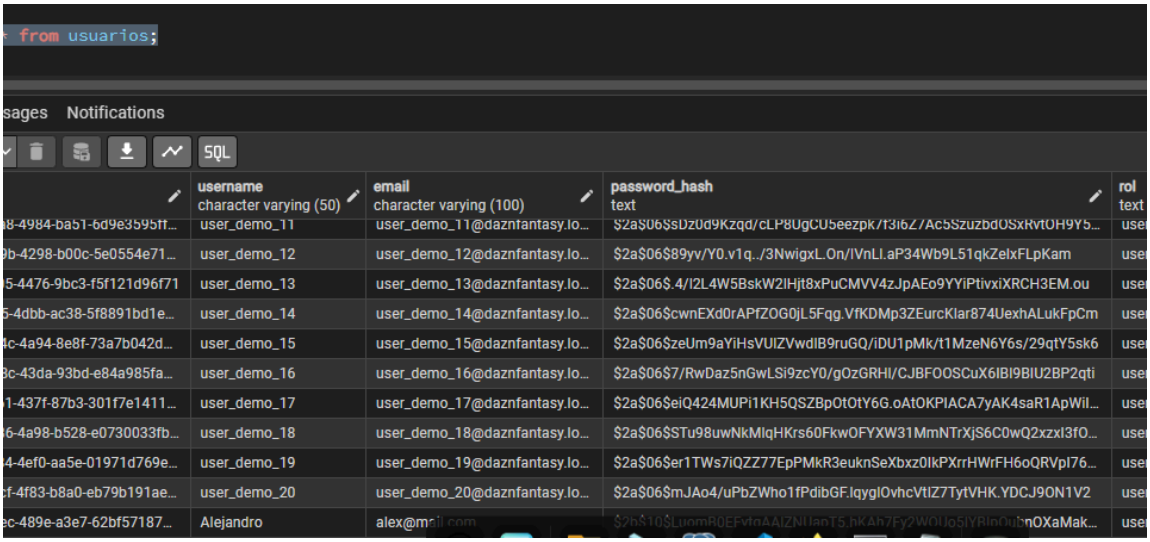


Figura 42 — Usuario Creado

En la parte inferior de la imagen podemos ver los datos del usuario que hemos creado. Vemos un UUID, identificador único generado por la base de datos, el username o nombre de usuario, el email, el password una vez se le ha aplicado el hasheado, el rol del usuario y la fecha de nacimiento.

Ahora, para terminar de ver la funcionalidad en Swagger, vamos a proceder a realizar un login desde el navegador.



Figura 43 — Login a través de Swagger

En la imagen anterior vemos los datos que le podemos pasar al backend, un username en formato string y un password en el mismo formato. De hacerse correctamente nos devuelve un código 200, petición correcta, y además nos indica que este proceso devuelve un token JWT.

Con este último paso podemos dar por finalizada la explicación del desarrollo de todas las partes de nuestro backend. Como hemos visto, hemos tratado de aplicar la mayor seguridad posible a nuestro backend, además de todas las funcionalidades necesarias, incluida la gestión de la documentación de la misma, que ayudará en temas de accesibilidad. Todo ello a través de los principios ya mencionados, modularidad, responsabilidad única y escalabilidad.

11.3 El frontend

Para desarrollar el frontend de nuestra aplicación, como ya explicamos en secciones anteriores, hemos decidido usar Angular. Para trabajar con este framework, hemos seguido los mismos principios que con el backend, teniendo en cuenta que son los mismos sobre los que se basa el desarrollo en Angular.

Por lo tanto, nos vamos a encontrar con una estructura muy similar. Divididas en diferentes componentes que se agrupan por su tipo y función.

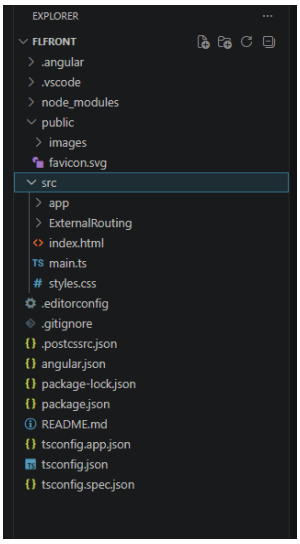


Figura 44 — Estructura básica del proyecto.

En este caso contamos con una serie de archivos de configuración y de gestión de dependencias. Dentro de la carpeta src encontraremos el núcleo de nuestra aplicación.

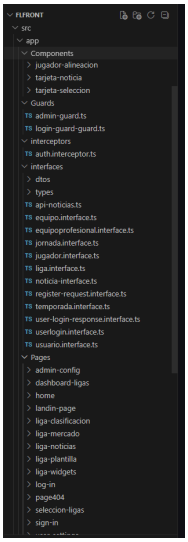


Figura 45 — Estructura extendida del proyecto en Angular

En la imagen anterior hemos visto como, al desplegar la carpeta src, se obtiene la visibilidad de todo el resto de componentes de nuestra aplicación.

En el caso que nos ocupa, hemos dividido la estructura en:

- **Componentes:** Los elementos visuales más pequeños que podemos encontrar dentro de una página, como botones, cards, banners, modales, etcétera.
- **Guardas e Interceptores:** Se encargan de proteger las diferentes rutas de nuestra aplicación y de modificar las cabeceras de las peticiones HTTP, en las que se debe de incluir el token JWT proporcionado por el backend para poder identificarse y autorizarse para usar la aplicación.
- **Interfaces, DTOs y Types:** Son el equivalente a los modelos de nuestro backend. Por norma general se deben de modelar siguiendo la misma estructura que se tiene en el backend, que a su vez toma esa estructura de la base de datos. Eso hace que mantengamos unos estándares de integridad a la hora de transferir los datos y nos ayuda a evitar errores al saber siempre que tipos de datos esperamos enviar o recibir.
- **Pages:** Las estructuras más grandes de nuestro proyecto en cuanto a HTML se refieren. Suelen ser landing pages, login y register pages y otro tipo de elementos que ayudan a la navegación y al uso de nuestra aplicación.
- **Services:** Tienen la misma función que en el backend. Elementos independientes que se especializan en realizar tareas específicas que pueden ser usadas por cualquiera del resto de componentes.

- **Archivos de rutas y de configuración:** Los primeros nos indican cual es la estructura en la que está dividida nuestra página, sus secciones, y quienes pueden acceder a esas secciones con ayuda de las guardas. Los archivos de configuración, como en otras partes de este proyecto nos ayudan añadiendo nuevas herramientas necesarias para el desarrollo o que herramientas del propio framework se van a utilizar. En este caso podemos englobar Tailwind que es el framework de CSS y estilos que vamos a utilizar para dar estructura y color a nuestra página.



Figura 46 — Interfaces o modelos

En este caso, para mostrar otro de sus usos, vemos el modelado de diferentes tipos de datos con los que esperamos trabajar. En el backend mostramos un modelo que lo que hace es reflejar las entidades de la base de datos. En el que nos ocupa ahora mostramos la estructura de datos que esperamos enviar o recibir al realizar una operación de registro de un usuario. Una interface permite que establezcamos un contrato en el que indicamos que una serie de datos deben de tener estructura y tipo concretos, así nos podemos asegurar que todos los datos que se van a enviar siguen el patrón correcto y son lo que nuestro backend espera que sean.

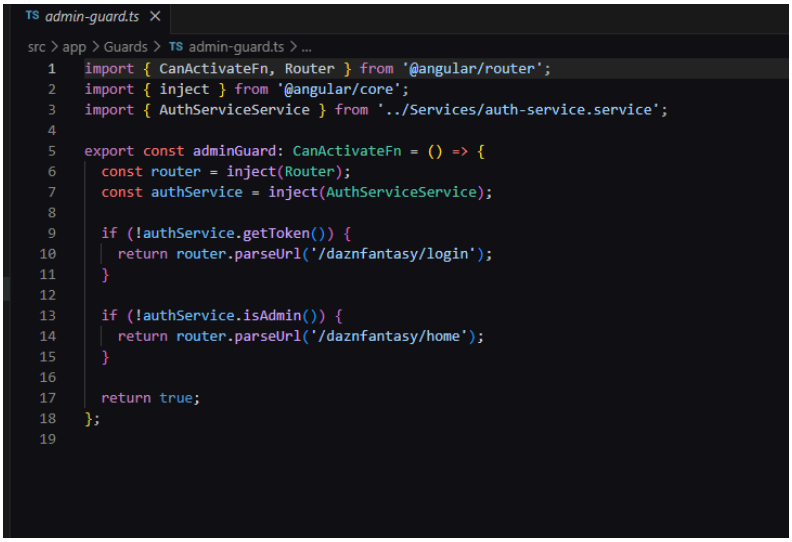


Figura 47 — Guarda de Administrador



Figura 48 — Guarda Usuario Regular

Para asegurarnos que los usuarios no naveguen a zonas no habilitadas para ellos y proteger nuestra aplicación de usos malintencionados usamos las guardas.

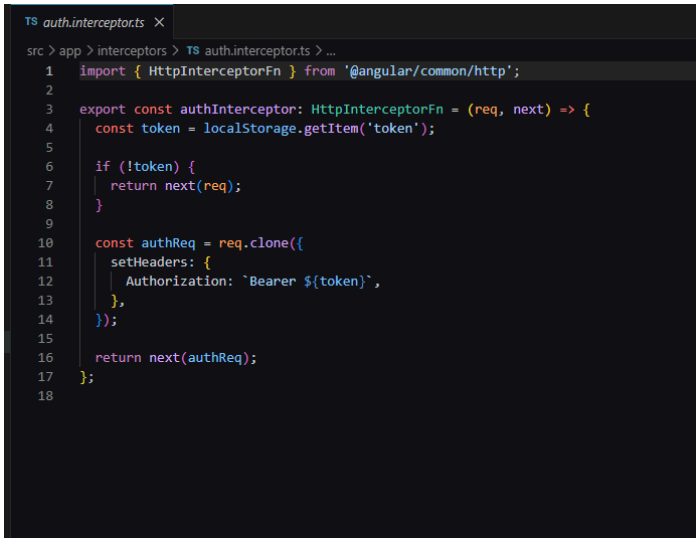


Figura 49 — Código de un interceptor.

Estos archivos se llaman así porque interceptan las diferentes peticiones que se hacen desde nuestra plataforma y modifican las cabeceras de las peticiones para añadirles el token de autenticación necesario.

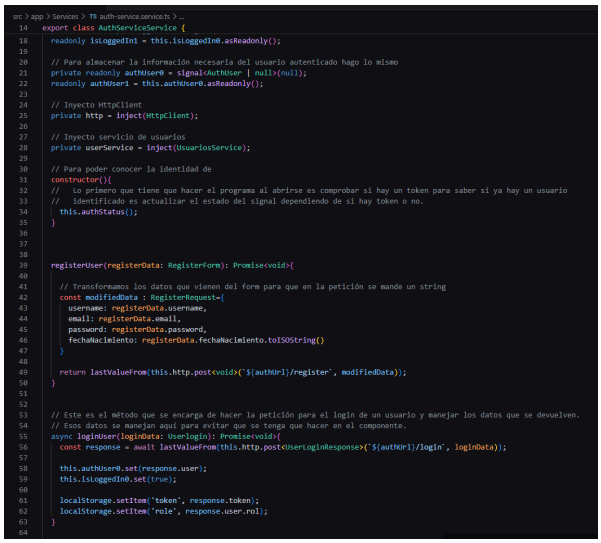
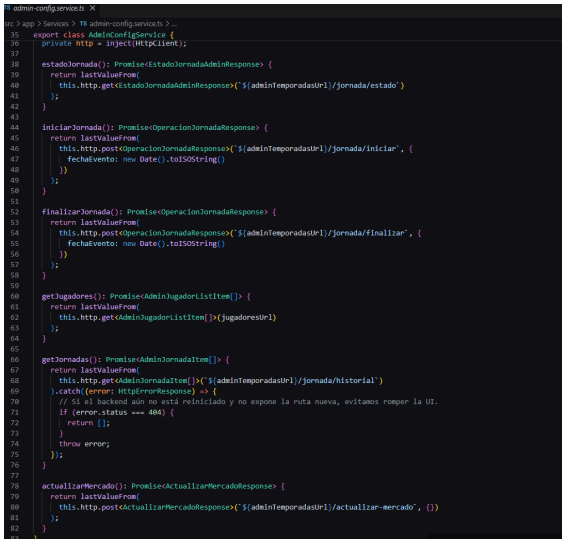


Figura 50 — Ejemplo de servicio en Angular

En esta imagen podemos ver un ejemplo del código que se encarga de gestionar procesos como los de registro y login de un usuario.



```
admin-config.service.ts
src > app > Services > TS admin-config.service.ts > ...
  export class AdminConfigService {
    private http = inject(HttpClient);

    estadoJornada(): Promise<EstadoJornadaAdminResponse> {
      return lastValueFrom(
        this.http.get<EstadoJornadaAdminResponse>(`${adminTemporadasUrl}/jornada/estado`)
      );
    }

    iniciarJornada(): Promise<OperacionJornadaResponse> {
      return lastValueFrom(
        this.http.post<OperacionJornadaResponse>(`${adminTemporadasUrl}/jornada/iniciar`, {
          fechaEvento: new Date().toISOString()
        })
      );
    }

    finalizarJornada(): Promise<OperacionJornadaResponse> {
      return lastValueFrom(
        this.http.post<OperacionJornadaResponse>(`${adminTemporadasUrl}/jornada/finalizar`, {
          fechaEvento: new Date().toISOString()
        })
      );
    }

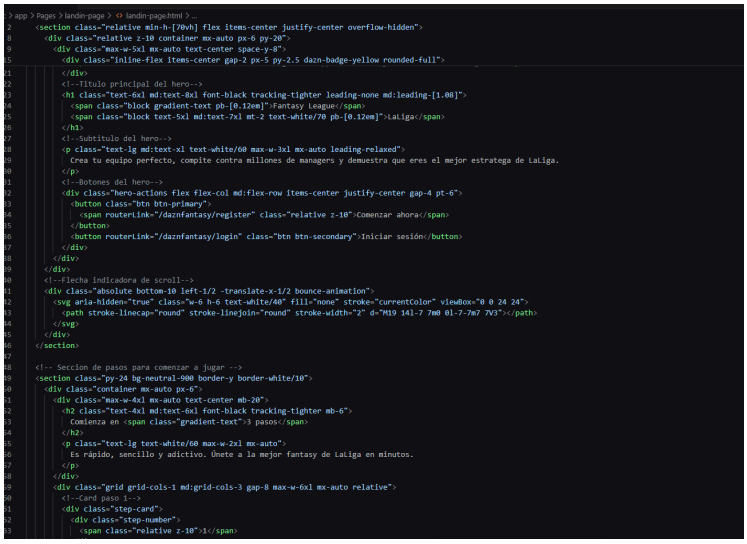
    getJugadores(): Promise<AdminJugadorListItem[]> {
      return lastValueFrom(
        this.http.get<AdminJugadorListItem[]>(jugadoresUrl)
      );
    }

    getJornadas(): Promise<AdminJornadaItem[]> {
      return lastValueFrom(
        this.http.get<AdminJornadaItem[]>(`${adminTemporadasUrl}/jornada/historial`)
      ).catch((error: HttpErrorResponse) => {
        // Si el backend aún no está reiniciado y no expone la ruta nueva, evitamos romper la UI.
        if (error.status === 404) {
          return [];
        }
        throw error;
      });
    }

    actualizarMercado(): Promise<ActualizarMercadoResponse> {
      return lastValueFrom(
        this.http.post<ActualizarMercadoResponse>(`${adminTemporadasUrl}/actualizar-mercado`, {})
      );
    }
```

Figura 51 — Ejemplo de servicio de administrador.

Para poder mostrar otro ejemplo de servicios, en este caso, nos vamos a centrar en aquellos propios de un administrador.

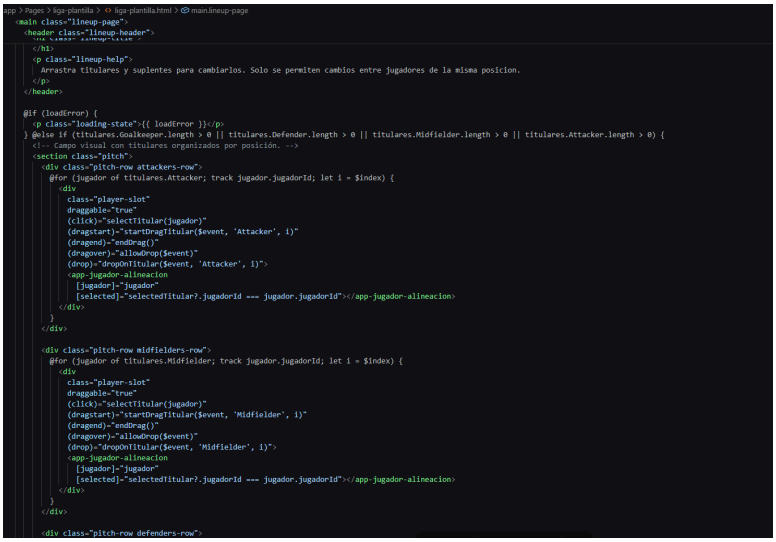


```
app > Pages > landin-page > <> landin-page.html > ...
<section class="relative min-h-[70vh] flex items-center justify-center overflow-hidden">
  <div class="relative z-10 container mx-auto px-6 py-20">
    <div class="max-w-5xl mx-auto text-center space-y-8">
      <div class="inline-flex items-center gap-2 px-5 py-2.5 dazn-badge-yellow rounded-full">
      </div>
      <!--Titulo principal del hero-->
      <h1 class="text-6xl md:text-8xl font-black tracking-tighter leading-none md:leading-[1.08]">
        <span class="block gradient-text pb-[0.12em]">Fantasy League</span>
        <span class="block text-5xl md:text-7xl mt-2 text-white/70 pb-[0.12em]">LaLiga</span>
      </h1>
      <!--Subtitulo del hero-->
      <p class="text-lg md:text-xl text-white/60 max-w-3xl mx-auto leading-relaxed">
        Crea tu equipo perfecto, compite contra millones de managers y demuestra que eres el mejor estratega de LaLiga.
      </p>
      <!--Botones del hero-->
      <div class="hero-actions flex flex-col md:flex-row items-center justify-center gap-4 pt-6">
        <button class="btn btn-primary">
          <span routerLink="/daznfantasy/register" class="relative z-10">Comenzar ahora</span>
        </button>
        <button routerLink="/daznfantasy/login" class="btn btn-secondary">Iniciar sesión</button>
      </div>
    </div>
  </div>
  <!--Flecha indicadora de scroll-->
  <div class="absolute bottom-10 left-1/2 -translate-x-1/2 bounce-animation">
    <svg aria-hidden="true" class="w-6 h-6 text-white/40" fill="none" stroke="currentColor" viewBox="0 0 24 24">
      <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 14l-7 7m0 0l-7-7m7 7V3"></path>
    </svg>
  </div>
</section>

<!-- Seccion de pasos para comenzar a jugar -->
<section class="py-24 bg-neutral-900 border-y border-white/10">
  <div class="container mx-auto px-6">
    <div class="max-w-4xl mx-auto text-center mb-20">
      <h2 class="text-4xl md:text-6xl font-black tracking-tighter mb-6">
        Comienza en <span class="gradient-text">3 pasos</span>
      </h2>
      <p class="text-lg text-white/60 max-w-2xl mx-auto">
        Es rápido, sencillo y adictivo. Únete a la mejor fantasy de LaLiga en minutos.
      </p>
    </div>
    <div class="grid grid-cols-1 md:grid-cols-3 gap-8 max-w-6xl mx-auto relative">
      <!--Card paso 1-->
      <div class="step-card">
        <div class="step-number">
          <span class="relative z-10">1</span>
```

Figura 52 — Fragmento de código html de la landing page

En la imagen superior vemos un ejemplo de como hemos estructurado nuestra página.



```
app > Pages > liga-plantilla > liga-plantilla.html > main.lineup-page
<main class="lineup-page">
  <header class="lineup-header">
    </h1>
    <p class="lineup-help">
      Arrastra titulares y suplentes para cambiarlos. Solo se permiten cambios entre jugadores de la misma posición.
    </p>
  </header>

  @if (loadError) {
    <p class="loading-state">{{ loadError }}</p>
  } @else if (titulares.Goalkeeper.length > 0 || titulares.Defender.length > 0 || titulares.Midfielder.length > 0 || titulares.Attacker.length > 0) {
    <!-- Campo visual con titulares organizados por posición. -->
    <section class="pitch">
      <div class="pitch-row attackers-row">
        @for (jugador of titulares.Attacker; track jugador.jugadorId; let i = $index) {
          <div
            class="player-slot"
            draggable="true"
            (click)="selectTitular(jugador)"
            (dragstart)="startDragTitular($event, 'Attacker', i)"
            (dragend)="endDrag()"
            (dragover)="allowDrop($event)"
            (drop)="dropOnTitular($event, 'Attacker', i)">
            <app-jugador-alineacion
              [jugador]="jugador"
              [selected]="selectedTitular?.jugadorId === jugador.jugadorId"></app-jugador-alineacion>
          </div>
        }
      </div>

      <div class="pitch-row midfielders-row">
        @for (jugador of titulares.Midfielder; track jugador.jugadorId; let i = $index) {
          <div
            class="player-slot"
            draggable="true"
            (click)="selectTitular(jugador)"
            (dragstart)="startDragTitular($event, 'Midfielder', i)"
            (dragend)="endDrag()"
            (dragover)="allowDrop($event)"
            (drop)="dropOnTitular($event, 'Midfielder', i)">
            <app-jugador-alineacion
              [jugador]="jugador"
              [selected]="selectedTitular?.jugadorId === jugador.jugadorId"></app-jugador-alineacion>
          </div>
        }
      </div>

      <div class="pitch-row defenders-row">
```

Figura 53 — Fragmento de código sección Alineaciones

En este otro ejemplo vemos un tipo de estructura totalmente distinto. Sigue las mismas convenciones ya mencionadas anteriormente, con HTML dando estructura y Tailwind gestionando los estilos. Pero en esta imagen vemos estructuras de control de Angular que nos permiten insertar elementos de forma dinámica, dependiendo de los datos que se quieran mostrar y de su estructura. Además, en esta sección en concreto, se han usado propiedades de Angular que nos permiten arrastrar algunos de esos elementos con el ratón pudiendo ubicarlos el usuario de la manera que el considere, gracias a la propiedad draggable.

Mientras en la primera captura veíamos una estructura compuesta únicamente por HTML/CSS en esta segunda ya vemos como Angular se integra en estos elementos para proporcionar flexibilidad, interactividad y reactividad. Flexibilidad porque se adapta a los datos que recibe del back, interactividad porque permite al usuario interactuar y recolocar dichos elementos y reactividad porque se actualiza en cuanto hay algún mínimo cambio en el flujo de datos.

Con todas estas características creemos haber construido una interfaz moderna, sencilla y muy intuitiva, que permitirá a nuestros usuarios moverse cómodamente por nuestra plataforma.

Sin embargo hay un pequeño detalle que podemos añadir, que nos permite obtener métricas del desempeño de nuestra aplicación y de como se desenvuelve. Estas herramientas son indicativos de lo que esta bien, lo que es aceptable y, aunque no nos guste, aquello que debemos de mejorar.

La herramienta de la que hablamos es **lighthouse**. Como podemos ver en su página (Google Developers, 2026), esta herramienta, de código abierto desarrollada por Google, nos permite observar las métricas de nuestras páginas web. A través de ellas podremos saber que es lo que hemos hecho bien, cuales son los elementos que fallan y lo que tenemos que hacer para mejorarlos.

Usando la terminal de nuestro IDE podemos ejecutar el diagnóstico que nos dirá exactamente lo que hemos hecho bien y lo que no hemos hecho tan bien.

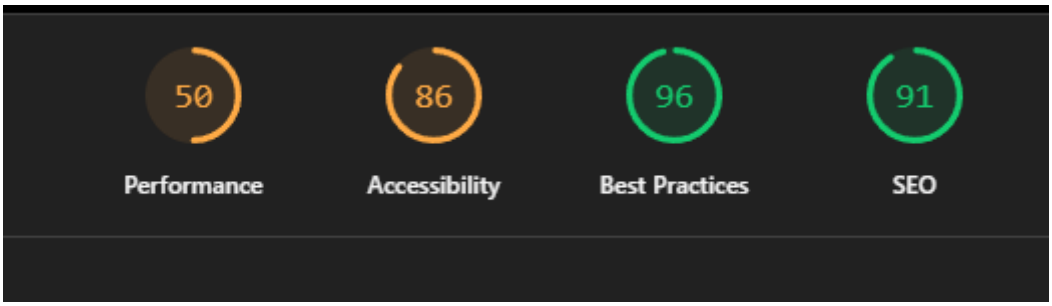


Figura 54 — Google Lighthouse

Hay que ser autocríticos. Aunque la media general de la puntuación es elevada, que no está nada mal, la puntuación en performance es un poco baja.

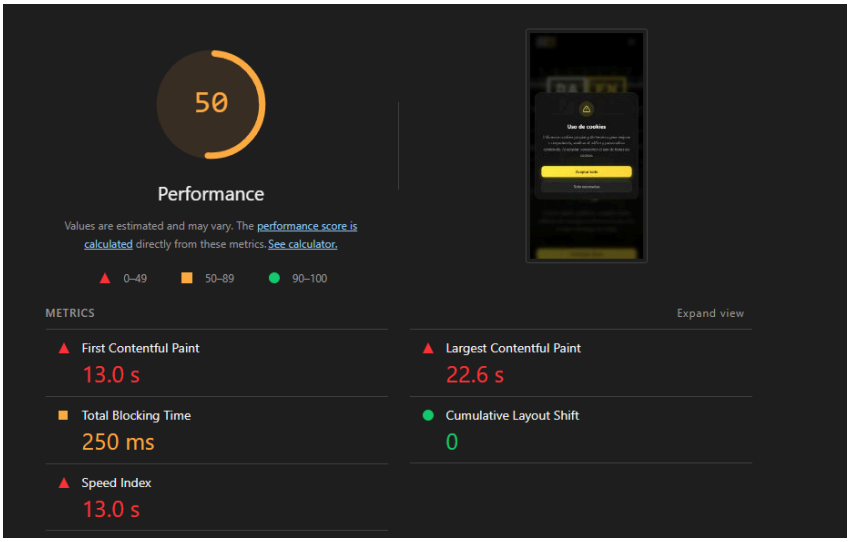


Figura 55 — Google Lighthouse Performance

El mayor problema, según las sugerencias del propio Lighthouse, es la carga inicial de algunos recursos. Si bien es cierto que ese tiempo de carga se mide en milisegundos, no parece tanto. Pero si tenemos en cuenta la cantidad de peticiones que puede recibir un sistema como el que estamos desarrollando esos pocos milisegundos, al final de un día pueden convertirse en unos cuantos minutos acumulados.

Sabiendo esto, podemos deducir que una de las primeras cosas que tenemos que hacer, es reducir el peso de las imágenes que se cargan al inicio, lo que aumentaría mucho la fluidez del mismo. También es cierto que este primer elemento incluye la petición de aceptación al usuario de las cookies. Aún así, se debe de encontrar una forma de reducir el tiempo de carga inicial.

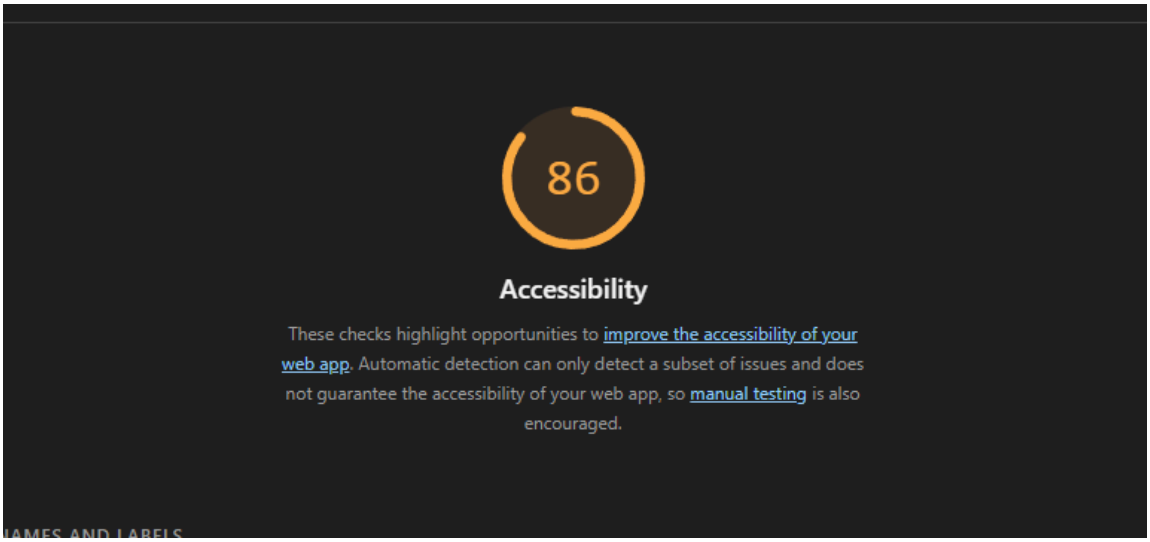


Figura 56 — Google Lighthouse Accesibility

La accesibilidad de Google mide que tan fácil es para cualquier tipo de usuario moverse y navegar por nuestro entorno. Elementos como el contraste de color, la navegación mediante teclado, compatibilidad con lectores de pantalla o uso de elementos ARIA (etiquetas y elementos HTML diseñados específicamente para accesibilidad). En nuestro caso, casi un 90 es perfectamente aceptable. Nuestro esquema de colores y la buena colocación de botones, enlaces y otros elementos interactivos hace que la puntuación suba en gran medida. La falta de elementos como Screen-Readers o un uso no muy elevado de elementos de tipo ARIA hace que no seamos capaces de llegar al 90.

Al respecto de las otras dos puntuaciones, hemos de decir que nos indican que seguimos buenas prácticas y el SEO de nuestra página es excelente.

Las buenas prácticas no solo se basan, según la propia documentación de Google Lighthouse, en una buena estructura de código, cosa que es más extensiva en el SEO, si no también mide las prácticas de seguridad, de modernidad del código (versiones de lenguaje, no uso de elementos deprecados), experiencia de desarrollo, uso eficiente de la consola no inundándola de mensajes o elementos y buena optimización de los recursos sin estar deformados por el CSS.

En cuanto al SEO, una buena estructura del código, de su anidamiento y su semántica.

Al analizar los datos vemos que hay puntos en los que mejorar, cosa en la que podremos enfocarnos en futuras iteraciones.

11.5 Arquitectura de seguridad

El proceso de la seguridad que se ha implementado en este proyecto ha tratado de estar a la altura del desarrollo moderno, usando estándares y prácticas actuales con una eficacia y robustez probadas.

Se va a seguir una ruta desde que el cliente se registra o accede a la página principal hasta la llegada de los datos al SGBD (Sistema Gestor de Base de Datos) para poder tener una imagen mucho más completa del trabajo realizado en este aspecto.

1 El usuario accede por primera vez al sitio web

La primera vez que el usuario entra a nuestra página lo hará siempre con restricciones por no estar registrado/logeado. Para ello, como veremos en la imagen siguiente, Angular nos posibilita crear un sistema de rutas, pública y privadas. Este nuevo usuario solo podrá acceder a la parte pública mientras no inicie sesión. Mediante el archivo de enrutamiento de Angular y el código HTML de los componentes, podemos esconder o mostrar elementos al usuario dependiendo de su estado.

Comprobaremos ahora la estructura del archivo de rutas definido en Angular que nos permite proteger la navegación por nuestro sitio web.

Angular **Archivo app.routes.ts**

```
export const routes: Routes = [
  {path: '', pathMatch: "full", redirectTo: 'daznfantasy'},
  {path: 'daznfantasy', children: [
      {path: '', pathMatch:'full', component: LandinPage},
      {path: 'login', component: LogIn},
      {path: 'register', component: SignIn},
      {
        path: '', canActivateChild: [loginGuardGuard], children: [
          {path: 'home', component: Home},
          {path: 'news', component: LigaNoticias},
          {path: 'leagues', component: SeleccionLigas},
          {path: 'user', component: UserSettings},
          {path: 'mercado', component: LigaMercado},
          {path: 'noticias', component: LigaNoticias},
          {path: 'widgets', component: LigaWidgets},
          {path: 'admin/config', component: AdminConfig, canActivate: [adminGuard]},
          {
            path: 'liga',
            component: DashboardLigas,
            children: [
              {path: '', pathMatch: 'full', redirectTo: 'clasificacion'},
              {path: 'clasificacion', component: LigaClasificacion},
              {path: 'plantilla', component: LigaPlantilla},
              {path: 'mercado', component: LigaMercado},
            ],
          },
        ],
      },
    ],
  },
  {path: 'page404', component: Page404},
  {path: '**', component: Page404},
  ];
```

En el fragmento superior vemos que solo aparecen como rutas públicas Home, Login y Register. A partir de la linea de register encontramos el siguiente indicador: **[login-GuardGuard]**. Este simple bloque hace que esas rutas no puedan ser accedidas si no se está registrado y logeado correctamente. En este momento no vamos a comentar todavía el funcionamiento de la **Guarda**, su funcionamiento tendrá más sentido una vez veamos otros pasos intermedios.

Tras el acceso inicial el siguiente paso lógico es el de registrarse o logearse. Aunque podrían parecer pasos similares siguen caminos distintos. La lógica de registro lleva una serie de pasos que el login no. Veremos más en detalle esas diferencias.

2 Proceso de registro

Cuando un usuario trata de registrarse en la plataforma se llevan a cabo una serie de procedimientos únicos de este paso. Entre ellos se encuentran la comprobación de que el usuario ya estuviese registrado con un email, por lo que no puede asociar otra cuenta al mismo, y la creación de una contraseña válida que luego pasará a hashearse para mayor seguridad.

Node.js **Middleware y Endpoints**

```
  routerAuth.post(
    "/auth/register",
    emptyFields(["username","email","password","fechaNacimiento"]),
    validateUsernameLength,
    fechaValidate(),
    registrarNuevoUsuario
);
```

La imagen nos muestra la definición de un endpoint y los middleware que usa ese endpoint como validaciones que aseguran la validez de los datos que se envían a nuestra base de datos. En la primera parte indicamos el tipo de petición, en este caso un **POST**. Seguido, entre paréntesis, indicamos el endpoint (fragmento de la url a donde se dirige nuestra plataforma dentro del backend). Separados por comas, tras el endpoint, vemos los diferentes Middlewares (ver siguientes apartados) que usará este endpoint como validadores. Por último se indica a que función de nuestro backend se va a llamar si todos los datos son correctos.

3 Middlewares y validación de los datos.

Al realizar peticiones un usuario a través del Frontend de nuestra aplicación, se hacen pasar los datos que se envían por funciones llamadas Middlewares. Estas funciones se dedican a validar el formato correcto de los datos que son enviados o la existencia de los mismos dentro de nuestro sistema. Por norma general, cada middleware, se encarga solamente de una tarea concreta, vease la longitud y composición de un campo de texto, el valor de un campo numérico, la validez de datos de tipo fecha o la existencia o no de un dato en nuestro sistema mediante consultas a la base de datos. Los datos enviados por la petición van recorriendo cada una de las funciones de middleware en el orden en las que se escriben. De no ser correcto el resultado en alguno de esos middlewares la operación se corta, normalmente, devolviendo un mensaje de error al usuario. Mientras los datos sean correctos van saltando a la siguiente función de middleware hasta llegar a la última, que, en caso positivo, dejará pasar nuestros datos a las funciones o servicios que se comunican con nuestra base de datos.

Para entender mejor estos conceptos revisaremos un fragmento de código en typescript, perteneciente a uno de los middlewares desarrollados. En el caso que se va a mostrar, la tarea asignada será la de comprobar que uno o más campos enviados hacia nuestro backend contengan información, no permitiendo valores nulos o vacíos.

Este tipo de operaciones se repiten una y otra vez en cada petición y sus funciones variarán dependiendo del tipo de datos que se quiera validar o de la autenticidad de los mismos.

Typescript **Código de un Middleware**

```typescript
import {Request, Response, NextFunction} from "express";

export const emptyFields = (fields: string[]) => {
    return (req: Request, res: Response, next: NextFunction) => {
        console.log(fields);

        // Primero probé con forEach, pero no rompe el bucle ni la petición cuando encuentra un error.

        for(const field of fields){
            // Recorro el array y por cada field lo que hago es que obtenga el valor de ese mismo field
            const valor = req.body[field];
            console.log(valor);
            // Por cada field hago una serie de validaciones, de no ser correcto, debo de romper el bucle
            if(valor === undefined ||
                valor === null ||
                typeof valor !== 'string' ||
                valor === "" ||
                valor.trim() ===""
            ){
                // Cuando no es válido devuelve una respuesta
                return res.status(400).json({
                    error: `El campo ${field} no tiene valor o su formato es incorrecto`
                });
            }else{
                console.log(`El valor de ${valor} es correcto`)
            }
        }
        // Si todo va bien salta al siguiente middleware
        next();
    };
};
```

Cuando todos los datos hayan sido validados por el middleware anterior pasarán al siguiente y así sucesivamente hasta que se hayan realizado todas las validaciones requeridas.

Los comentarios en el código ayudan a entender que se está haciendo en cada momento.

En el caso que los ocupa, el registro, una vez validados los campos en el formato que se quieren debemos pasar al registro propiamente dicho.

4 Hasheado de una contraseña

Para que todas las contraseñas de los usuarios se almacenen de forma segura y que no sea fácil para posibles atacantes su interceptación o que se use fuerza bruta para iniciar sesión seguiremos los siguiente pasos.

Typescript **Hashing**

```
const hash = await bcrypt.hash(password, 10);
```

Bcrypt (AcademiaLab, 2024) es una herramienta que nos permites hashear nuestros passwords para almacenarlos de forma segura. Así, aunque algún atacante se haga con las contraseñas almacenadas, no podrá obtener su valor ya que este es un proceso unidireccional. Una vez hasheada una contraseña solo podremos comparar su valor con una cadena candidata.

Lo que hace el pequeño fragmento de código que aparece en la imagen anterior es aplicar un algoritmo, en este caso de fuerza 10, que alterará toda la cadena de texto. Así se convertirá en un nuevo fragmento que no tendrá nada que ver con el fragmento original.

Adicionalmente, si quisiésemos añadir más complejidad, podríamos ejecutar lo que se conoce como **«Salt & Pepper»** (Baeldung, 2024) . Este proceso lo que hace es añadir más y más modificaciones a la cadena de texto, haciendo mucho más difícil conocer el valor original, aunque añade tiempo adicional al proceso.

Sequelize **Comprobación de existencia, almacenamiento y registro**

```
    const existe = await Usuario.findOne({
    where: {
        [Op.or]: [{ username }, { email }]
    }
});

if (existe) {
    return res.status(400).json({
        error: 'El username o email ya existen'
    });
}


const usuarioCreado = await Usuario.create({
    username,
    email,
    passwordHash: hash,
    fechaNacimiento,
});

return res.status(201).json(`Nuevo usuario creado ${usuarioCreado.username}`);
```

Con todos los datos preparados se comprueba que no exista ya un usuario con el mismo username o el mismo email. De ser así se interrumpe el intento de registro, en caso contrario se registra al usuario y se devuelve un mensaje al frontend.

5 Login, validación de Bcrypt y emisión de JWT

Una vez nos hemos registrado, el usuario tratará de logearse. Volverá a aparecer **bcrypt** para comprobar mediante el algoritmo que el password del usuario coincide. Pero aquí, la parte importante es **la emisión de un token JWT**.

Una vez el usuario, en el formulario de login, envía sus datos, estos volverán a pasar por los correspondientes middlewares. Una vez validados deberán de mandarse a nuestro servicio de login que realizará lo siguiente:

TypeScript **Validación de Password y emisión de token JWT**

```
    export async function loginService(username: string, password: string){

    const usuario = await Usuario.findOne({ where:{username} });

    if(!usuario) throw new Error("Credenciales");

    const ok = await bcrypt.compare(password, usuario.passwordHash);

    if(!ok) throw new Error("Credenciales");

    const token = jwt.sign({
        sub: usuario.usuarioId,
        rol: usuario.rol
    }, process.env.JWT_SECRET!, {expiresIn: 3600});

    return {
        token,
        user:{
            uuid: usuario.usuarioId,
            username: usuario.username,
            rol: usuario.rol
        }
    };
}
```

De nuevo bcrypt aparece, pero esta vez para comparar el valor del password enviado con el que se ha almacenado del usuario.

Una vez se comprueba que es correcto, mediante la librería importada de **JsonWebToken**, se creará y firmará un nuevo token **jwt**. Este token es una cadena de texto que, en nuestro caso, transporta la ID del usuario, su nombre de usuario y su rol. Acto seguido se devuelve al frontend. La cadena puede ser de una longitud variable, según como la configuremos, y puede transportar diferentes datos. De ser interceptada, cualquier modificación que se realice sobre ella invalida su firma y modifica su contenido de no conocer la clave con la que se generó.

6 De vuelta al Frontend

Una vez nuestro backend devuelve un login correcto, con su correspondiente token, el frontend debe de encargarse de almacenarlo y gestionarlo. Su uso en el frontend nos sirve para poder navegar a zonas restringidas para usuarios no registrados o, en caso de ser su rol, poder acceder a zonas propias de usuarios con privilegios elevados.

El frontend debe de encargarse de guardar el token de manera que siempre sea accesible al realizar futuras peticiones y, en caso de cerrar sesión, del borrado del mismo para que no quede ningún rastro en nuestro navegador. A partir de este momento, entran en escena los interceptores provistos por **Angular**.

7 Interceptores y peticiones.

Con nuestra identificación disponible gracias a jwt podremos navegar a ciertas zonas protegidas sin necesidad de identificarnos para cada una de ellas. Esta función la realizan los **Interceptores**. Estos elementos son funciones que, cada vez que el frontend hace una petición al backend, intercepta esa petición y comprueba las direcciones a las que se dirigen. De ser una dirección que requiere autenticación, el interceptor comprobará que tengamos almacenado un token válido. De no ser así la petición no podrá realizarse, bien porque no se realiza desde el front, bien porque un middleware del back no permite que se hagan ciertas peticiones sin token. En caso de tener token, el interceptor **inyectará** ese token en la **cabecera de nuestra petición** para así identificarnos correctamente.

Angular **Interceptores**

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
const token = localStorage.getItem('token');

if (!token) {
  return next(req);
}

const authReq = req.clone({
  setHeaders: {
    Authorization: `Bearer ${token}`,
  },
});

return next(authReq);
};
```

En el fragmento de código anterior vemos como el interceptor trae el token que tenemos almacenado con `localStorage.getItem('token')`. Después evalua que efectivamente exista un token. De no ser así, interrumpe la operación. En caso afirmativo con `req.clone()`, el interceptor inyecta el token en la petición para que el usuario pueda autenticarse correctamente en el backend. Si no se ejecutase esta inyección, los middleware del back que comprueban que se envía un token correctamente firmado en la petición devolverían un error **HTTP 401 UNAUTHORIZED**.

Tras todos estos pasos volvemos al punto en el que empezamos. Con los tokens en nuestro poder y estando correctamente autenticados, nuestro primer elemento, **las guardas**, nos permitirán el paso a los diferentes espacios dentro de nuestra aplicación. Si no los tuviesemos, nos devolverían un código de error, en este caso, **HTTP 403 FORBIDDEN**. Este error puede darse o bien porque no estamos correctamente logeados en nuestra aplicación o, en otros casos, porque el rol que tenemos asignados no está autorizado a acceder a determinados espacios.

8 Las Guardas y el token

Ya con el token correctamente almacenado, las **Guardas** permitirán acceder al usuario a zonas que anteriormente no estaban habilitadas para el.

Angular **Código de una Guarda**

```
    export const loginGuardGuard: CanActivateFn = (route, state) => {
    const router = inject(Router);

    if(localStorage.getItem('token')){
      return true;
    }
    return router.parseUrl('/daznfantasy/login');
};
```

Aquí el proceso de funcionamiento. Se vuelve a comprobar la existencia de un token en nuestro navegador. De ser así, la función retornará true, permitiendo acceso a diferentes zonas.

Dependiendo de las necesidades podemos definir tantas **Guardas** como sean necesarias, bien sea para diferentes tipos de usuarios, privilegios o necesidades.

Este ciclo comentado en este apartado se repetirá tantas veces como se hagan peticiones, del tipo que sea y, gracias a él, podremos controlar y garantizar la seguridad de nuestro sistema.

11.4 Demostración de funcionalidad

Para esta sección vamos a mostrar uno de los flujos de nuestra aplicación que toca más componentes. Vamos a partir desde la creación de un usuario mediante el formulario de registro. Nos logearemos con ese usuario. Procederemos a la creación de una liga, lo que también incluye la creación de un equipo. Al crear un equipo se ejecutan de forma automática dos procesos en el backend. El primero es un sorteo que elige de entre todos los jugadores que no tienen equipo en la liga correspondiente y se le asignan 14 al equipo del usuario. Una vez acabado ese proceso se ejecuta automáticamente otro. De entre esos 14 se seleccionan los 11 que conformarán la alineación inicial del equipo del usuario. Vamos a verlo paso a paso.

11.4.1 Registro y Login

Lo primero que haremos será acceder a nuestra plataforma. Para ello, como por ahora solo se ejecuta en un entorno local, accederemos a la siguiente URL:http://localhost:4200/daznfntsy.

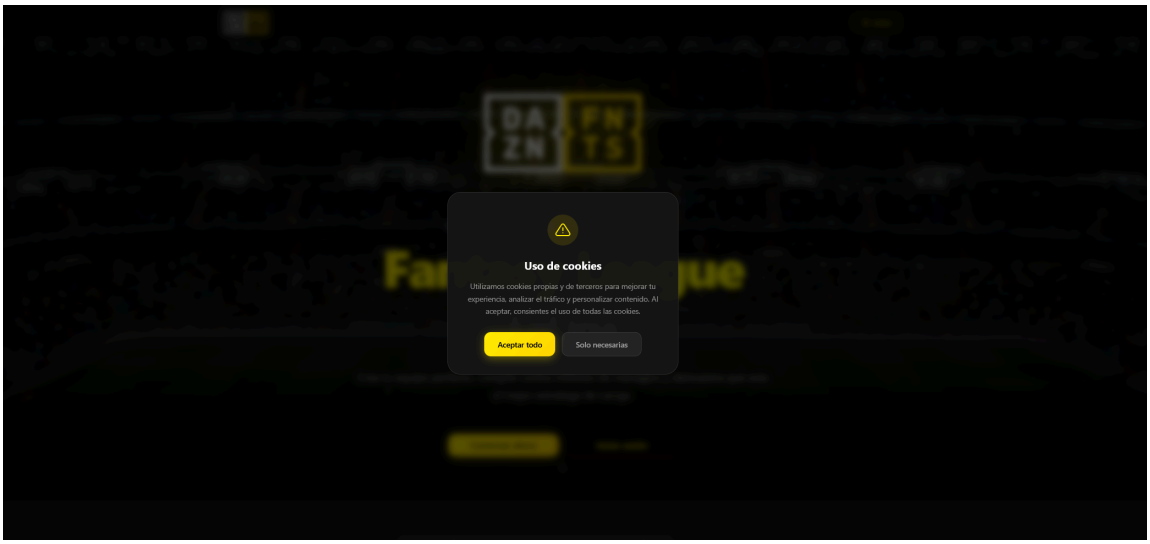


Figura 57 — Página de Bienvenida o Landing

Al acceder a la dirección indicada anteriormente llegamos a la página de bienvenida de nuestra aplicación. Vemos, gracias al HTML y CSS un estilo muy marcado, siguiendo el libro

de marcas proporcionado por nuestro cliente. Los colores negro y amarillo propios de DAZN. Al entrar por primera vez, el frontend comprueba que en el almacenamiento local del navegador del usuario no hay almacenadas cookies de sesión. Al no tenerlas aparece un modal que solicita y da la opción al usuario de elegir el tipo de cookies que desea aceptar. Una vez aceptemos, da igual si son todas o solo las necesarias, este modal no volverá a aparecer en visitas posteriores.

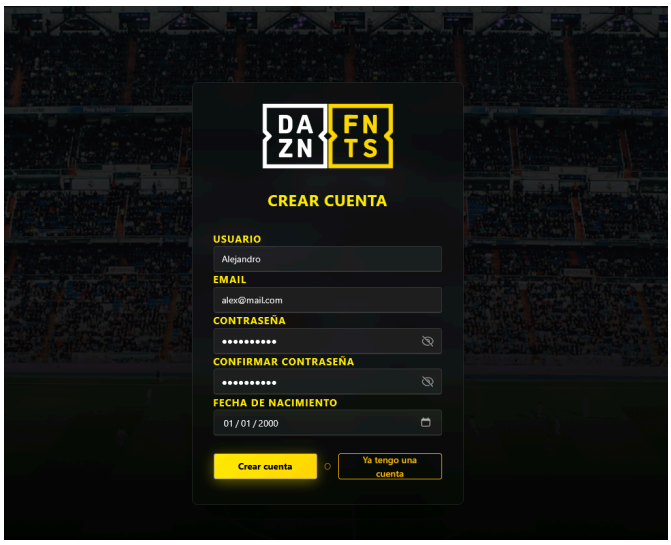


Figura 58 — Formulario de registro y login

Ya pasada la presentación inicial la página nos lleva directamente a la página de registro/login. El usuario aún así puede navegar, pero solo a una sección de noticias abierta a todos los usuarios. Como explicamos en la sección del front y del back, las guardas, interceptores y la falta de un token JWT firmado por el back hacen que un usuario no autenticado no pueda navegar a mas secciones. Una vez rellenamos nuestro formulario de registro pulsamos en crear cuenta.

Como ya vimos en la subsección de Swagger en el backend, estos datos serán enviados, procesado y se almacenarán en la base de datos.

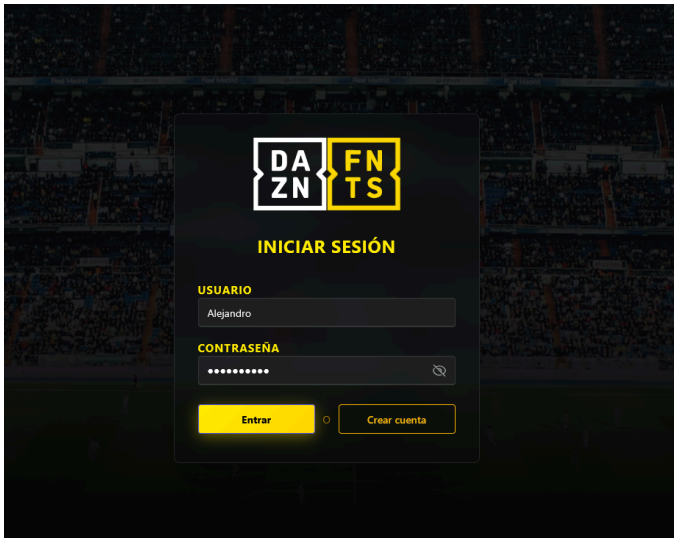


Figura 59 — Logeandose y Registrandose

Una vez registrados podremos indicar mediante el botón de «Ya tengo cuenta» o el botón de login en un menú en la parte superior de la página que vamos a logearnos. Se nos mostrará el siguiente formulario, que solo nos pide nuestro usuario y nuestra contraseña. Si los datos no son correctos aparecerá una banda de color rojo/rosa indicando algún error o que las credenciales no son correctas. En caso de serlo, nos llevará a la pagina principal para usuarios registrados.

11.4.2 Página principal para usuarios registrados

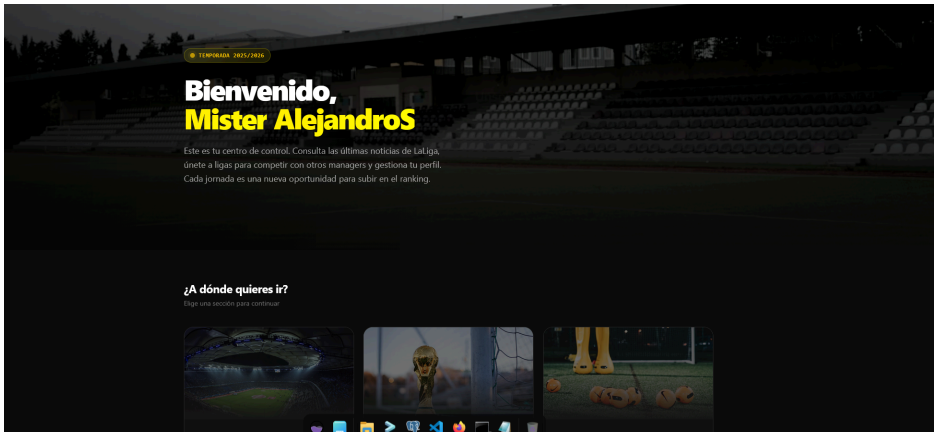


Figura 60 — Landing Page Usuarios Registrados

Una vez logeados correctamente se nos llevará a esta página, con una serie de subsecciones. De izquierda a derecha la sección de noticias, la gestión de ligas y equipos y por último la gestión de perfil del usuario.

11.4.3 Gestión de ligas

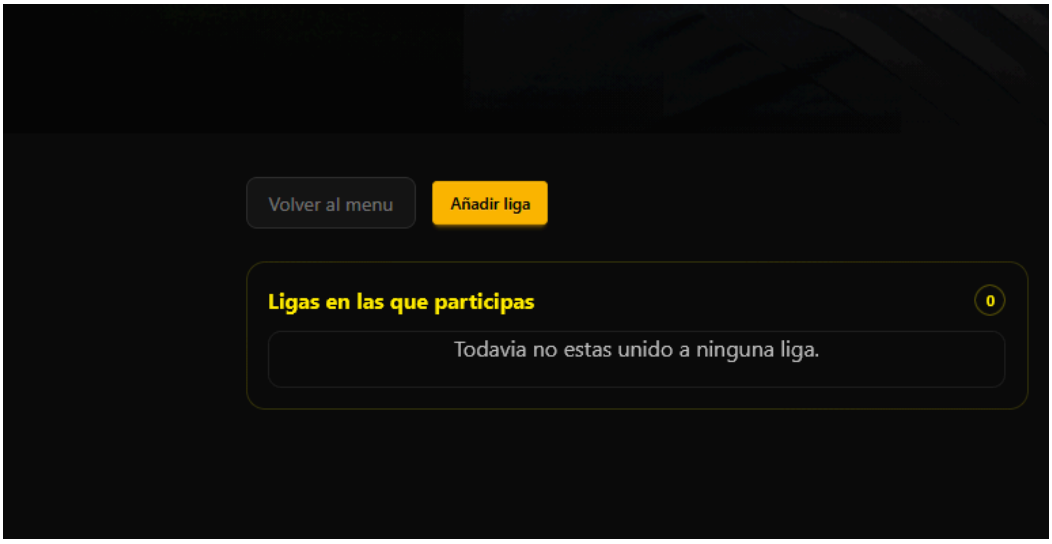


Figura 61 — Creando nueva liga.

En la sección de ligas tenemos una sección con un listado que nos muestra las ligas disponibles para unirse. Esta sección solo muestra aquellas ligas cuyos usuarios activos son menos de 20. Esto lo proporciona el backend mediante una consulta sencilla a la base de datos.

Sin embargo, en nuestro caso lo que nos interesa es que este usuario cree una nueva liga. Como vemos, debajo del botón de Añadir liga, hay un listado en el que vemos las ligas en las que participa el usuario. En este caso, al ser un usuario recién creado, todavía no es participante en ninguna.

Para seguir el flujo de funcionamiento vamos a pulsar en Añadir liga.

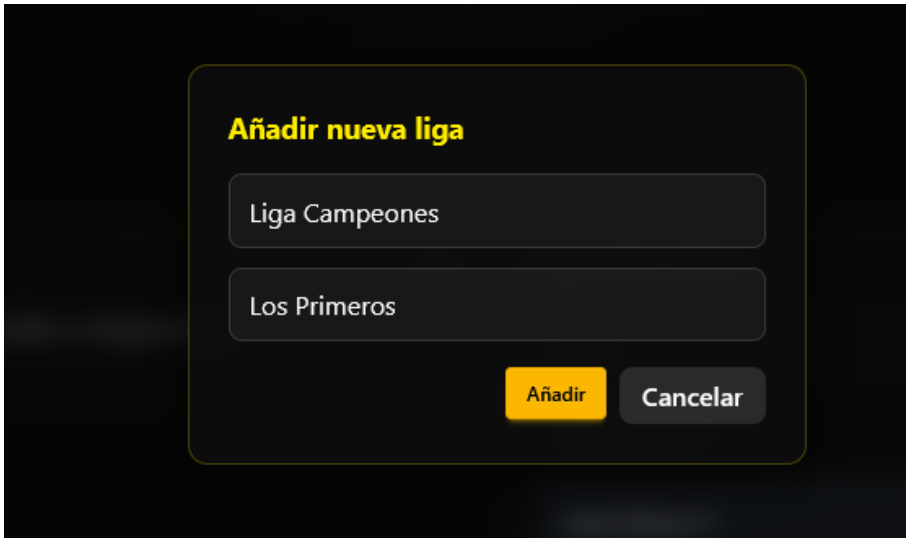


Figura 62 — Rellenando los datos requeridos para crear una liga

Una vez seleccionamos Añadir liga, saltará un modal con un formulario. El mismo nos pide que indiquemos el nombre de la liga que queremos crear y un nombre para el equipo del usuario en esa liga concreta. Se decidió que debía hacerse al mismo tiempo pues no se vio ningún sentido a que un usuario crease una liga y no tuviese un equipo participante. De ahí que estos dos datos se pidan al mismo tiempo.

Una vez rellenado el formulario, pulsamos Añadir cuando estemos conformes y estemos decididos a crear nuestra liga personal.

11.4.4 Sorteo de jugadores y de alineación.

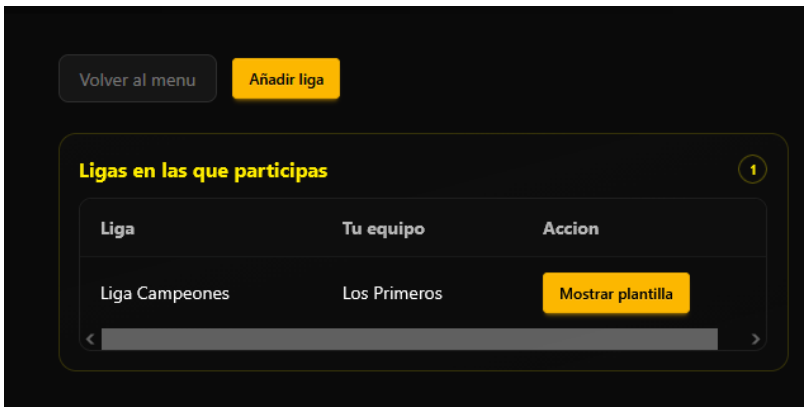


Figura 63 — Liga creada.

Al pulsar añadir se guarda en nuestra base de datos los datos referentes a nuestra liga y a nuestro equipo. Vemos también que el listado que antes estaba vacío nos muestra la liga que acabamos de crear y el equipo con el que participamos en dicha liga.

En cuanto a los sorteos, ya se han realizado al pulsar en el botón de Añadir. De existir algún error en alguno de esos procesos habríamos recibido un mensaje de error y no se habrían creado ninguno de estos dos elementos y tendríamos que intentar registrar de nuevo nuestra competición y nuestro equipo. Para poder ver el resultado de los dos sorteos debemos de pulsar en Mostrar plantilla.



Figura 64 — Página de plantilla y alineación.

En la imagen vemos el resultado de los dos sorteos. El primero elige 14 jugadores, que serán los iniciales de nuestro equipo. De esos 14 jugadores, por la forma en que se preparó el sorteo, 2 serán porteros, 5 serán defensas, 4 centrocampistas y 3 delanteros.

Una vez finalizado ese sorteo, se ejecuta el segundo. De los jugadores disponibles el sistema elije un portero titular, 4 defensas titulares, 3 centrocampistas y 3 delanteros.

Con esto finalizado el usuario ya tiene todo lo necesario para disfrutar de la competición. Sin embargo, si no está de acuerdo con los jugadores obtenidos puede tratar de comprar alguno de los disponibles en el mercado. Si accede a esa sección mediante el menú superior izquierdo verá lo siguiente.

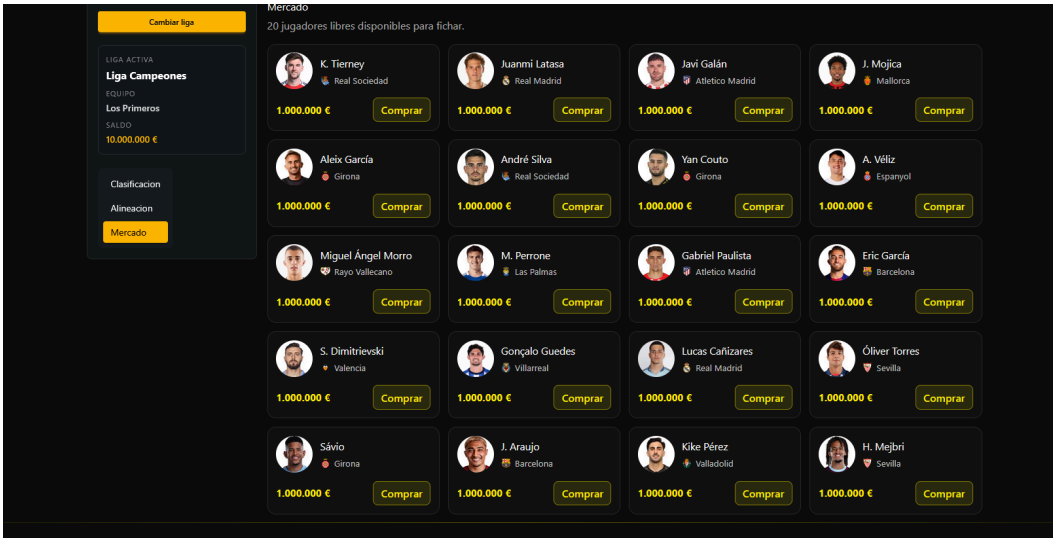


Figura 65 — Mercado de Jugadores

Una vez al día, el administrador refrescará el mercado de jugadores disponibles para fichar. Los usuarios pueden acceder a el en cualquier momento. El sistema, cuando el administrador lo ejecuta, selecciona 20 jugadores aleatorios de entre aquellos que no figuren en la plantilla de ningún usuario que participe en esta liga. Si dispone de saldo positivo y mayor que el precio del jugador puede comprarlo de forma automática si pulsa el botón junto a la imagen y nombre del jugador.

Con esto queda demostrada una de las principales funcionalidades de nuestra aplicación, desde la creación de un usuario hasta la creación de liga y equipo para poder disfrutar de la competición.

12 DESPLIEGUE DE LA APLICACIÓN

Una vez finalizado el desarrollo de las diferentes partes de nuestro proyecto, el último paso consistió en desplegar la aplicación para que fuese accesible desde Internet. Para ello decidimos utilizar dos plataformas diferentes. Por un lado, Vercel para alojar el frontend desarrollado con Angular y, por otro, Railway para alojar tanto el backend desarrollado con Node.js y Express como la base de datos PostgreSQL.

El objetivo de esta separación es mantener cada uno de los componentes de la aplicación de forma independiente. Esto facilita las tareas de mantenimiento y permite actualizar cada parte sin afectar al resto del sistema.

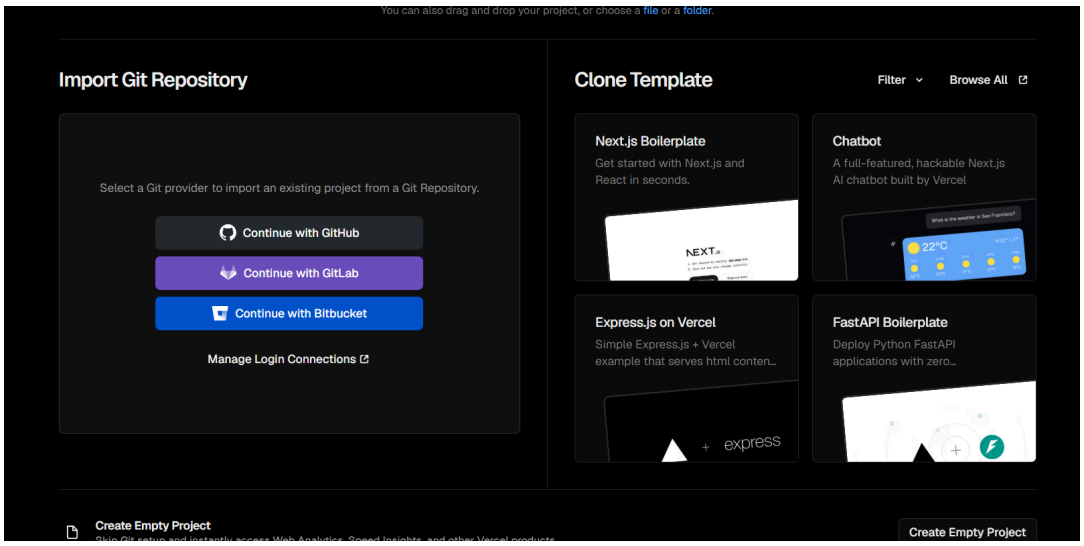


Figura 66 — Página principal de Vercel

Para desplegar el frontend comenzamos accediendo a Vercel. Esta plataforma permite importar proyectos directamente desde repositorios remotos, simplificando enormemente el proceso de publicación de aplicaciones web.

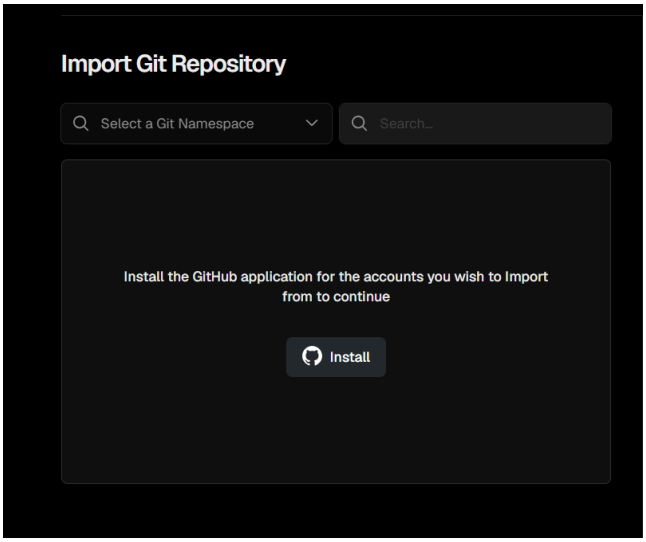


Figura 67 — Selección de cuenta de GitHub en Vercel

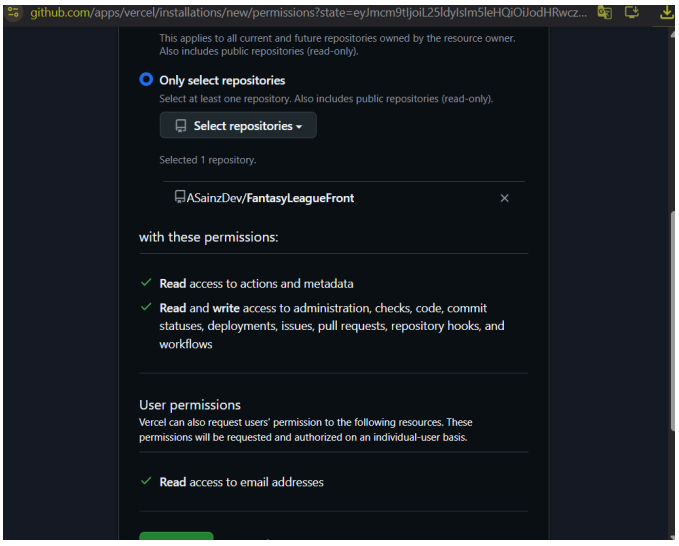


Figura 68 — Selección del repositorio del frontend

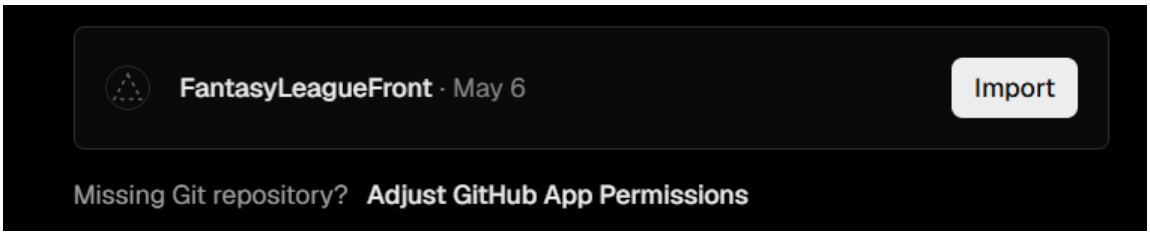


Figura 69 — Importación del proyecto en Vercel

Como podemos observar en las imágenes anteriores, Vercel permite conectar nuestra cuenta de GitHub, seleccionar el repositorio correspondiente e importarlo para comenzar el proceso de despliegue.

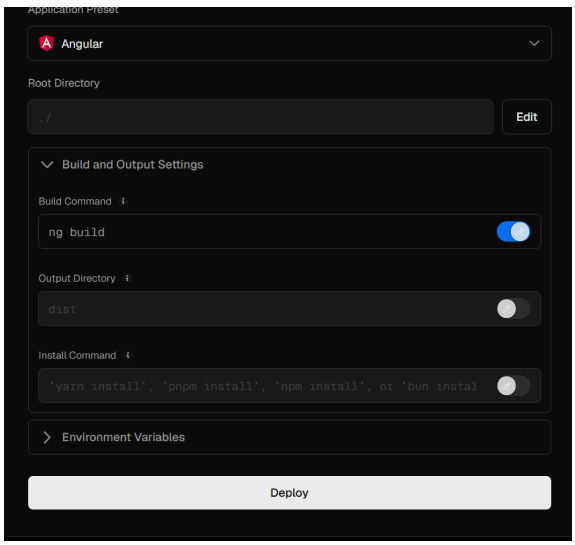


Figura 70 — Configuración del proyecto y variables de entorno

Antes de completar el proceso fue necesario configurar determinados parámetros del proyecto. Entre ellos encontramos las variables de entorno utilizadas para la comunicación con el backend. Por motivos de seguridad dichos valores han sido ocultados en las capturas mostradas.

Una vez preparado el frontend, procedimos al despliegue del backend y de la base de datos utilizando Railway.

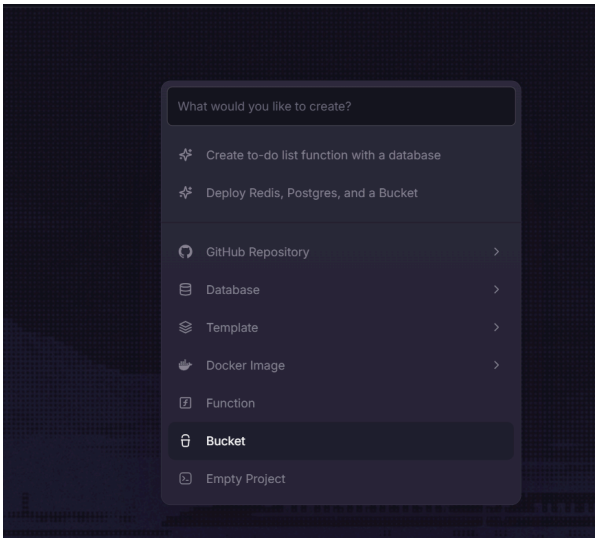


Figura 71 — Página principal de Railway

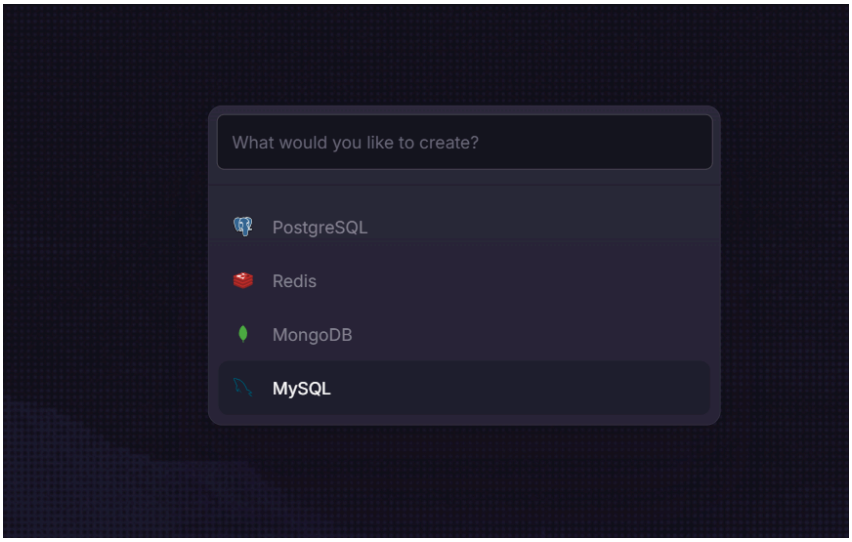


Figura 72 — Selección de PostgreSQL como base de datos

Railway ofrece plantillas para desplegar diferentes servicios. En nuestro caso utilizamos PostgreSQL como sistema gestor de base de datos.

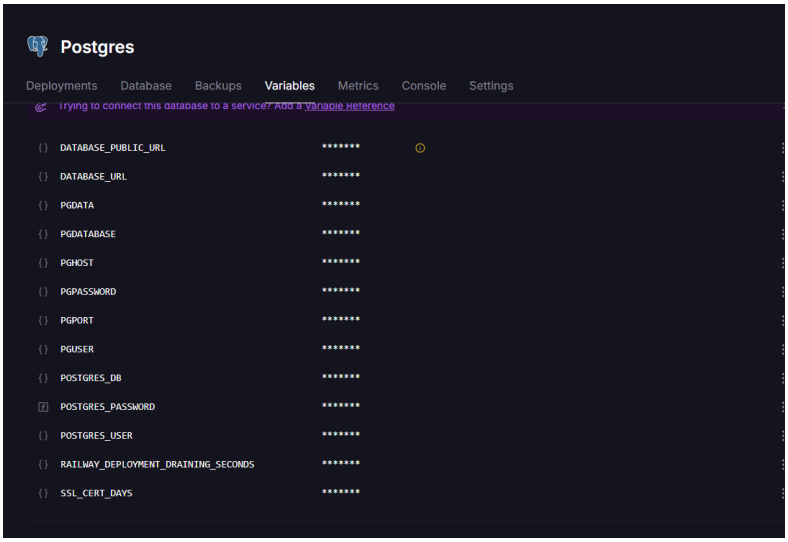


Figura 73 — Variables de entorno de PostgreSQL

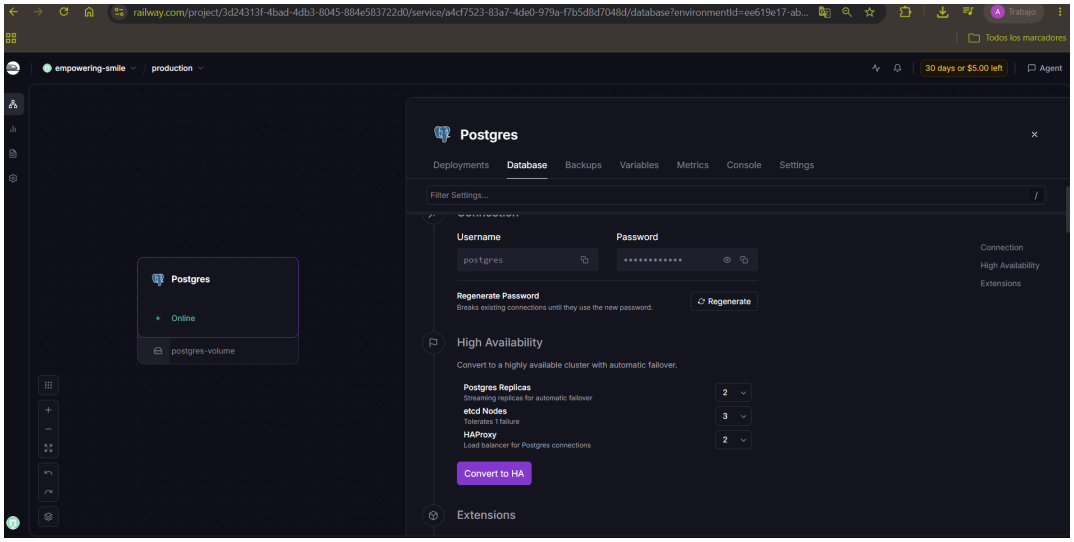


Figura 74 — Configuración adicional de PostgreSQL

Durante la creación de la base de datos, Railway genera automáticamente gran parte de la configuración necesaria para establecer la conexión. De la misma forma que en el caso anterior, algunos datos sensibles han sido ocultados para evitar exponer credenciales de acceso.

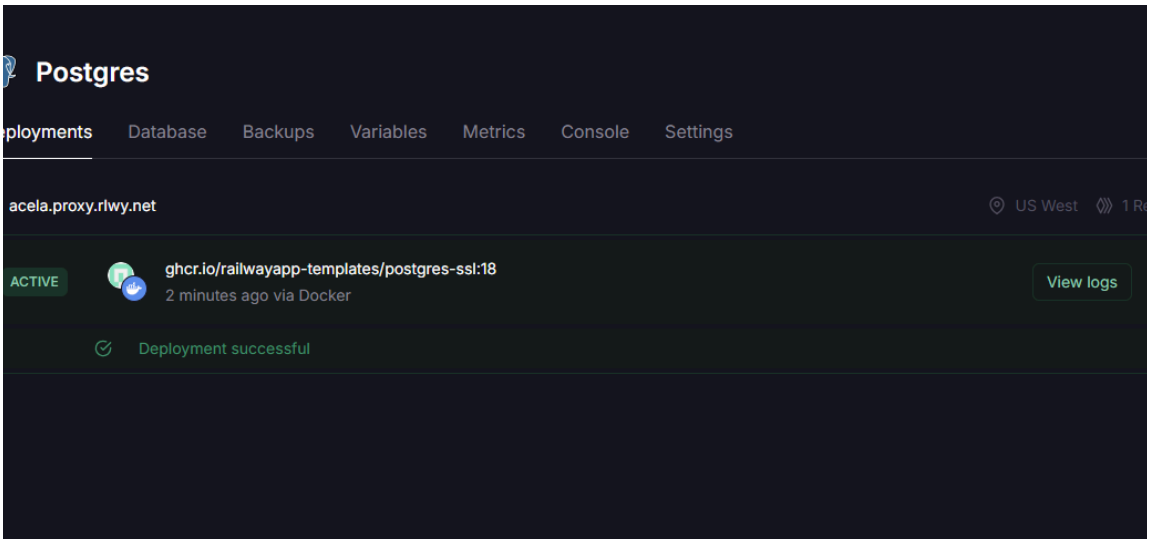


Figura 75 — Despliegue completado de PostgreSQL

Una vez finalizado el despliegue de la base de datos, procedimos a desplegar el backend.

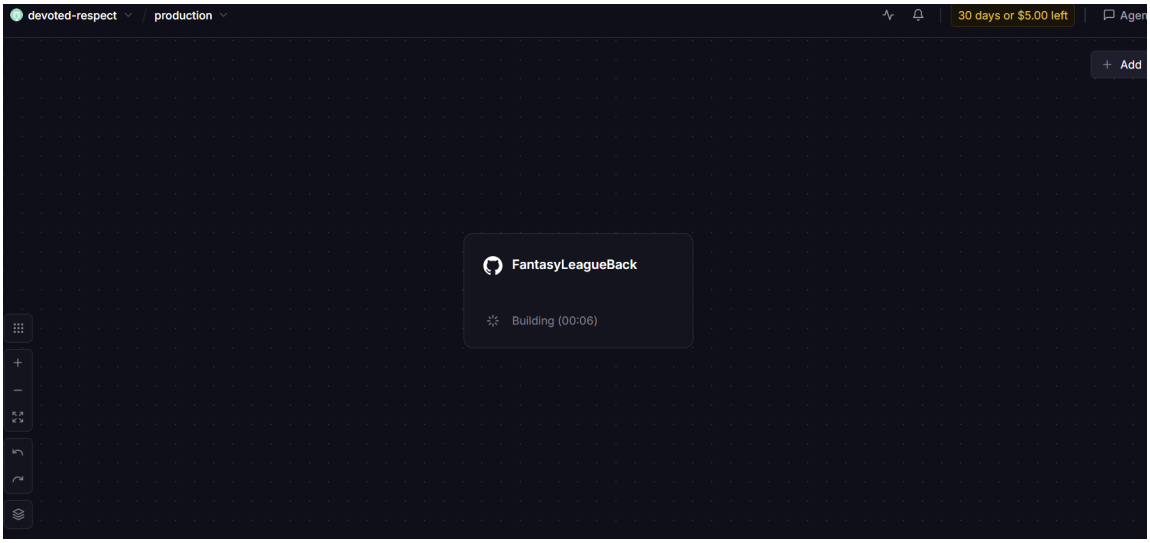


Figura 76 — Selección del repositorio del backend

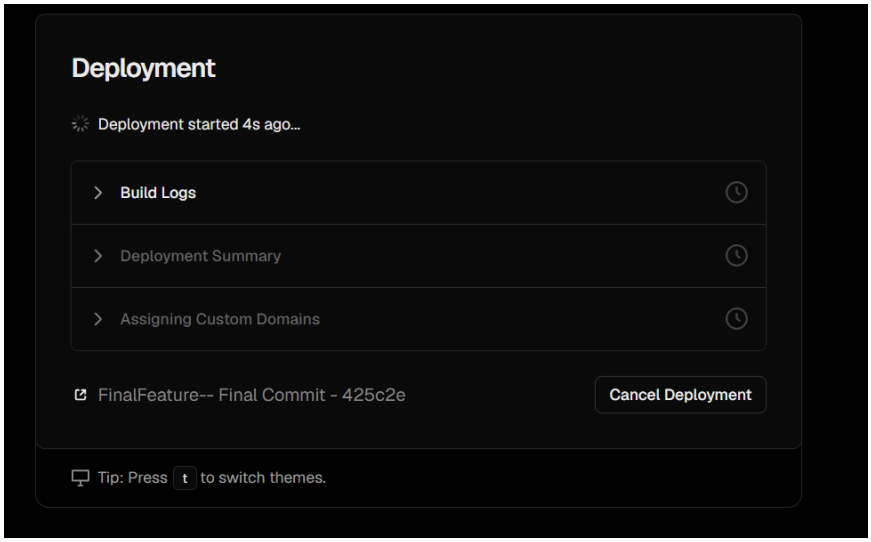


Figura 77 — Proceso de despliegue del backend

Al igual que ocurre con Vercel, Railway permite enlazar repositorios remotos y realizar despliegues automáticos. Una vez completado el proceso, el backend quedó disponible y conectado con la base de datos desplegada previamente.

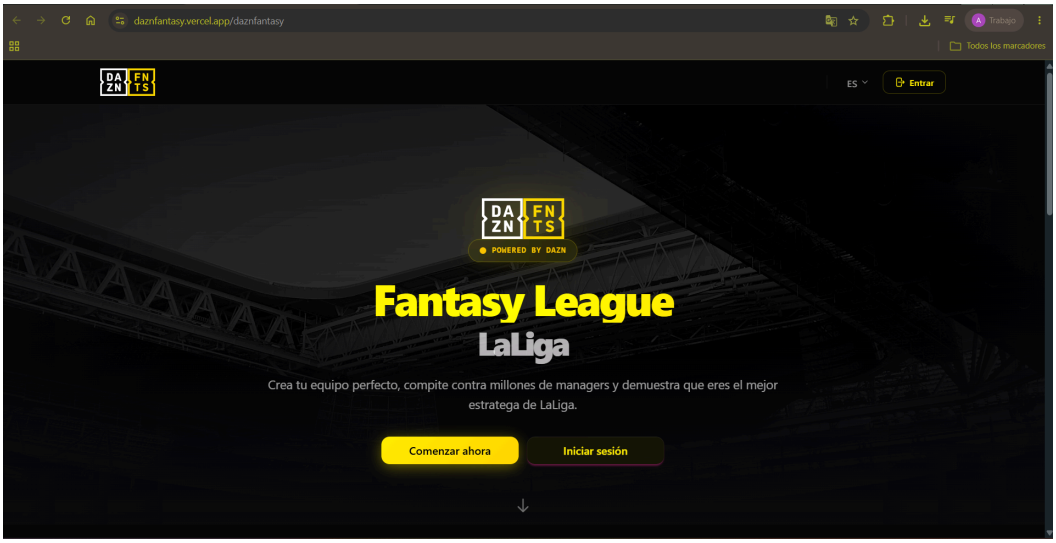


Figura 78 — Aplicación desplegada y accesible desde Internet

Tras finalizar la configuración de todos los servicios, la aplicación quedó accesible mediante la dirección proporcionada por la plataforma de despliegue.

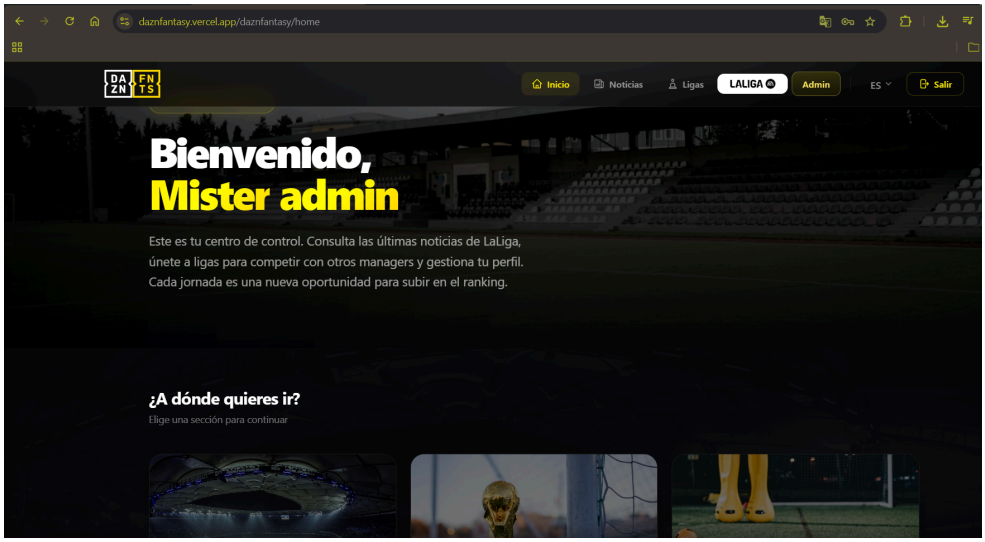


Figura 79 — Acceso autenticado a la aplicación

La imagen anterior demuestra el correcto funcionamiento de todo el sistema una vez desplegado. En ella podemos observar a un usuario autenticado utilizando la aplicación. Esto implica que frontend, backend, base de datos y sistema de autenticación se encuentran comunicándose correctamente entre sí.

Con este último paso podemos dar por finalizado el desarrollo del proyecto. La aplicación ha sido desplegada satisfactoriamente y se encuentra preparada para ser utilizada en un entorno real, permitiendo validar de forma práctica todas las funcionalidades desarrolladas a lo largo de este trabajo.

13. FUTURAS ITERACIONES.

12.1 Próximas Iteraciones y Escalabilidad del Sistema

Para garantizar el crecimiento sostenido de la plataforma **DAZN Fantasy League**, se podrían tener en cuenta una serie de mejoras de nuestro sistema, lo que lo harían mas robusto, estable y eficaz.

1. Capa de Caché con Redis

Implementando **Redis** gran parte de nuestra base de datos viviría en la memoria RAM, mucho más rápida de acceder, liberando así a nuestra BD de operaciones de **lectura-escritura** innecesarias. (Redis Software, 2026) .

2. Seguridad a nivel de BD (PostgreSQL)

Mejora en la estructura de nuestra base de datos implementando la **diferenciación por schemas** y la **row-level-security** lo que impediría el acceso a datos a usuarios no autorizados. (The PostgreSQL Global Development Group, 2026) .

3. Automatización con Cron-Jobs

Algunas tareas de nuestro sistema son realizadas **manualmente** por el administrador. La implementación de tareas automáticas mediante **cron-jobs** convertiría a nuestro administrador en un supervisor, acelerando mucho la ejecución de operaciones repetitivas.

4. Optimización del Frontend

Operaciones para mejorar el **rendimiento y respuesta** de nuestra interfaz reduciendo peso y tamaño de algunos recursos visuales para una velocidad óptima. Estos puntos son clave para mejorar la interacción con nuestra plataforma.

13 Conclusiones

Gracias al desarrollo de este proyecto hemos podido poner en práctica todos los conocimientos adquiridos durante nuestro tiempo de estudio. A raíz de todo este esfuerzo se han adquirido principios sólidos, criterios de diseño y desarrollo y la capacidad de enfrentarnos a los retos que supone la creación desde cero de una aplicación.

En gran medida hemos cumplido con los objetivos que nos planteamos al iniciar este proyecto. Aún así, a lo largo de todo el proceso, se han producido cambios notables. Algunas ideas se han mejorado, otras se han mantenido íntegras desde el comienzo y, como se espera, algunas han desaparecido por completo ya que el aprendizaje de nuevas tecnologías dejaba otras de lado. Si bien es cierto que todavía se podrían añadir muchas funcionalidades e integrar una gran diversidad de tecnologías, estamos satisfechos con el resultado obtenido.

Durante todo el desarrollo hemos tenido que asumir errores, resolver dificultades y adquirir nuevos conocimientos para conseguir el resultado final. Hemos tenidos que formarnos como equipo, aprender las metodologías que son el día a día de los profesionales y, lo más importante, trabajar como uno. Cada uno de nosotros se ha hecho cargo de sus funciones, sin descuidar la atención al resto ya que, en definitiva, como cada una de las capas de nuestro proyecto, hemos sido parte de un conjunto mayor.

Finalmente, todo el contenido de este texto, que nos ha servido para repasar todo lo hecho y ponerlo al alcance de los usuarios. Hemos ido cubriendo cada acción y decisión que hemos tomado, desde el modelado de los datos, su presentación y, en última instancia, su seguridad e integridad, sin olvidarnos por supuesto, de la documentación y comunicación del mismo. Se podrían añadir muchos más elementos que harían crecer este proyecto, pero como ya se dijo en el capitulo anterior, eso queda para futuras iteraciones.

Edición y Maquetación

La presente documentación ha sido redactada íntegramente mediante Markdown y maquetada utilizando Typst (Typst, 2026) . Typst es una herramienta profesional para la edición y maquetación de documentos de carácter oficial y corporativo, reutilizable y programable, el cual nos permite respetar los estándares APA dándoles un estilo personal y adecuado a nuestra visión de proyecto.

Bibliografía

A. G. Martins. (s. f.). *Método SCRUM: Ventajas y Desventajas*. https://www.ofiprix.com/es/blog/ventajas-desventajas-usar-metodo-scrum

AcademiaLab. (2024,). *Bcrypt*. https://academia-lab.com/enciclopedia/bcrypt/

AEC Consultoras. (2024,). *Informe del sector tecnológico en España*. https://aecconsultoras.com/informe/informe-la-consultoria-espanola-el-sector-en-cifras-2024/

Agencia Tributaria. (2026,). *Amortización Simplificada*. https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-videos-folletos/manuales-practicos/folleto--actividades-economicas/3-impuesto-sobre-renta-personas-fisicas/3_5-estimacion--directa-simplificada/3_5_4-tabla-amortizacion-simplificada.html

As-Biwenger. (s. f.). *As-Biwenger*. https://biwenger.as.com/?lang=es

Asepeyo. (2025,). *Bases y tipos de cotización 2025*. https://www.asepeyo.es/duda/bases-y--tipos-de-cotizacion/

ASEST. (s. f.). *Estructura organizacional de una startup tecnológica*. https://asest.es/story/estructura-organizacional-startup-tecnologica/

Baeldung. (2024, septiembre 3). *Salt and Pepper in Password. Security*. https://www.baeldung.com/cs/password-salt-pepper

CIC Consulting Informático. (s. f.). *CIC*. https://www.cic.es/

Comunio. (s. f.). *Comunio*. https://www.comunio.es/

Echometer. (2025,). *Las 20 estadísticas de Scrum más importantes*. https://echometerapp.com/es/estadisticas-del-scrum/

Evenbytes. (s. f.). *Evenbytes*. https://evenbytes.com/es/

Firebase. (s. f.). *Costos y Hosting de Apps*. https://firebase.google.com/docs/app-hosting/costs?hl=es-419

Fundación VASS. (2024,). *Facturación de las empresas de servicios digitales*. https://www.fundacionvass.org/tic-monitor-septiembre-2024/

Futmondo. (s. f.). *Futmondo*. https://www.futmondo.com/

Global Market Insights. (2023,). *Fantasy Sports Market*. https://www.gminsights.com/es/industry-analysis/fantasy-sports-market

Google Developers. (2026, mayo 28). *Lighthouse: Optimize your web apps*. https://developer.chrome.com/docs/lighthouse

habitissimo. (s. f.). *Presupuestos de mantenimiento de edificio*. https://www.habitissimo.es/presupuestos/mantenimiento-edificio

Iconify. (s. f.). *Iconify*. https://iconify.design/

Indeed. (s. f.-a). *Salario de Junior QA en España*. https://es.indeed.com/career/junior-qa/salaries?from=top_sb

Indeed. (s. f.-b). *Salario medio de un DevOps en España*. https://es.indeed.com/career/devops/salaries?from=top_sb

Indeed. (s. f.-c). *Salario medio Técnico de Sistemas en Cantabria*. https://es.indeed.com/career/t%C3%A9cnico-de-sistemas/salaries/Cantabria-provincia?from=top_sb

Inetum. (s. f.). *Inetum*. http://www.inetum.com/es

INMUNE. (2023,). *5 beneficios de trabajar con metodología Scrum*. https://immune.institute/blog/metodologia-scrum-caracteristicas/

Jones, M., Bradley, J., & Sakimura, N. (2015,). *JSON Web Token (JWT)*. https://www.rfc-editor.org/rfc/rfc7519

Jooble. (s. f.-a). *Salario Medio de un Junior Developer en España*. https://es.jooble.org/salary/developer-junior/Cantabria#yearly

Jooble. (s. f.-c). *Salario Medio de un Scrum Master en Cantabria*. https://es.jooble.org/salary/scrum-master/Cantabria#yearly

Jooble. (s. f.-b). *Salario Medio de un Senior Developer en Cantabria*. https://es.jooble.org/salary/desarrollador-senior/Cantabria#yearly

Lenovo. (s. f.). *ThinkPad L14 Gen 6*. https://www.lenovo.com/es/es/p/laptops/thinkpad/thinkpadl/lenovo-thinkpad-l14-gen-6-14-inch-amd/len101t0131#tech_specs

OpenAPI Initiative. (2023,). *OpenAPI Specification (Swagger)*. https://swagger.io/specification/

Plain Concepts. (s. f.). *Plain Concepts*. https://www.plainconcepts.com/

The PostgreSQL Global Development Group. (2026, mayo 28). *PostgreSQL Documentation: Row-Level Security Policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

Redis Software. (2026, mayo 28). *Redis Documentation*. https://redis.io/docs/

Seguridad Social Española. (2025,). *Tipos de Cotización*. https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/10721/10957/9932/4315

Simple Icons. (s. f.). *Simple Icons*. https://simpleicons.org/

Stats Perform. (s. f.). *Stats Perform*. https://www.statsperform.com/

Talent.com. (s. f.). *Salario medio de un Diseñador Web en España*. https://es.talent.com/salary?job=dise%C3%B1ador+web

Typst. (2026,). *Typst. The new foundation for documents*. https://typst.app/

UNIR. (2025,). *Metodología Scrum: qué es, ventajas y cómo aplicarla*. https://www.unir.net/revista/ingenieria/metodologia-scrum/

Vaadin. (2026, febrero 18). *10 Best Angular alternatives in 2026. Compared*. https://vaadin.com/blog/best-angular-alternatives-for-web-development